

LLMWiki

A Semantic Operating Layer for Local Knowledge

Architecture Proposal — Internal Draft

Part I – IV, Chapters 1 – 12

Jeoung Mingyu
jmkjmk1023@gmail.com



July 29, 2026

Version 0.1.0-draft

Mission Statement:

LLMWiki is not a Retrieval-Augmented Generation framework. It is a semantic operating layer that continuously transforms heterogeneous digital artifacts into an evidence-aware knowledge space.

CONFIDENTIAL — INTERNAL TECHNICAL DRAFT

Copyright © 2026 Jeoung Mingyu. All rights reserved.

Architecture Status: Proposed Implementation Status: Partial

Evaluation Status: Planned

Patent Status: Unfiled / Confidential — Not for public distribution.

Project Governance

Role	Name	Contact
Chief System Architect	Jeoung Mingyu	jmkjmk1023@gmail.com
Lead Researcher	Jeoung Mingyu	jmkjmk1023@gmail.com
Patent Attorney	TBD	TBD

This document is a living technical specification.
All design decisions are recorded in DESIGN_DECISIONS.md.
Never contradict prior architectural decisions without explicit review.

Contents

I	Foundation and Architecture	1
1	Architecture	2
1.1	Design Philosophy	2
1.2	Three-Layer Architecture	3
1.2.1	Layer 1: Ingestion Pipeline	3
1.2.2	Layer 2: Core Knowledge Space	3
1.2.3	Layer 3: Query and Execution	4
1.3	Filesystem as Event Log, Projections as Derived State	4
1.3.1	Directory Layout	4
1.4	Data Flow Architecture	5
1.4.1	Ingestion Pipeline	6
1.4.2	Query Pipeline	6
1.5	External Conversation Sources	6
1.6	Evidence Context Boundary	7
1.7	Crash Consistency and Recovery	8
1.7.1	Event Lifecycle	8
1.7.2	Idempotent, Resumable Projection	8
1.7.3	Crash Recovery Scanner	8
1.7.4	Atomic Commit: POSIX and Windows	9
1.7.5	What Live-Copy Backups Do Not Give You	9
1.8	Core Invariants	10
1.9	The Filesystem as Lineage Recorder	10
1.10	Comparison with Prior Art	11
1.11	Patent-Relevant Novelty	11
1.12	Module Interface Contracts	13
1.13	Runtime Configuration	14
1.14	Extension Points	15
1.15	Summary	15
2	Semantic Parsing	17
2.1	Design Goals	17
2.2	Parser Architecture	17
2.2.1	Parser Trait	17
2.2.2	Parser Registry	18
2.2.3	Office and HWP Format Coverage	18
2.3	Semantic Unit Model	19

2.3.1	Unit Type Taxonomy	20
2.3.2	Metadata Schema	20
2.3.3	Canonicalization	21
2.4	Language-Specific Parsers	21
2.4.1	Tree-sitter Based Parsers (Code)	21
2.4.1.1	Supported Languages	22
2.4.2	Markdown Parser	22
2.4.3	PDF Parser	22
2.4.4	OpenAPI / AsyncAPI Parser	23
2.4.5	SQL DDL Parser	23
2.4.6	Structured Data Parser (YAML/JSON/TOML)	23
2.5	Structure-Aware Chunking	23
2.5.1	Hierarchical Chunking Algorithm	24
2.5.2	Sliding Window (Fallback)	24
2.5.3	Semantic Chunking (Experimental)	24
2.5.4	Scale-Free Chunking by Graph Renormalization	25
2.6	Entity and Relation Extraction	26
2.6.1	Rule-Based Extraction	26
2.6.2	LLM-Assisted Extraction (Optional)	26
2.6.3	Entity Resolution	27
2.7	Incremental Parsing	27
2.8	Embedding Generation	28
2.8.1	Supported Models	28
2.8.2	Batch Inference	28
2.9	Parser Extensibility Guide	28
2.10	Summary	29
3	Knowledge Graph & Semantic Index	30
3.1	Knowledge Graph	30
3.1.1	Data Model	30
3.1.2	Graph Store Backend	31
3.1.2.1	Storage Layout (RocksDB)	31
3.1.3	Entity Resolution	32
3.1.4	Incremental Graph Updates	32
3.1.5	Graph Traversal Primitives	33
3.2	Semantic Index	33
3.2.1	HNSW Vector Index	33
3.2.2	BM25 Keyword Index	34
3.2.3	Graph Adjacency Index	34
3.2.4	Hybrid Search	34
3.2.5	Index Update Strategy	35
3.3	Query-Time Graph Expansion	35
3.4	Performance Characteristics	35
3.5	Patent-Relevant Novelty	35
3.6	Summary	36

4	Retrieval and Evidence Gate	37
4.1	Evidence Gate: Core Principle	37
4.2	Why Not Reranking?	38
4.3	Retrieval Pipeline	38
4.3.1	Stage 1: Query Embedding	38
4.3.2	Stage 2: Hybrid Search	39
4.3.3	Stage 3: Evidence Gate	39
4.3.4	Multi-Stage Proofreading Cascade	39
4.3.5	Stage 5: Evidence Context Selection and WC/1 Transport	41
4.3.6	Stage 6: Synthesis, Then Claim Support and Coverage Verification	42
4.4	Threshold Calibration: Supervised Score Calibration	42
4.4.1	Calibration Methods	43
4.4.2	Empirical Null Thresholding: Negative-Only FAR Estimation	43
4.4.3	Confidence Intervals: Coverage Calibration	45
4.5	Global Consistency of Evidence (Experimental Research Hypothesis)	45
4.5.1	What This Construction Does Not Yet Specify	47
4.6	Query Planner Integration	48
4.7	Evidence Gate Config	48
4.8	Evaluation Metrics	48
4.9	Comparison with Prior Art	48
4.10	Patent-Relevant Novelty	49
4.11	Summary	50
5	Agent Runtime	51
5.1	Design Goals	51
5.2	Runtime Architecture	51
5.2.1	Core Components	52
5.3	Plan Execution Model	52
5.3.1	Query Plan Structure	52
5.3.2	Execution Algorithm	53
5.3.3	Step Kinds	53
5.4	Evidence Context Assembly	53
5.5	Hierarchical Conversation Memory	54
5.6	Tool System	54
5.6.1	Tool Trait	54
5.6.2	Built-in Tools	55
5.6.3	Custom Tools	55
5.7	LLM Client Abstraction	55
5.8	State Management	56
5.9	Budget as Allocation, Not Truncation	56
5.10	Streaming Protocol	57
5.11	Error Handling and Retries	58
5.12	Conversation Management	58
5.13	Patent-Relevant Novelty	58
5.14	Summary	59

6	Evaluation Framework	60
6.1	Evaluation Philosophy	60
6.2	Benchmark Suite	60
6.2.1	Internal Benchmarks	60
6.2.2	External Benchmarks (Adapted)	61
6.3	Metric Definitions	61
6.3.1	Retrieval Metrics	61
6.3.2	Answer Quality Metrics	62
6.3.3	System Property Metrics	62
6.3.4	Context Efficiency Metrics	62
6.4	Evaluation Pipeline	63
6.4.1	Context-Shaping Acceptance Experiment	63
6.4.2	Configuration Matrix	63
6.4.3	Statistical Rigor	64
6.5	Regression Detection	64
6.6	Qualitative Evaluation	65
6.7	Patent-Relevant Novelty	65
6.8	Summary	65
7	Deployment and Operations	67
7.1	Deployment Tiers	67
7.2	Deployment Modes	68
7.3	Container Images	68
7.3.1	Image Variants	68
7.4	Kubernetes Deployment [Enterprise Cluster tier]	69
7.4.1	Helm Chart Structure	69
7.4.2	Key Configuration	69
7.4.3	Stateful Considerations	70
7.5	Local-First Deployment	70
7.5.1	Single Binary	70
7.5.2	Docker Compose (Local Stack)	71
7.5.3	Air-Gapped Installation	71
7.6	Configuration Management	72
7.7	Monitoring and Observability	72
7.7.1	Metrics (Prometheus)	72
7.7.2	Distributed Tracing (OpenTelemetry)	73
7.7.3	Structured Logging	73
7.7.4	Health Checks	73
7.8	Backup and Recovery	73
7.8.1	Backup Strategy	73
7.8.2	Automated Backup (CronJob)	74
7.8.3	Disaster Recovery	74
7.9	Upgrades and Migrations	74
7.9.1	Version Compatibility	74
7.9.2	Index Migration	75

7.9.3	Graph Migration	75
7.10	Discord Team-Memory Adapter	75
7.11	Security Hardening	76
7.12	Performance Tuning	76
7.13	Patent-Relevant Novelty	76
7.14	Summary	77
8	Mutation and Approval Engine	78
8.1	Why This Is a Fourth Layer, Not a Tool Call	78
8.2	Pipeline	78
8.3	Mutation Events Are WAL Events	79
8.4	Interfaces	79
8.5	Approval Policy	81
8.6	Scheduled and Recurring Mutation	81
8.7	Format Coverage	81
II	Reference and Extension	83
9	API Reference	84
9.1	API Overview	84
9.2	Authentication	84
9.2.1	Bearer Token (OIDC)	84
9.2.2	API Key	84
9.2.3	Anonymous (Development)	85
9.3	Core Endpoints	85
9.3.1	Health	85
9.3.2	Query	85
9.3.3	Ingestion	86
9.3.4	Repository Management	87
9.3.5	Graph Operations	88
9.3.6	Index Operations	88
9.3.7	Unit Access	88
9.3.8	Evidence Gate	88
9.3.9	Discord Source Operations	89
9.3.10	Administration	89
9.4	WebSocket API	90
9.5	gRPC API	90
9.6	Error Handling	90
9.7	Rate Limiting	91
9.8	SDKs	91
9.9	OpenAPI Specification	92
9.10	API Versioning Policy	92
9.11	Patent-Relevant Novelty	92
9.12	Summary	93

10 Developer Guide	94
10.1 Development Environment	94
10.1.1 Prerequisites	94
10.1.2 Repository Structure	94
10.1.3 Build and Test	95
10.2 Extension Points Deep Dive	95
10.2.1 Custom Parser	95
10.2.2 Custom Chunker	96
10.2.3 Custom Embedder	97
10.2.4 Custom Graph Store	98
10.2.5 Custom Index Backend	98
10.2.6 Custom Evidence Verifier	98
10.2.7 Custom Query Planner Strategy	99
10.2.8 Custom Agent Tool	99
10.2.9 Custom Auth Provider	100
10.3 Testing Extensions	100
10.3.1 Parser Tests	100
10.3.2 Integration Tests	101
10.3.3 Property-Based Testing	101
10.4 Debugging and Profiling	102
10.4.1 Logging	102
10.4.2 Profiling	102
10.5 Contributing	102
10.5.1 Code Style	102
10.5.2 PR Process	102
10.5.3 Adding a New Volume	103
10.6 Release Process	103
10.7 Summary	103
 III Intellectual Property and Research	 104
11 Patent Draft	105
11.1 Field of the Invention	105
11.2 Background	105
11.3 Summary of the Invention	105
11.4 Independent Claims	105
11.5 Dependent Claims	109
11.6 Design Decisions Supporting Novelty	110
11.7 Specification Support	110
11.8 Prior Art Research Notes (Informal)	111
11.9 Potential Design-Arounds and Defense	112
 12 Research Paper: LLMWiki - A Semantic Operating Layer for Evidence-Aware Knowledge Management	 114
12.1 Introduction	114

12.2	Related Work	115
12.2.1	Retrieval-Augmented Generation	116
12.2.2	Graph-Enhanced RAG	116
12.2.3	Evidence and Attribution	116
12.2.4	Prompt Compression and Context Selection	116
12.2.5	Long-Dialogue and External Memory	116
12.2.6	Local and Deterministic Systems	117
12.2.7	Methods Imported from Adjacent Sciences	117
12.3	System Architecture	117
12.3.1	Three-Layer Decomposition	117
12.3.2	Filesystem as Write-Ahead Log	118
12.3.3	Content Addressing	118
12.3.4	Evidence Gate	118
12.3.5	DAG Query Planner	118
12.3.6	Evidence Context Boundary	118
12.3.7	Label-Free Significance via Discordant Pairing	118
12.3.8	Proofreading Cascade	119
12.3.9	Cohomological Evidence Closure (Research Hypothesis)	119
12.4	Evaluation	120
12.4.1	Benchmarks	120
12.4.2	Baselines	120
12.4.3	Metrics	120
12.4.4	Performance Targets (Preliminary, Unverified)	121
12.4.5	Ablation Studies (Preliminary, Unverified)	122
12.4.6	Calibration Analysis (Planned)	122
12.5	Discussion	122
12.5.1	Why Evidence Gate Works	122
12.5.2	Local-First Implications	123
12.5.3	Limitations	123
12.6	Conclusion	123
IV	Appendices	125
.1	Mathematical Notation	126
.2	Complete Configuration Schema	126
.3	API Schemas	128
.3.1	Query Request	128
.3.2	Query Response	129
.4	Evaluation Protocol	130
.4.1	Dataset Construction	130
.4.2	Metrics Computation	130
.4.3	Statistical Rigor	131
.5	Glossary of Key Terms	131
A	Data Model Definitions	138
A.1	Content Identity vs. Occurrence Identity	138

A.2	SemanticUnit	138
A.3	UnitOccurrence	139
A.4	ArtifactVersion	139
A.5	Chunk	140
A.6	Entity	140
A.7	Relation	141
A.8	QueryPlan	141
A.9	ToolSchema	142
B	Configuration Reference	144
B.1	Full config.toml	144
B.2	Environment Variable Overrides	145
C	Parser Grammar Specifications	146
C.1	Python (tree-sitter-python)	146
C.2	JavaScript/TypeScript	146
C.3	Rust	147
C.4	Go	147
C.5	SQL	147
C.6	Design Decision Log	149
D	Extended Related Work	151
D.1	Dense Retrieval	151
D.2	Sparse Retrieval	151
D.3	Hybrid Retrieval	151
D.4	Graph-Enhanced Retrieval	151
D.5	Recursive/Tree-Based Retrieval	151
D.6	Evidence and Attribution	151
D.7	Local/Private LLMs	151
D.8	Vector Databases	152
D.9	Graph Databases	152
D.10	Parsing and Embedding	152

List of Figures

1.1	Three-Layer Architecture of LLMWiki. Filesystem acts as write-ahead log (WAL) for all layers.	3
1.2	End-to-end data flow: ingestion pipeline (top) and query pipeline (bottom).	5
2.1	Parser pipeline: detection, routing, parsing, and unit emission.	18
4.1	Retrieval + Evidence Gate pipeline.	38
5.1	Agent Runtime components.	51
6.1	Automated evaluation pipeline.	64
12.1	Schematic illustration (not measured data) of the intended effect of isotonic calibration on cross-encoder reliability. Curves are hypothetical examples of the qualitative shape expected, not observed calibration error.	123

List of Tables

1.1	LLMWiki vs. Prior Art: Architectural Positioning, by Mechanism	12
1.2	LLMWiki Extension Points	15
2.1	Core Semantic Unit Types by Parser Domain	20
2.2	Tree-sitter Languages Supported (v0.1)	22
2.3	Local Embedding Models (Recommended)	28
3.1	Graph Store Backend Comparison	31
3.2	Expected Performance (Single Node, SSD)	36
4.1	Reranker vs. Evidence Gate	38
4.2	Evidence Gate Evaluation Metrics	49
4.3	Retrieval + Verification in Prior Art	49
5.1	Built-in Tools	55
6.1	LLMWiki Internal Benchmarks	61
7.1	Deployment Tiers: Cumulative Complexity, Independent Sufficiency	67
7.2	LLMWiki Deployment Modes	68
7.3	Tuning Knobs by Workload	76
9.1	Error Codes	91
9.2	Official SDKs	92
11.1	Design Decisions Supporting Novelty	110
12.1	Performance Targets on LLMWikiBench — design goals from preliminary internal runs, NOT a published or reproducible evaluation. LLMWikiBench and the evaluation harness have not been released; no confidence intervals are reported because no statistically controlled evaluation has been run.	121
12.2	Projected Component Contribution (Evidence F1@10) — design estimates from preliminary internal runs, not a controlled ablation study	122
3	Notation Summary	126
C.1	Key Design Decisions	150

List of Algorithms

1	Ingestion Pipeline (event-sourced)	6
2	Query Execution Pipeline	6
3	Unit Canonicalization	21
4	Hierarchical Chunking	24
5	Semantic Chunking via Embedding Similarity	25
6	Entity Resolution	27
7	Incremental Unit Diff	28
8	Entity Resolution	32
9	Incremental Graph Update	32
10	Hybrid Search	35
11	Evidence Gate	39
12	Calibration Procedure	43
13	False Alarm Rate Estimation without Relevance Labels	44
14	Consistency Gate	46
15	Plan Execution	53
16	Budget-Aware Step Scheduling	57
17	Evidence Recall@k	130

Acronyms

Part I

Foundation and Architecture

Architecture

LLMWiki is not a Retrieval-Augmented Generation (RAG) framework. It is a **semantic operating layer** that continuously transforms heterogeneous digital artifacts into an evidence-aware knowledge space. This volume establishes the architectural foundations: the three-layer decomposition, the filesystem-as-WAL principle, the data flow invariants, and the module interface contracts that all subsequent volumes build upon.

1.1 Design Philosophy

Design Principle 1.1 (Source Artifacts and Event Log are Authoritative). Two things in LLMWiki are authoritative, and nothing else is. The user’s source artifacts (files, API responses, database rows) are the authoritative content. The append-only mutation/event log in `.llmwiki/wal/` is the authoritative history of every change derived from them. Everything else the system maintains—semantic units, the property knowledge graph, the vector index, the BM25 index—is a *projection*: a materialized, rebuildable view computed by replaying the event log. A projection can be deleted and regenerated from the log without loss; the log itself cannot.

Design Principle 1.2 (Graph as Canonical Query-Time Representation). Among the projections, the property knowledge graph is the canonical representation for query-time reasoning: it is what the query planner and evidence gate query first, and vector/keyword indices exist to accelerate or widen that graph-centric retrieval. “Canonical” here is a statement about which projection query execution treats as primary, not a claim about durability—durability is the event log’s job (previous principle), not the graph’s.

Design Principle 1.3 (Evidence Gate as First-Class Module). Retrieval results must pass a mandatory cross-encoder verification stage before synthesis. This is not a reranker; it is an evidence filter with calibrated confidence thresholds.

Design Principle 1.4 (Local-First, LLM-Optional). All critical-path operations (parsing, chunking, embedding, graph update, indexing, evidence verification) run locally with zero external API dependencies. LLMs are used only for planning, entity extraction (optional), and answer synthesis.

Design Principle 1.5 (Content Addressability). Every semantic unit is identified by SHA-256 of its canonical form. Identical content implies identical identifier across all indices, enabling deduplication, caching, and verifiable lineage.

Design Principle 1.6 (Incremental Consistency). After any ingestion, the system state (G, I, U) - graph, index, units - satisfies: a query executed against the live state produces the same result as replaying the ingestion log.

Design Principle 1.7 (Deterministic Replay). Given identical `.llmwiki/` directory and identical query `q`, two executions produce bitwise-identical results (modulo non-deterministic LLM sampling, which is confined to planning/synthesis).

1.2 Three-Layer Architecture

The system decomposes into three layers with strictly unidirectional data flow: Ingestion to Core to Query.

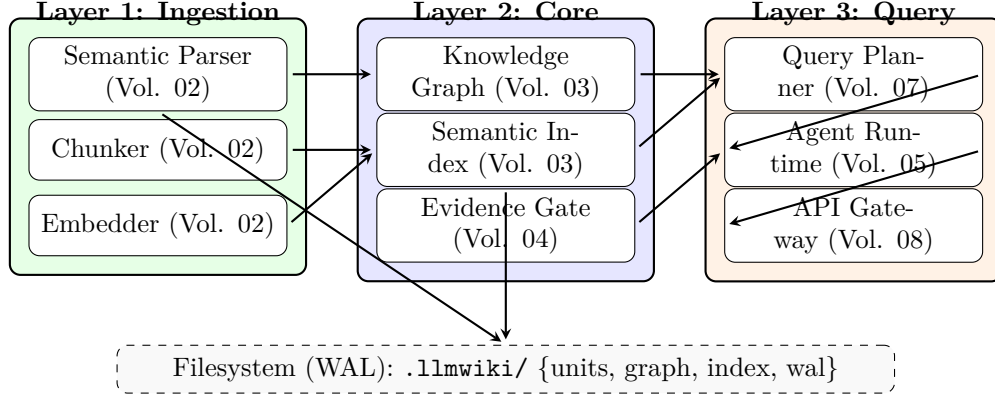


Figure 1.1: Three-Layer Architecture of LLMWiki. Filesystem acts as write-ahead log (WAL) for all layers.

1.2.1 Layer 1: Ingestion Pipeline

Responsible for parsing heterogeneous artifacts into semantic units - content-addressed, typed, and annotated fragments. Modules:

- Semantic Parser (Vol. 02): Language-aware parsing (code, markdown, PDF, OpenAPI, SQL DDL, etc.)
- Chunker: Hierarchical, structure-aware segmentation with overlap control
- Embedder: Local embedding models (BGE, E5, nomic-embed) with batch inference

1.2.2 Layer 2: Core Knowledge Space

The persistent, queryable knowledge representation. Three tightly coupled modules:

- Knowledge Graph (Vol. 03): Property graph with typed nodes/edges, entity resolution, and incremental updates
- Semantic Index (Vol. 03): Hybrid index (HNSW vector + BM25 keyword + graph adjacency)
- Evidence Gate (Vol. 06): Cross-encoder verification + citation extraction + confidence calibration

1.2.3 Layer 3: Query and Execution

Composable query execution with planning and observability:

- Query Planner (Vol. 07): DAG-based plan compilation, cost-based optimization, subquery decomposition
- Agent Runtime (Vol. 05): Tool execution, state management, streaming responses
- API Gateway (Vol. 08): REST/gRPC/WebSocket contracts, authentication, rate limiting

1.3 Filesystem as Event Log, Projections as Derived State

Design Decision 1.8. Decision: Use the local filesystem (directory structure `.llmwiki/`) to host an append-only event log as the sole durability mechanism, instead of a dedicated log service (e.g., Kafka, Redis Streams, WAL-G). The graph, vector, and keyword indices are projections computed from that log and carry no durability obligation of their own.

- Design Rationale 1.9.**
1. Zero operational dependency: No separate service to deploy, monitor, or backup.
 2. Deterministic replay: replaying `wal/` from empty projections reproduces graph/index state exactly (Algorithm 1, §1.7).
 3. Content-addressed deduplication: Units stored as `units/<hash>.json` are naturally deduplicated.
 4. Backup equals copy: `tar`, `rsync`, `borg`, `restic` work natively on the log; projections do not need to be backed up at all, only rebuilt.

- Design Trade-off 1.10.**
- Concurrency: Single-writer per repository; multi-writer requires external locking (file lock or SQLite WAL mode).
 - Scale: Directory entry count grows with unit count. Mitigation: sharded subdirectories (`units/ab/cd/ef...`).
 - Durability: `fsync` on commit; configurable `sync_on_write` flag.
 - Replay cost: rebuilding graph/vector/keyword projections from a long log is $O(\log \text{length})$, not $O(1)$; §1.7 amortizes this with periodic checkpoints.

1.3.1 Directory Layout

```
.llmwiki/
|-- wal/                                # AUTHORITATIVE: append-only event log
    ↳ (JSONL)
|   |-- 2024-01-15-001.jsonl           # Each event carries a monotonic sequence
    ↳ number
|   |-- ...
|-- units/                             # PROJECTION: content-addressed semantic
    ↳ units
|   |-- ab/cd/ef1234.json             # SHA-256 prefix sharding
|   |-- ...
```

```

|-- graph/                                # PROJECTION: knowledge graph, rebuildable
  ↳ from wal/
  | |-- nodes/                            # Node partitions (by type)
  | |-- edges/                            # Edge partitions (by relation type)
  | |-- indexes/                          # Graph-local indexes (adjacency, labels)
  | |-- _projection.meta                   # last_applied_sequence, projection schema
  ↳ version
|-- index/                                # PROJECTION: semantic search indexes,
  ↳ rebuildable from wal/
  | |-- hnsw/                             # HNSW vector index (disk-backed)
  | |-- bm25/                             # BM25 inverted index (tantivy)
  | |-- adjacency/                        # Graph adjacency for traversal
  | |-- _projection.meta                   # last_applied_sequence, projection schema
  ↳ version
|-- checkpoints/                          # Periodic full-projection snapshots (bound
  ↳ replay cost)
  | |-- 0000042000/                       # Snapshot at wal sequence 42000
|-- planner/                              # Query plan cache
  | |-- plans/
|-- evidence/                             # Evidence gate caches
  | |-- cross-encoder/
|-- config.toml                           # Repository configuration

```

Every entry under `graph/` and `index/` is disposable: deleting the directory and replaying `wal/` from the last `checkpoints/` snapshot regenerates it. Only `wal/` (and the original source artifacts, which live outside `.llmwiki/`) must never be treated as disposable.

1.4 Data Flow Architecture

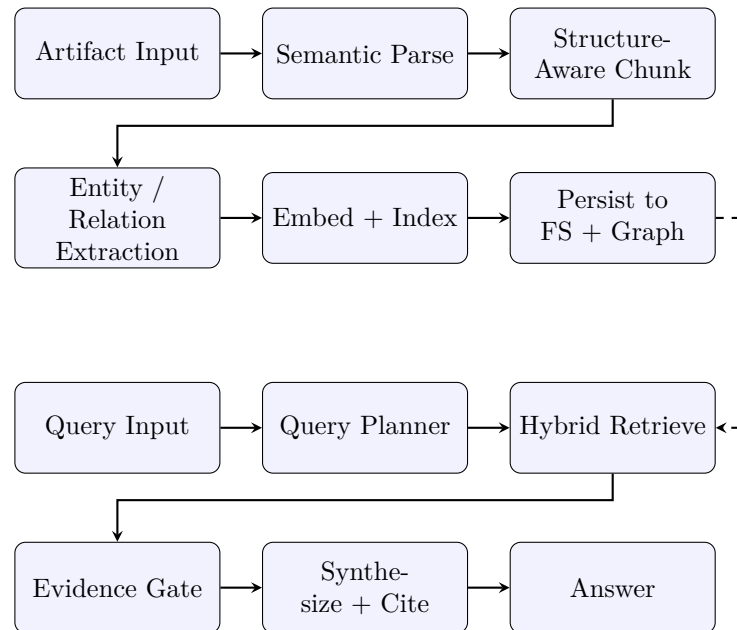


Figure 1.2: End-to-end data flow: ingestion pipeline (top) and query pipeline (bottom).

1.4.1 Ingestion Pipeline

Algorithm 1 Ingestion Pipeline (event-sourced)

Require: Artifact A (file, API response, database row)

Ensure: Event e durably committed at sequence n ; projections eventually consistent with n

- 1: $U = \text{Parse}(A)$ ▷ Language-aware parser
 - 2: $U = \text{Chunk}(U)$ ▷ Hierarchical, overlap-controlled
 - 3: $\Delta G = \text{ExtractEntitiesRelations}(U)$ ▷ Local LLM or rule-based
 - 4: $\Delta I = \text{Embed}(U)$ ▷ Batch local inference
 - 5: $e = \text{MakeEvent}(U, \Delta G, \Delta I)$ ▷ state = PREPARED
 - 6: $n = \text{AppendWAL}(e)$ ▷ Atomic durable append; state $\rightarrow$ COMMITTED (§1.7)
 - 7: $\text{ApplyGraph}(e, n)$ ▷ Idempotent; no-op if $\text{graph.last_applied_sequence} \geq n$
 - 8: $\text{ApplyIndex}(e, n)$ ▷ Idempotent; no-op if $\text{index.last_applied_sequence} \geq n$
 - 9: **return** $U, \Delta G, \Delta I$ ▷ Event state $\rightarrow$ PROJECTED once both projections advance past n
-

A query is answerable as soon as step 6 (COMMITTED) completes; it does not wait for step 9 (PROJECTED). Between those steps the event is durable but not yet reflected in graph/index reads, which is the ordinary read-after-write lag of an event-sourced system, not a correctness gap—§1.7 makes precise what a reader is guaranteed to see at each stage.

1.4.2 Query Pipeline

Algorithm 2 Query Execution Pipeline

Require: Query q , context budget B , evidence threshold τ

Ensure: Answer a with citations C

- 1: $P = \text{Plan}(q)$ ▷ Planner outputs DAG of subqueries
 - 2: $R = \emptyset$
 - 3: **for all** subquery s in P **do**
 - 4: $R_s = \text{HybridRetrieve}(s)$ ▷ Vector + BM25 + Graph
 - 5: $R = R \cup R_s$
 - 6: $E = \text{EvidenceGate}(R, \tau)$ ▷ Cross-encoder verify
 - 7: **if** $E = \emptyset$ **then**
 - 8: **return** $\text{Refuse}(\text{"insufficient evidence"})$
 - 9: $E_B = \text{SelectContext}(E, B)$ ▷ Top- k default; budgeted coverage experimental
 - 10: $(W, M) = \text{SerializeWCOne}(q, E_B)$ ▷ Verbatim evidence + host-side id map
 - 11: $a = \text{Synthesize}(W)$ ▷ LLM cites small integer handles
 - 12: $C = \text{ResolveCitations}(a, M)$ ▷ Handle $\rightarrow$ occurrence and source span
 - 13: $a' = \text{VerifyClaims}(a, C)$ ▷ Claim support + evidence coverage
 - 14: **return** a', C
-

1.5 External Conversation Sources

An external conversation system is ingested as an event source, not copied into an LLM prompt. For Discord, historical messages arrive through paginated REST reads and

live changes through Gateway dispatches [Discord, 2026b,a]. Both paths normalize to the same append-only contract:

Discord event $\rightarrow$ durable journal $\rightarrow$ {message state, Markdown, index, memory}.

Message updates and deletions are new events. The current channel view is a projection obtained by replay, and every model-visible message carries the stable citation `discord:<channel_id>:<message_id>`.

Design Principle 1.11 (Conversation Memory is a Projection). An LLM-generated summary may accelerate reading but is never authoritative. Raw events remain sufficient to rebuild current messages, reply relationships, attachment metadata, channel/thread Markdown, and the model-summary input.

The reference implementation under `integrations/discord_llmwiki/` uses an idempotent SQLite journal, atomic Markdown publication, REST backfill, an optional Gateway adapter, and a provider-neutral summary boundary. It is executable and tested, but it is not yet connected to the production Rust graph/index runtime specified in this volume.

1.6 Evidence Context Boundary

The evidence gate decides *which units are admissible*; it does not decide how the surviving set consumes the reader’s finite context. LLMWiki therefore places a narrow evidence-context boundary between verification and synthesis:

$$\mathcal{R} \xrightarrow{\text{time scope}} \mathcal{R}_t \xrightarrow{\mathcal{G}_\tau} \mathcal{E} \xrightarrow{\text{select under } B} \mathcal{E}_B \xrightarrow{\text{WC}/1} (W, M). \quad (1.1)$$

W is the model-visible prompt payload and M is a host-only map from small integer handles to `(content_id, occurrence_id, document, span)`. The order is invariant: selection never precedes verification and cannot turn an empty gate result into an answer.

Design Principle 1.12 (Compress the Envelope, Preserve the Evidence). Only repeated metadata and machine identifiers are removed from the model-visible envelope. Evidence text is transmitted verbatim. A transport optimisation may reduce token cost but may not weaken human auditability, citation resolution, or deterministic replay.

WC/1 transport. The line-oriented WC/1 contract groups blocks by source document, replaces UUIDs with response-local integer handles, and line-count-prefixes each block so arbitrary Markdown and source code remain unambiguous. The fixed contract instruction is query-independent and eligible for prompt-prefix caching. The reference implementation measures total-prompt reductions of 30.3% (English) and 32.8% (Korean) against the equivalent JSON fallback on cold prompts, including the fixed prefix; cached-prefix measurements are 31.7% and 36.6%. These are tokenizer measurements, not answer-quality results.

Budgeted selection. When verified evidence exceeds B , the production-safe baseline remains score-ordered top- k . A deterministic reference packer instead maximises weighted concept coverage per token with a singleton guard, following budgeted submodular selection [Lin and Bilmes, 2010, Peng et al., 2025, Bala, 2026]. Its exploratory fixture improves the proxy objective under tight budgets, but coverage is not answer quality; it cannot replace top- k until the $(\text{JSON}, \text{WC}/1) \times (\text{top-}k, \text{packed})$ model A/B in Vol. 06 succeeds.

Integration status. `eval/scripts/wire_contract.py` and `eval/scripts/evidence_packer.py` are executable reference components with deterministic tests. They are not yet called by the BM25-only `run_eval.py` scaffold or by a production runtime. Unknown contract versions and failed capability negotiation fall back to JSON at the host boundary rather than guessing.

1.7 Crash Consistency and Recovery

Algorithm 1 has three commit points (`AppendWAL`, `ApplyGraph`, `ApplyIndex`) that are not a single atomic transaction. A process can be killed between any two of them. This section specifies what a killed process leaves behind and how a subsequent process recovers it, so that “the WAL is authoritative” (design principle, §1.1) is a checkable property rather than an aspiration.

1.7.1 Event Lifecycle

Every event e carries a monotonically increasing sequence number n , assigned at append time, and passes through three states:

- **PREPARED:** e is constructed in memory; not yet durable. A crash here loses nothing observable—the event never existed from any reader’s perspective.
- **COMMITTED:** e is durably appended to `wal/` under sequence n . From this point, e is part of the authoritative history and *must* eventually be reflected in every projection, even if the process that wrote it never runs again.
- **PROJECTED:** every projection p (graph, HNSW, BM25) satisfies $p.\text{last_applied_sequence} \geq n$.

A reader asking “is my write visible” is asking whether the relevant projections have reached **PROJECTED** for that event’s sequence, not whether the event is **COMMITTED**—these are different questions, and the API layer (Vol. 08) reports both.

1.7.2 Idempotent, Resumable Projection

Each projection directory (`graph/`, `index/`) carries a `_projection.meta` file recording its own `last_applied_sequence` and a `schema/projection-logic` version. `ApplyGraph(e, n)` and `ApplyIndex(e, n)` first check this marker: if $n \leq \text{last_applied_sequence}$, the call is a no-op. This makes re-application safe after a crash, a redeploy, or an operator manually re-running `ApplyIndex` on a range of sequences—the operation is idempotent by construction, not by convention.

1.7.3 Crash Recovery Scanner

On startup, LLMWiki runs a recovery scan before serving queries:

1. Read `last_applied_sequence` from every projection’s `_projection.meta`.
2. Let $n_{\min}$ be the smallest such value across all projections (the least-caught-up projection).

3. Replay `wal/` events with sequence $> n_{\min}$ against every projection whose marker is behind. Because application is idempotent (§above), projections that were already ahead of $n_{\min}$ simply no-op on events they’ve already applied.
4. If `wal/` has been compacted past $n_{\min}$ (a checkpoint boundary), recovery starts from the nearest `checkpoints/` snapshot at or before $n_{\min}$ instead—this is the case Invariant 1.18 (§1.9) requires to be reported explicitly if the snapshot itself is missing or corrupt, rather than silently skipped.

This scanner is what makes the directory-layout claim in §1.3.1 (“every entry under `graph/` and `index/` is disposable”) literally true rather than aspirational: deleting a projection directory is equivalent to setting its `last_applied_sequence` to that of the nearest checkpoint, and the scanner rebuilds it on next startup.

1.7.4 Atomic Commit: POSIX and Windows

Step 6 of Algorithm 1 (AppendWAL) requires that a single event either becomes fully durable or leaves no trace—no torn writes. The mechanism differs by platform and neither reduces to a bare `rename(2)` call:

- **POSIX (Linux, macOS):** write the event to a temp file in the same directory as the target WAL segment, `fsync` the temp file, `rename(2)` it into place (atomic within a filesystem, per POSIX), then `fsync` the containing directory so the rename itself is durable across a power loss—the directory `fsync` is the step most implementations omit and is required for the atomicity guarantee to survive a crash, not just a process kill.
- **Windows (NTFS):** `MoveFileEx` with `MOVEFILE_REPLACE_EXISTING` provides the rename step, but NTFS does not guarantee the same crash-durability semantics as POSIX `rename` without an explicit flush; LLMWiki additionally calls `FlushFileBuffers` on the temp file before the move and, where available, opens the target directory handle to flush metadata after it. Folding’s local-first deployment target includes Windows, so this path is exercised in CI, not treated as a POSIX-only afterthought.
- In both cases, a segment is appended to, never modified in place; only the temp-file-then-move step is atomic, and it is used solely to publish a new immutable segment or checkpoint, never to edit an existing one.

1.7.5 What Live-Copy Backups Do Not Give You

Copying `.llmwiki/` with `rsync` or `git` while the process is running (as §1.3’s rationale claims for backups) is safe for `wal/` segments, because segments are immutable once published and the copy tool sees either the whole segment or none of it. It is *not* safe as a substitute for a consistent snapshot of `graph/` or `index/`: RocksDB, HNSW, and Tantivy each maintain multi-file on-disk structures that are only mutually consistent at their own checkpoint boundaries, and a naive directory copy taken mid-write can capture a torn cross-file state that neither the old nor the new version can open. LLMWiki therefore exposes a `snapshot()` API that requests a consistent checkpoint from each store (RocksDB checkpoint API, HNSW/Tantivy segment-boundary flush) and only then is the result eligible for `checkpoints/`; ordinary backup tools should target `wal/` plus the most recent `checkpoints/` entry, not a live copy of `graph/` or `index/`.

1.8 Core Invariants

Invariant 1.13 (Content Addressability). For all u in U : $\text{id}(u) = \text{SHA256}(\text{canonical}(u))$. Identical content implies identical identifier across all indices.

Invariant 1.14 (Evidence Closure). For all a in Answers : $\text{citations}(a)$ subset of U and for all c in $\text{citations}(a)$: $\text{verify}(c) \geq \tau$.

Invariant 1.15 (Incremental Consistency). For every projection $p \in \{G, I, U\}$ (graph, index, units) and every sequence n , once $p.\text{last_applied_sequence} \geq n$, querying p yields the same result as replaying `wal/` events $1..n$ into an empty projection (§1.7). This is a per-projection, per-sequence statement, not a claim that G , I , and U are mutually synchronized at every instant—they advance independently and the recovery scanner is what reconciles them after a crash.

Invariant 1.16 (Local-First Determinism). Given identical `.llmwiki/` directory and identical query q , two executions produce bitwise-identical results (modulo non-deterministic LLM sampling, which is confined to planning/synthesis).

1.9 The Filesystem as Lineage Recorder

Invariants 1.15 and 1.16 assert that history can be replayed. They do not say how history is *reconstructed* when the record is incomplete, and every long-lived deployment produces an incomplete record: WAL segments are compacted, old snapshots are pruned, repositories are cloned from a mid-life state. A replay guarantee that holds only over an unbroken log is a guarantee that expires.

Developmental biology has spent a decade on exactly this reconstruction problem. CRISPR lineage recorders accumulate irreversible edits at engineered target sites across cell divisions, so that the division history of an organism can be inferred from the barcodes of its terminal cells alone — the intermediate states are gone, and the tree is recovered from the endpoint. Recent surveys note that recorders which log a *deterministic temporal ordering* of mutations enable precise reconstruction of deep lineages [Kang et al., 2026], which is precisely the argument for an append-only WAL. More usefully, the field has formalised inference under a degraded record: Bayesian phylodynamic frameworks reconstruct lineage trees with calibrated uncertainty from barcode data with dependent target sites and missing observations [Zwaans et al., 2025].

Two failure modes from that literature transfer to LLMWiki without modification.

Homoplasy. An identical edit arising independently in two lineages falsely implies shared ancestry. Content addressability (Invariant 1.13) makes this endemic rather than occasional: two units produced by unrelated derivation paths that happen to canonicalise identically receive the same identifier *by design*. This is the deduplication property, and it is desirable for storage and indexing. It is also destructive of provenance, because it collapses distinct derivations into one node. Identity and ancestry are different relations and must not share a key.

Invariant 1.17 (Lineage Distinctness). Content identity does not imply derivation identity. For a unit u , $\text{id}(u)$ addresses content; the derivation of u is recorded separately as the set of WAL edges producing it. Provenance is therefore a directed acyclic graph over content identifiers, never a tree, and a query for the origin of u returns a set of derivations rather than a unique parent.

Dropout. A target site that fails to be read yields a missing character, not a wrong one, and treating the two alike corrupts the tree. The analogue is a compacted WAL segment: the derivation edge is unobserved, not absent. Recording compaction boundaries explicitly lets reconstruction distinguish *no derivation occurred* from *the derivation is unobservable*, which is the difference between a false claim and an honest one.

Invariant 1.18 (Replay Degradation is Explicit). Where the WAL has been compacted, replay returns a result annotated with the reconstruction’s uncertainty rather than an unqualified answer. A deployment that cannot verify a replay reports that it cannot, and never silently substitutes a plausible history.

This is the mechanism that makes the auditability claim survive contact with operations. A system that guarantees reproducibility only while nothing has been pruned has guaranteed reproducibility only in the laboratory; the useful statement is that reproducibility degrades in a declared, measurable way.

1.10 Comparison with Prior Art

This is a statement about which mechanisms each system’s published design uses, not a claim that the other systems are deficient on every row — GraphRAG and HippoRAG are graph-centric by design and the comparison above is meant to locate LLMWiki’s specific combination (event-sourced durability, mandatory multi-stage gating, and content/location identity separation) rather than to imply the others lack graphs or verification of any kind.

1.11 Patent-Relevant Novelty

Patent Claim 1.19. Claim 1: A semantic operating layer characterized by: (a) content-addressed semantic units stored in a filesystem-backed write-ahead log, (b) a property knowledge graph as the canonical knowledge representation, (c) a hybrid semantic index combining vector, keyword, and graph adjacency, (d) an evidence gate that verifies retrieval results via cross-encoder scoring before synthesis, and (e) a DAG-based query planner that decomposes queries into verifiable subqueries.

Patent Claim 1.20. Claim 2: The system of Claim 1 wherein the filesystem serves as the sole durability mechanism, enabling deterministic replay via directory synchronization without auxiliary transaction logs.

Patent Claim 1.21. Claim 3: The system of Claim 1 wherein the evidence gate operates as a mandatory pipeline stage, rejecting any retrieval result that fails cross-encoder verification against a calibrated threshold, thereby enforcing evidence closure.

Patent Discussion 1.22. Prior art distinctions:

- RAG (Lewis et al., 2020): Vector index is primary; no graph; no evidence gate; API-dependent.
- GraphRAG (Edge et al., 2024): Graph constructed via LLM; no content-addressed units; no filesystem WAL; no mandatory evidence gate.
- HippoRAG (Gutiérrez et al., 2024): Personalized PageRank over graph; no hybrid index; no deterministic replay.

Table 1.1: LLMWiki vs. Prior Art: Architectural Positioning, by Mechanism

Property	RAG (Lewis et al.)	GraphRAG	HippoRAG	LLMWiki
Graph role	No graph; dense index over chunks	LLM-constructed graph is the retrieval structure (community summarization)	LLM-extracted graph, but retrieval seeds from vector search into Personalized PageRank	Property graph built by parsing (not LLM extraction where avoidable); query planner treats it as canonical, vectors/BM25 accelerate it (§1.1)
Vector index role	Primary retrieval mechanism	Secondary; entity/community embeddings for graph construction	Seed step into PPR, not the terminal ranking	Projection: rebuildable from the event log, not authoritative (§1.3)
Durability / replay	Whatever the vector DB provides; not specified as a system property	Not specified as a system property	Not specified as a system property	Append-only event log is the sole durability layer; graph/index rebuild via replay (§1.7)
Evidence verification before synthesis	Optional reranking, not gating; top- k always returned	No mandatory verification stage	No mandatory verification stage	Mandatory multi-stage gate; can return zero candidates (§4.2, Vol. 04)
Local / API-optional	Reference implementation uses external embedding + generation APIs; not architecturally excluded	Reference implementation uses an LLM for graph construction and summarization	Reference implementation uses an LLM for graph extraction	Critical path (parsing, chunking, embedding, indexing, verification) runs with zero external API calls by design (§1.1)
Content identity	Not addressed by the base method	Not addressed by the base method	Not addressed by the base method	SHA-256 content addressing, with content identity kept separate from location identity (Appendix B §A.1)
Query decomposition	Single retrieval pass	Single retrieval pass over the constructed graph	Single PPR pass	DAG-based planner decomposes into subqueries with per-subquery budgets (§1.2.3)

- LightRAG (Guo et al., 2024): Dual-level retrieval; no evidence gate; no DAG planner.
- RAPTOR (Sarathi et al., 2024): Recursive summarization tree; no graph; no filesystem WAL.
- Zep/Graphiti (Rasmussen et al., 2025) [Rasmussen et al. \[2025\]](#): the closest prior art for temporal validity. Every edge carries a bitemporal interval (`valid_at/invalid_at`) and superseded facts are invalidated rather than deleted, so LLMWiki must *not* claim validity intervals or as-of querying as novel. Distinctions: entity extraction is LLM-based, so graph construction is not reproducible from identical input; there is no mandatory evidence gate and no refusal outcome; and ingestion cost scales with LLM tokens per episode rather than CPU.
- Glean (Meta, 2024) [Meta Open Source \[2024\]](#): revision-scoped incremental code indexing at monorepo scale (indexes the diff between a base and current revision in $O(\text{changes})$), parser-driven and therefore deterministic. This anticipates version-anchored code graphs; LLMWiki must not claim that either. Distinction: Glean is a code-intelligence index with no retrieval-refusal semantics — there is no notion of declining to answer for want of evidence.
- Kythe (Google) [Google](#): per-compilation-unit node/edge streams keyed by VName; same distinction as Glean.

Note on what remains after the above: validity intervals (Zep) and version-anchored deterministic code graphs (Glean, Kythe) are each established prior art, so neither is claimed here as novel in isolation. The element without an identified antecedent is the *conjunction*: an evidence gate whose refusal decision is itself scoped to a validity bound, so that “not answerable from the corpus as of t ” is a reproducible first-class outcome carrying the time bound that produced it. Glean and Kythe have no refusal semantics; Zep has no gate; the GraphRAG family has no temporal axis. That conjunction is exercised by executable tests in `eval/scripts/test_temporal_gate.py` rather than asserted here.

1.12 Module Interface Contracts

Interface 23 (Parser).

```
fn parse(&self, artifact: Artifact) -> Result<Vec<SemanticUnit>>;
fn supported_mimes(&self) -> Vec<MimeType>;
fn schema(&self) -> ParserSchema;
}
```

Interface 24 (Chunker).

```
fn chunk(&self, unit: SemanticUnit, config: ChunkConfig) ->
  ↪ Vec<Chunk>;
fn config_schema(&self) -> ChunkConfigSchema;
}
```

Interface 25 (Embedder).

```
fn embed_batch(&self, texts: &[String]) -> Result<Vec<Vec<f32>>>;
fn dimension(&self) -> usize;
fn model_id(&self) -> ModelId;
}
```

```

Interface 26 (GraphStore).
    fn upsert_nodes(&self, nodes: Vec<Node>) -> Result<()>;
    fn upsert_edges(&self, edges: Vec<Edge>) -> Result<()>;
    fn traverse(&self, start: NodeId, pattern: TraversalPattern) ->
        ↪ Result<Vec<Path>>;
    fn neighborhoods(&self, ids: &[NodeId], hops: u32) ->
        ↪ Result<Subgraph>;
}

Interface 27 (SemanticIndex).
    fn upsert(&self, units: Vec<IndexedUnit>) -> Result<()>;
    fn search_vector(&self, query: Vec<f32>, k: usize) ->
        ↪ Result<Vec<ScoredUnit>>;
    fn search_bm25(&self, query: &str, k: usize) ->
        ↪ Result<Vec<ScoredUnit>>;
    fn search_hybrid(&self, query: HybridQuery, k: usize) ->
        ↪ Result<Vec<ScoredUnit>>;
}

Interface 28 (EvaluationGate).
    fn verify(&self, query: &str, candidates: Vec<ScoredUnit>, threshold:
        ↪ f32)
        -> Result<Vec<VerifiedUnit>>;
    fn calibrate(&self, dataset: CalibrationSet) ->
        ↪ Result<CalibrationCurve>;
}

Interface 29 (QueryPlanner).
    fn plan(&self, query: Query, context: PlanContext) ->
        ↪ Result<QueryPlan>;
    fn optimize(&self, plan: QueryPlan, stats: IndexStats) ->
        ↪ Result<QueryPlan>;
    fn explain(&self, plan: QueryPlan) -> PlanExplanation;
}

```

1.13 Runtime Configuration

```

# .llmwiki/config.toml
[repository]
id = "llmwiki-main"
created_at = "2024-01-15T10:30:00Z"
version = "0.1.0"

[ingestion]
parsers = ["python", "markdown", "pdf", "openapi", "sql", "yaml", "json"]
chunker = { type = "ast-aware", max_tokens = 512, overlap = 50 }
embedder = { model = "bge-m3", batch_size = 32, device = "cpu" }

[graph]
store = "rocksdb"

```

```

entity_resolution = { enabled = true, threshold = 0.85 }
index_types = ["btree", "adjacency", "label"]

[index]
hnsf = { m = 16, ef_construction = 200, ef_search = 64 }
bm25 = { k1 = 1.2, b = 0.75 }
hybrid = { vector_weight = 0.6, keyword_weight = 0.3, graph_weight = 0.1 }

[evidence_gate]
model = "cross-encoder/ms-marco-MiniLM-L-6-v2"
threshold = 0.72
calibration = "isotonic"
batch_size = 16

[planner]
max_depth = 5
max_branching = 4
cost_model = "learned"
cache_ttl = "1h"

[api]
bind = "127.0.0.1:8080"
auth = "none"
rate_limit = 100

```

1.14 Extension Points

Table 1.2: LLMWiki Extension Points

Extension Point	Interface	Volume
Custom Parser	Parser	Vol. 02
Custom Chunker	Chunker	Vol. 02
Custom Embedder	Embedder	Vol. 02
Graph Store Backend	GraphStore	Vol. 03
Index Backend	SemanticIndex	Vol. 03
Evidence Verifier	EvidenceGate	Vol. 04
Context Selection Policy	EvidenceContextBuilder	Vol. 04/05
Query Planner Strategy	QueryPlanner	Vol. 07
Agent Tool	Tool	Vol. 05
Auth Provider	AuthProvider	Vol. 08

1.15 Summary

LLMWiki’s architecture is defined by six differentiating choices:

1. **Graph-first, vectors-second:** The property graph is the canonical query-time representation; vectors are derived projections for similarity search.

2. **Event log as durability layer:** Zero operational dependencies; graph/vector/key-word projections are rebuildable from the append-only log, which is the only thing that must never be lost.
3. **Content-addressed units:** SHA-256 identities enable deduplication, caching, and verifiable lineage.
4. **Evidence gate as mandatory stage:** No retrieval result reaches synthesis without cross-encoder verification.
5. **DAG query planner:** Queries decompose into verifiable subqueries with cost-based optimization.
6. **Evidence-preserving context boundary:** Verified evidence is selected under budget, transported without rewriting, and resolved back to exact source occurrences.

These choices are not incremental improvements over RAG - they represent a different architectural category: a semantic operating layer.

The subsequent volumes detail each module's algorithms, data structures, APIs, and implementation considerations.

Semantic Parsing

This volume specifies the *semantic parsing* layer—the bridge between raw artifacts (source code, documents, schemas, APIs) and the content-addressed semantic units that populate the knowledge graph and search indices. Unlike traditional document chunking, semantic parsing extracts *typed, structured, hierarchically organized* units with explicit semantic annotations.

2.1 Design Goals

Design Principle 2.1 (Language Awareness). Parsers understand the syntax and semantics of their target language (Python, SQL, OpenAPI, Markdown, etc.), producing units that reflect logical constructs (functions, classes, tables, sections) rather than arbitrary text spans.

Design Principle 2.2 (Structure Preservation). The hierarchical structure of the source artifact is preserved in the unit tree. A Python class contains method units; a Markdown document contains section units; an OpenAPI spec contains operation units.

Design Principle 2.3 (Content Addressability). Each semantic unit is identified by SHA-256 of its canonical representation. Identical logical constructs yield identical identifiers regardless of source file location.

Design Principle 2.4 (Incremental Re-parse). When an artifact changes, only affected units are re-parsed. The parser emits a diff of (added, modified, removed) unit IDs.

Design Principle 2.5 (Extensibility). New parsers are added via the `Parser` trait (Vol. 01, §1.23) without modifying core ingestion logic.

2.2 Parser Architecture

2.2.1 Parser Trait

Every parser implements the `Parser` interface (Vol. 01, §1.23):

```
trait Parser {  
  fn parse(&self, artifact: Artifact) -> Result<Vec<SemanticUnit>>;  
  fn supported_mimes(&self) -> Vec<MimeType>;  
  fn schema(&self) -> ParserSchema;  
}
```

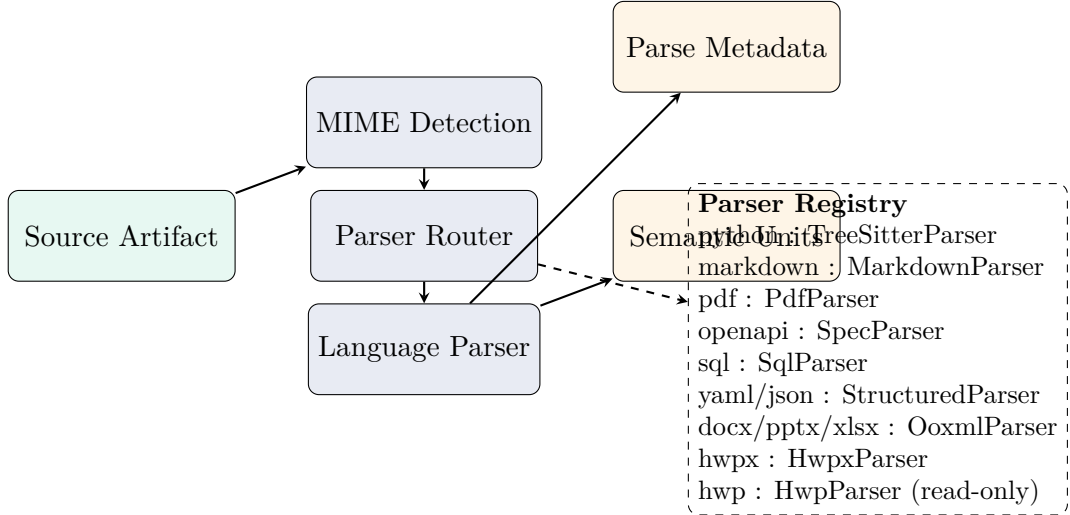


Figure 2.1: Parser pipeline: detection, routing, parsing, and unit emission.

Artifact encapsulates the raw bytes, path, MIME type, and optional metadata (encoding, language hints). **SemanticUnit** is the canonical output (defined below). **ParserSchema** describes the unit types this parser produces (e.g., `PythonFunction`, `SQLTable`, `MarkdownSection`).

2.2.2 Parser Registry

The registry maps MIME types (and file extensions as fallback) to parser instances. Parsers are loaded dynamically at startup from `config.toml` [`ingestion.parsers`].

```
# Config example
[ingestion.parsers]
python = { type = "tree-sitter", grammars = ["python"] }
markdown = { type = "markdown", extract_frontmatter = true }
pdf = { type = "pdf", ocr = false }
openapi = { type = "openapi", version = "3.1" }
sql = { type = "sql", dialect = "postgres" }
docx = { type = "ooxml", kind = "wordprocessing", track_changes =
  ↪ "as-metadata" }
pptx = { type = "ooxml", kind = "presentation" }
xlsx = { type = "ooxml", kind = "spreadsheet" }
hwp = { type = "hwp" }
hwp = { type = "hwp", mode = "read-only" } # see Sec.
  ↪ parsing:office-formats
```

2.2.3 Office and HWP Format Coverage

DOCX, PPTX, XLSX, and HWPX are all XML-in-a-zip container formats (OOXML for the first three, the HWPX schema for the fourth); this section states what `OoxmlParser`/`HwpParser` extract as `UnitOccurrences` and how the corresponding `DocumentWriter` (Vol. 12 §8.4) writes back, since read and write coverage are tracked separately (Vol. 12 §8.7 is authoritative on write support; this section is read-side).

- **DOCX**: paragraphs and headings as `Section` units; tables as `Table` units with cell-

level **UnitOccurrences**; embedded comments and tracked-changes (**w:ins/w:del**) are extracted as metadata attached to the paragraph they annotate, not discarded — an evidence-gate citation into a document under active review should be able to say whether the cited text is an accepted change, a pending insertion, or a pending deletion.

- **PPTX**: each slide is a unit; text boxes, tables, and speaker notes within a slide are child units (span type **slide_object_id**, Appendix B **UnitOccurrence**); embedded charts/images are recorded as opaque objects with position metadata (§8.7), not parsed for content.
- **XLSX**: each sheet is a unit; cells and named ranges are child units (span type **sheet_cell**); formulas are extracted as both their formula text and (when the workbook carries a cached value) their last-computed value, tagged separately so retrieval can distinguish “the formula is” from “the value was.”
- **HWPX**: the XML-based Hangul Word Processor format; structurally closer to DOCX (paragraph/section-based) and parsed on the same principles above via **HwpXParser**.
- **HWP (binary)**: read-only in this revision. The legacy binary HWP format has no publicly specified write-back path that preserves round-trip fidelity comparable to OOXML/HWPX; **HwpParser** extracts **UnitOccurrences** for retrieval, but Vol. 12’s approval pipeline does not offer HWP as a mutation target until a **DocumentWriter** for it exists and passes **FormatValidator** (Vol. 12 §8.7). This is a stated gap, not a silent one: a mutation request against a **.hwp** file is rejected at **PatchPlanner** with an explicit “format not writable” error rather than silently falling back to a lossy conversion.

2.3 Semantic Unit Model

Definition 2.6 (Semantic Unit and Its Occurrence). A parser emits two related but distinct records per parsed fragment, not one (Appendix B §A.1 gives the full schema; this is the parser-facing view of the same split):

$$u = (\text{id}, \text{type}, \text{content}, \text{metadata}, \text{children}), \quad o = (\text{occurrence_id}, \text{content_id}, \text{span}, \text{artifact_version_id}) \quad (2.1)$$

where:

- **id** (= **content_id**) = SHA-256(canonical(content, type)) — **not** a function of **span**. Two occurrences of identical content at different spans (the same helper function copy-pasted into two files, a repeated boilerplate header) share this id; that sharing is what makes deduplication work, and it is destroyed if span is hashed into the id.
- **type** ∈ {Function, Class, Section, Table, Operation, ...}
- **content**: canonical text representation (whitespace-normalized)
- **metadata**: typed key-value annotations (see §2.3.2)
- **children**: ordered list of child unit IDs (hierarchy)
- **occurrence_id** = SHA-256(artifact_version_id, span) — unique per location, even when **content_id** repeats

- `span`: (`start_line`, `end_line`, `start_col`, `end_col`) in source for text/code; other formats use other span types (Appendix B `UnitOccurrence.span_type`)

A parser that only emits *u* and discards *o* (or that folds span into *id* as an earlier draft of this definition did) cannot answer “where did this content come from” once a second occurrence exists, which is exactly the failure mode Appendix B §A.1 documents.

2.3.1 Unit Type Taxonomy

Table 2.1: Core Semantic Unit Types by Parser Domain

Domain	Unit Types	Key Metadata
Python	Module, Class, Function, Method, Decorator, Import	signature, decorators, type_hints, docstring
JavaScript/TS	Module, Class, Function, ArrowFunction, Interface, Type	signature, jsdoc, exports
Markdown	Document, Section, CodeBlock, List, Table, Link	heading_level, language, alt_text
PDF	Page, Paragraph, Table, Figure, Header, Footer	bbox, font_info, page_num
OpenAPI	Info, Server, Path, Operation, Schema, Parameter	method, path, status_codes, auth
SQL DDL	Database, Schema, Table, Column, Index, Constraint	data_type, nullable, default, references
YAML/JSON	Document, Mapping, Sequence, Scalar	json_path, schema_ref
Dockerfile	Instruction, Stage, Arg, Env, Label	instruction, value, stage
Protobuf	Message, Enum, Service, Field, RPC	field_number, type, label

2.3.2 Metadata Schema

Each unit type defines a typed metadata schema. Examples:

```
# Python Function metadata
{
  "signature": "def foo(x: int, y: str = 'a') -> bool",
  "decorators": ["@staticmethod", "@cache"],
  "type_hints": {"x": "int", "y": "str", "return": "bool"},
  "docstring": "Returns True if...",
  "is_async": false,
  "is_generator": false,
  "complexity": 3
}

# SQL Table metadata
{
  "columns": [
    {"name": "id", "type": "BIGINT", "nullable": false, "primary_key":
      ↪ true},
    {"name": "email", "type": "VARCHAR(255)", "nullable": false, "unique":
      ↪ true}
  ],
  "indexes": [{"name": "idx_email", "columns": ["email"], "unique":
    ↪ true}],
  "foreign_keys": [{"column": "user_id", "references": "users.id"}],
  "engine": "InnoDB",
  "charset": "utf8mb4"
}
```

```
# OpenAPI Operation metadata
{
  "method": "POST",
  "path": "/api/v1/users",
  "operation_id": "createUser",
  "parameters": [{"name": "body", "in": "body", "schema": "UserInput"}],
  "responses": {"201": "User", "400": "Error"},
  "security": [{"BearerAuth": []}],
  "deprecated": false
}
```

2.3.3 Canonicalization

Content-addressability requires deterministic canonicalization:

Algorithm 3 Unit Canonicalization

Require: Unit u with fields ($type, content, span, metadata, children$)

Ensure: Canonical byte string C for hashing

- | | |
|---|----------------------|
| 1: $C \leftarrow type \parallel$ | ▷ Type prefix |
| 2: $C \leftarrow C \parallel \text{NORMALIZEWHITESPACE}(content)$ | ▷ Canonical text |
| 3: $C \leftarrow C \parallel \text{SORTKEYS}(metadata)$ | ▷ Deterministic JSON |
| 4: $C \leftarrow C \parallel \text{SORT}(children)$ | ▷ Sorted child IDs |
| 5: return C | |
-

This ensures: (a) semantically identical units in different files produce identical IDs, (b) formatting changes that don't affect semantics (whitespace, comment reordering) don't change IDs if metadata excludes them.

2.4 Language-Specific Parsers

2.4.1 Tree-sitter Based Parsers (Code)

For programming languages, we use [tree-sitter](#) for robust, incremental, error-tolerant parsing.

Design Decision 2.7. Decision: Use tree-sitter as the primary parsing engine for all supported programming languages.

Design Rationale 2.8. 1. **Incremental re-parse:** tree-sitter reuses syntax tree nodes for unchanged regions ($O(\log n)$ edits).

2. **Error tolerance:** Recovers gracefully from syntax errors; produces partial trees.

3. **Unified API:** Same query language (SCM) across 40+ languages.

4. **Bindings:** Rust ([tree-sitter](#)), Python ([tree-sitter-languages](#)), Node.js available.

5. **Grammar maintenance:** Community-maintained grammars; we vendor specific versions.

Tree-sitter Query Patterns. Semantic units are extracted via SCM queries. Example for Python functions:

```
(function_definition
  name: (identifier) @name
  parameters: (parameters) @params
  return_type: (type)? @return_type
  body: (block) @body
  decorators: (decorator)* @decorators
) @function
```

The query captures are mapped to `SemanticUnit` fields. A `QueryMapper` per language handles this transformation.

2.4.1.1 Supported Languages

Table 2.2: Tree-sitter Languages Supported (v0.1)

Language	Grammar Version	Unit Types
Python	0.20.1	Module, Class, Function, Method, Decorator, Import, TypeAlias
JavaScript	0.20.0	Module, Class, Function, ArrowFunction, Interface, Type, Import
TypeScript	0.20.0	(extends JS) + TypeAnnotation, Generic
Rust	0.20.1	Module, Struct, Enum, Fn, Impl, Trait, Macro, Use
Go	0.20.0	Package, Struct, Func, Method, Interface, Type, Import
Java	0.20.1	Class, Interface, Method, Field, Constructor, Annotation
C/C++	0.20.1	Function, Struct, Class, Macro, Include, Namespace
C#	0.20.0	Class, Method, Property, Interface, Attribute, Using
SQL	0.20.0	(see §2.4.5)

2.4.2 Markdown Parser

Uses `pulldown-cmark` (Rust) or `markdown-it` (Python) with extensions:

- GitHub Flavored Markdown (GFM): tables, task lists, strikethrough
- Frontmatter (YAML/TOML/JSON) extraction
- Heading anchoring and TOC generation
- Code block language detection
- Link/reference resolution

Units produced: `Document`, `Section(level)`, `Paragraph`, `CodeBlock(lang)`, `Table`, `List`, `Link`, `Image`.

2.4.3 PDF Parser

Uses `pdfplumber` (Python) or `lopdf` (Rust) with optional OCR (`tesseract`):

- Text extraction with position (bbox, font, size)
- Table detection (via `pdfplumber` table extraction)

- Figure/image extraction (bbox, optional OCR)
- Reading order reconstruction (column-aware)

Units: Page, Paragraph, Table, Figure, Header, Footer.

2.4.4 OpenAPI / AsyncAPI Parser

Parses OpenAPI 3.0/3.1 and AsyncAPI 2.x/3.x specs. Produces units for every reusable component:

- Info, Server, Path, Operation
- Schema (with JSON Schema resolution)
- Parameter, RequestBody, Response
- SecurityScheme, Tag, ExternalDocs

Cross-references (`$ref`) are resolved and stored as graph edges in the semantic units.

2.4.5 SQL DDL Parser

Uses `sqlglot` (Python) for multi-dialect parsing (PostgreSQL, MySQL, SQLite, BigQuery, Snowflake, T-SQL). Produces units for: Database, Schema, Table, Column, Index, Constraint, View, Function, Trigger.

Column-level lineage is extracted: Column units reference source columns in `SELECT` clauses.

2.4.6 Structured Data Parser (YAML/JSON/TOML)

For configuration and data files:

- JSON Schema / OpenAPI schema detection (if `$schema` present)
- Path-based units: each mapping key becomes a unit with JSON path metadata
- Array elements indexed with position
- Merge with schema annotations when available

2.5 Structure-Aware Chunking

After parsing, semantic units may exceed embedding model context windows. The `Chunker` splits units while preserving semantic boundaries.

Interface 2.0 (`Chunker`).

```
fn chunk(&self, unit: SemanticUnit, config: ChunkConfig) ->
    ↪ Vec<Chunk>;
fn config_schema(&self) -> ChunkConfigSchema;
}
```

```
struct ChunkConfig {
    max_tokens: usize,
```

```

    overlap_tokens: usize,
    respect_boundaries: bool,      // Don't split inside function/class
    preserve_hierarchy: bool,     // Include parent context
    strategy: ChunkStrategy,      // "hierarchical", "sliding",
    ↪ "semantic"
}

```

2.5.1 Hierarchical Chunking Algorithm

Default strategy for code and structured documents:

Algorithm 4 Hierarchical Chunking

Require: Unit u with children $c_1, \dots, c_n$, config cfg

Ensure: Chunks $\mathcal{C}$ each $\leq cfg.max_tokens$

```

1:  $\mathcal{C} \leftarrow \emptyset$ 
2:  $current \leftarrow \emptyset, current\_tokens \leftarrow 0$ 
3: for all child  $c \in children(u)$  in order do
4:    $child\_tokens \leftarrow TOKENCOUNT(c.content)$ 
5:   if  $current\_tokens + child\_tokens > cfg.max\_tokens$  and  $current \neq \emptyset$  then
6:      $\mathcal{C} \leftarrow \mathcal{C} \cup \{current\}$ 
7:     if  $cfg.overlap\_tokens > 0$  then
8:        $current \leftarrow TAIL(current, cfg.overlap\_tokens)$ 
9:        $current\_tokens \leftarrow TOKENCOUNT(current)$ 
10:    else
11:       $current \leftarrow \emptyset, current\_tokens \leftarrow 0$ 
12:     $current \leftarrow current \parallel c.content$ 
13:     $current\_tokens \leftarrow current\_tokens + child\_tokens$ 
14: if  $current \neq \emptyset$  then
15:    $\mathcal{C} \leftarrow \mathcal{C} \cup \{current\}$ 
16: return  $\mathcal{C}$ 

```

2.5.2 Sliding Window (Fallback)

For plain text or when hierarchical structure is unavailable:

```

# Sliding window with sentence boundary awareness
strategy = "sliding"
window_size = 512      # tokens
overlap = 50           # tokens
split_on = ["\n\n", "\n", ". ", "! ", "? ", "; "]

```

2.5.3 Semantic Chunking (Experimental)

Uses embedding similarity to find natural boundaries:

Algorithm 5 Semantic Chunking via Embedding Similarity**Require:** Sentences $s_1, \dots, s_n$, threshold θ , `max_tokens`**Ensure:** Chunks $\mathcal{C}$

```

1:  $embs \leftarrow \text{EMBED}(s_1, \dots, s_n)$ 
2:  $boundaries \leftarrow \{1\}$ 
3: for  $i = 2$  to  $n$  do
4:    $sim \leftarrow \text{COSINE}(embs[i-1], embs[i])$ 
5:   if  $sim < \theta$  then
6:      $boundaries \leftarrow boundaries \cup \{i\}$ 
7:  $boundaries \leftarrow boundaries \cup \{n+1\}$ 
8:  $\mathcal{C} \leftarrow \emptyset$ 
9: for all consecutive  $(b_j, b_{j+1}) \in boundaries$  do
10:   $chunk \leftarrow \text{MERGESENTENCES}(s_{b_j}, \dots, s_{b_{j+1}-1})$ 
11:  if  $\text{TOKENCOUNT}(chunk) > \text{max\_tokens}$  then
12:     $\mathcal{C} \leftarrow \mathcal{C} \cup \text{SLIDINGWINDOW}(chunk)$ 
13:  else
14:     $\mathcal{C} \leftarrow \mathcal{C} \cup \{chunk\}$ 
15: return  $\mathcal{C}$ 

```

2.5.4 Scale-Free Chunking by Graph Renormalization

The three strategies above share a defect that no amount of tuning removes: each commits to a scale. `window_size = 512` is a claim that the natural unit of meaning in the corpus is 512 tokens wide, and that claim is false for every corpus — a call site, a function, a module, and a subsystem are all legitimate retrieval targets at different query granularities. Fixing the scale at ingestion time forces the retrieval stage to reconstruct the levels that chunking destroyed.

Coarse-graining across scales without committing to one is the subject of the renormalization group. The construction repeatedly integrates out short-range degrees of freedom, generating a flow through a family of effective descriptions in which structures that survive many steps are the physically meaningful ones. Recent work carries this to graphs directly: [Schmidt et al. \[2025\]](#) define a renormalization procedure over the coarse-grained Laplacian that produces reduced representations across scales while preserving both dynamics and large-scale topology, and note explicitly that this differs from clustering methods such as Louvain — renormalization captures characteristic structure *across* scales rather than partitioning at one.

Since LLMWiki already maintains a knowledge graph over units (Vol. 03), the machinery applies without a new data structure.

Definition 2.10 (Renormalization Chunk Ladder). Let G_0 be the unit graph with edges weighted by containment, adjacency, and reference. A chunk ladder is the sequence $G_0 \rightarrow G_1 \rightarrow \dots \rightarrow G_L$ where each $G_{\ell+1}$ is obtained by spectral coarse-graining of G_ℓ , and the chunk set at level ℓ is the set of vertices of G_ℓ , each carrying the concatenated content of the units it subsumes.

Observation 2.11 (Persistence Selects Boundaries). A chunk boundary that survives many renormalization steps separates parts of the corpus that remain weakly coupled under coarse-graining; a boundary that dissolves after one step was an artifact of local token statistics. Boundary quality is therefore measurable as persistence across the ladder,

replacing the unfalsifiable question “is 512 the right window?” with the answerable one “at which level does this boundary disappear?”

Retrieval indexes every level of the ladder, and the query planner selects the level rather than the ingestion configuration doing so: a query naming a specific symbol resolves against G_0 , while a query about a subsystem’s responsibilities resolves against a coarse level whose vertices correspond to that subsystem. The cost is index size, which grows as $\sum_\ell |V(G_\ell)|$ — bounded by a small multiple of $|V(G_0)|$ because coarse-graining contracts the vertex set geometrically.

Status. This is the least mature of the mechanisms specified in this document. Spectral coarse-graining of the unit graph is well-defined and implementable, but the claim that renormalization-persistent boundaries yield better retrieval than tuned fixed-width chunking is an empirical one that the evaluation framework of Vol. 06 must settle before it is relied upon. It is specified here because the ladder subsumes the existing strategies as special cases — $L = 0$ recovers hierarchical chunking exactly — and can therefore be adopted without discarding the current pipeline.

2.6 Entity and Relation Extraction

After chunking, lightweight extraction identifies entities and relations within and across units.

2.6.1 Rule-Based Extraction

Zero-LLM approach using tree-sitter queries and regex patterns:

- **Code:** Function calls, imports, class inheritance, type references
- **Markdown:** Links, cross-references, image references
- **OpenAPI:** Schema references, parameter bindings, auth requirements
- **SQL:** Table/column references, foreign keys, join conditions
- **Config:** Environment variable references, file path references

Produces (source_unit, target_unit, relation_type, confidence) tuples.

2.6.2 LLM-Assisted Extraction (Optional)

For complex relations (e.g., implicit semantic links, cross-file call graphs), a local LLM can be invoked:

```
# Extraction prompt template (Vol. 05 Agent Runtime)
EXTRACT_ENTITIES_RELATIONS = """
Given the following semantic units from a {language} codebase,
extract entities and relations in JSON format.
```

```
Units:
{units_json}
```

```
Entity types: {config.entity_types}
```

Relation types: {config.relation_types}

Output format:

```
{
  "entities": [{"id": "...", "type": "...", "name": "...", "unit_id":
    ↪ "..."}],
  "relations": [{"source": "...", "target": "...", "type": "...",
    ↪ "confidence": 0.0-1.0}]
}
```

2.6.3 Entity Resolution

Extracted entities are resolved against the knowledge graph (Vol. 03):

Algorithm 6 Entity Resolution

Require: Extracted entity e with name n , type t ; graph G

Ensure: Resolved node ID or new node

```
1:  $candidates \leftarrow \text{GRAPHLOOKUP}(G, name = n, type = t)$ 
2: if  $candidates = \emptyset$  then
3:   return  $\text{CREATE\_NODE}(G, e)$  ▷ New entity
4: else
5:    $best \leftarrow \arg \max_{c \in candidates} \text{SIMILARITY}(c, e)$ 
6:   if  $\text{SIMILARITY}(best, e) \geq \tau_{er}$  then
7:     return  $best.id$  ▷ Merge
8:   else
9:     return  $\text{CREATE\_NODE}(G, e)$  ▷ Distinct entity
```

Similarity combines: name fuzzy match, type compatibility, context overlap (shared neighbors), embedding cosine similarity.

2.7 Incremental Parsing

When a file changes, re-parsing only affected regions is critical for latency.

Tree-sitter Incremental Parse. Tree-sitter accepts the old tree and the edit (byte range + new text). It returns a new tree with reused nodes. We compute the diff at unit level:

Algorithm 7 Incremental Unit Diff**Require:** Old units U_{old} , new tree T_{new} , edit range E **Ensure:** (*added*, *modified*, *removed*) unit ID sets

- 1: $U_{new} \leftarrow \text{EXTRACTUNITS}(T_{new})$
- 2: $affected \leftarrow \{u \in U_{old} \mid u.span \cap E \neq \emptyset\}$
- 3: $unchanged \leftarrow U_{old} \setminus affected$
- 4: $modified \leftarrow \{u_{new} \in U_{new} \mid \exists u_{old} \in affected : u_{new}.id = u_{old}.id \wedge u_{new} \neq u_{old}\}$
- 5: $added \leftarrow \{u_{new} \in U_{new} \mid u_{new}.id \notin U_{old}.ids\}$
- 6: $removed \leftarrow \{u_{old} \in affected \mid u_{old}.id \notin U_{new}.ids\}$
- 7: **return** (*added*, *modified*, *removed*)

Only *added* $\cup$ *modified* units are re-embedded and re-indexed. *removed* units are tombstoned in indices.

2.8 Embedding Generation

The **Embedder** trait (Vol. 01, §1.25) abstracts local embedding models.

2.8.1 Supported Models

Table 2.3: Local Embedding Models (Recommended)

Model	Dim	Max Tok	Speed (CPU)	MTEB	License
BGE-M3	1024	8192	~500 tok/s	62.3	MIT
E5-Large-v2	1024	512	~400 tok/s	61.5	MIT
Nomic-Embed-Text-v1.5	768	8192	~600 tok/s	60.1	Apache-2.0
BGE-Small-En-v1.5	384	512	~1200 tok/s	58.2	MIT
Multilingual-E5-Large	1024	512	~350 tok/s	63.1	MIT

2.8.2 Batch Inference

All embeddings use batched inference for throughput:

```
# Embedder config
batch_size = 32
max_seq_len = 512
normalize = true
device = "cpu" # or "cuda", "mps"
onnx_runtime = true # optimized inference
```

Embeddings are stored as `float32` in the HNSW index (Vol. 03) and as `float16` in the unit metadata for memory efficiency.

2.9 Parser Extensibility Guide

To add a custom parser:

1. Implement `Parser` trait
2. Register in `config.toml`
3. Define unit types and metadata schemas
4. (Optional) Add tree-sitter grammar or SCM queries
5. Write tests against `testdata/` fixtures

```
# Example: Custom parser for GraphQL SDL
[ingestion.parsers.graphql]
type = "custom"
module = "my_parsers.graphql_parser"
class = "GraphQLParser"
supported_mimes = ["application/graphql", "text/x.graphql"]
extensions = [".graphql", ".gql"]
```

2.10 Summary

The semantic parsing layer transforms heterogeneous artifacts into a uniform representation of content-addressed, hierarchically structured, metadata-rich semantic units. Key properties:

1. **Language-aware:** Tree-sitter for code, specialized parsers for structured formats
2. **Hierarchical:** Units form trees reflecting source structure
3. **Content-addressed:** SHA-256 identifiers enable deduplication and lineage
4. **Incremental:** Tree-sitter's incremental parsing + unit-level diff
5. **Extensible:** Parser trait + registry for new formats
6. **Metadata-rich:** Typed annotations per unit type for downstream use

The next volume (Vol. 03) details how these units populate the knowledge graph and semantic indices.

Knowledge Graph & Semantic Index

This volume specifies the *knowledge graph* (property graph with typed nodes/edges) and the *semantic index* (hybrid search: vector + keyword + graph adjacency). These two modules form Layer 2 Core (Vol. 01, §1.2.2) and are tightly coupled: the graph provides structural semantics, the index provides fast retrieval, and both share the same content-addressed unit identifiers.

3.1 Knowledge Graph

3.1.1 Data Model

Definition 3.1 (Property Graph). A *property graph* $G = (V, E, \lambda_V, \lambda_E, \sigma_V, \sigma_E)$ where:

- V : set of nodes (entities, concepts, units)
- $E \subseteq V \times V$: set of directed edges (relationships)
- $\lambda_V : V \rightarrow \mathcal{L}_V$: node label function (type)
- $\lambda_E : E \rightarrow \mathcal{L}_E$: edge label function (relation type)
- $\sigma_V : V \rightarrow \mathcal{P}(\text{Key} \times \text{Value})$: node properties
- $\sigma_E : E \rightarrow \mathcal{P}(\text{Key} \times \text{Value})$: edge properties

Node Types.

- **Unit**: A semantic unit from parsing (Vol. 02). Properties: `id` (SHA256), `type`, `span`, `file_path`, `language`, `metadata`(JSON).
- **Entity**: Resolved entity (class, function, table, concept). Properties: `name`, `type`, `canonical_id`, `description`, `embedding_ref`.
- **File**: Source artifact. Properties: `path`, `hash`, `size`, `mtime`, `mime`.
- **Module/Package/Namespace**: Logical grouping. Properties: `name`, `parent_id`.

Edge Types.

- CONTAINS: File $\rightarrow$ Unit, Module $\rightarrow$ Unit, Unit $\rightarrow$ Unit (nesting)
- DEFINES: Unit $\rightarrow$ Entity (unit defines entity)
- REFERENCES: Unit $\rightarrow$ Entity (unit references entity)
- CALLS: Entity $\rightarrow$ Entity (function call)
- INHERITS: Entity $\rightarrow$ Entity (class inheritance)
- IMPORTS: Module $\rightarrow$ Module
- RELATES: Entity $\leftrightarrow$ Entity (semantic similarity, co-occurrence)
- NEXT_TO: Unit $\rightarrow$ Unit (sequential proximity in source)

Content-Addressed Identity. Every `Unit` node has `id = SHA256(canonical_form)`. `Entity` nodes have `canonical_id` derived from name + type + context hash. This ensures deterministic identity across re-ingestion.

3.1.2 Graph Store Backend

The `GraphStore` trait (Vol. 01, §1.26) admits multiple backends:

Table 3.1: Graph Store Backend Comparison

Backend	Embedded	Scale	Transactions	License
RocksDB (custom)	Yes	10B+ edges	Yes (WAL)	Apache-2.0
SQLite (with ext)	Yes	100M edges	Yes (WAL)	Public Domain
Redwood (Rust)	Yes	1B edges	Yes	MIT
Neo4j Embedded	No	100B edges	Yes	GPL/Commercial
Kuzu	Yes	10B edges	Yes	MIT

Default: RocksDB-based custom store (`llmwiki-graph`). Nodes and edges stored as separate column families with prefix encoding for efficient prefix scans (by type, by partition).

3.1.2.1 Storage Layout (RocksDB)

Column Families:

-- nodes:{type}	# Key: node_id (fixed 32 bytes), Value: serialized props
-- edges:{rel_type}	# Key: src_id(32) dst_id(32), Value: edge props
-- adj:out:{type}	# Key: src_id, Value: list of (rel_type, dst_id) - adjacency
-- adj:in:{type}	# Key: dst_id, Value: list of (rel_type, src_id) - reverse adj
-- idx:label:{label}	# Key: label node_id, Value: empty (label index)
-- idx:prop:{key}	# Key: prop_value node_id, Value: empty (property index)
`-- meta	# Key: "stats", "version", "config", Value: JSON

3.1.3 Entity Resolution

When a parser emits a **Unit** that **DEFINES** or **REFERENCES** an entity, the graph layer resolves it to a canonical **Entity** node.

Algorithm 8 Entity Resolution

Require: Unit u with entity mention $m = (name, type, context)$

Ensure: Resolved **Entity** node e

```

1:  $candidates \leftarrow \text{LOOKUPBYNAMETYPE}(G, m.name, m.type)$ 
2: if  $candidates = \emptyset$  then
3:   return  $\text{CREATEENTITY}(m, u)$ 
4:  $best \leftarrow \arg \max_{c \in candidates} \text{SCORE}(c, m, u)$ 
5: if  $\text{SCORE}(best, m, u) \geq \tau_{er}$  then
6:    $\text{UPDATEENTITY}(best, u)$  ▷ Merge context, update embedding
7:   return  $best$ 
8: else
9:   return  $\text{CREATEENTITY}(m, u)$  ▷ New distinct entity

```

Scoring function:

$$\begin{aligned}
 \text{SCORE}(c, m, u) = & \alpha \cdot \text{FuzzyMatch}(c.name, m.name) \\
 & + \beta \cdot \text{TypeCompat}(c.type, m.type) \\
 & + \gamma \cdot \text{ContextOverlap}(c.context, u.context) \\
 & + \delta \cdot \text{CosineSim}(c.embedding, u.embedding)
 \end{aligned}$$

Default weights: $\alpha = 0.3, \beta = 0.2, \gamma = 0.2, \delta = 0.3$. Threshold $\tau_{er} = 0.85$ (configurable).

3.1.4 Incremental Graph Updates

On ingestion diff ($added, modified, removed$) units (Vol. 02, Alg. 7):

Algorithm 9 Incremental Graph Update

Require: Diff sets A (added), M (modified), R (removed)

Ensure: Updated graph G'

```

1: for all  $u \in R$  do
2:    $\text{DELETENODE}(u.id)$  ▷ Tombstone
3:    $\text{DELETEEDGES}(u.id)$  ▷ All incident edges
4: for all  $u \in A \cup M$  do
5:    $\text{UPSERTUNITNODE}(u)$ 
6:    $entities \leftarrow \text{EXTRACTENTITIES}(u)$ 
7:   for all  $e \in entities$  do
8:      $resolved \leftarrow \text{RESOLVEENTITY}(e, u)$  ▷ Alg. 8
9:     if  $e.relation = \text{DEFINES}$  then
10:       $\text{UPSERTEDGE}(u.id, resolved, \text{DEFINES})$ 
11:     else
12:       $\text{UPSERTEDGE}(u.id, resolved, \text{REFERENCES})$ 
13:    $\text{UPSERTADJACENCY}(u)$  ▷ Contains, Next_To
14:  $\text{COMMITTRANSACTION}()$ 
15: return  $G$ 

```

All operations within a single ingestion are batched in one RocksDB write batch (atomic).

3.1.5 Graph Traversal Primitives

The `traverse` and `neighborhoods` methods (Vol. 01, §1.26) support:

- **Pattern matching:** Cypher-like declarative patterns (e.g., `(a:Function)-[:CALLS*1..3]->(b:Function)`)
- **Bounded BFS/DFS:** k -hop neighborhoods with edge-type filters
- **Personalized PageRank:** For HippoRAG-style retrieval (Vol. 04)
- **Shortest path:** Between two entities
- **Subgraph extraction:** Induced subgraph for context packing

```
# TraversalPattern examples
pattern = {
  "start": { "type": "Function", "name": "process_order" },
  "hops": 3,
  "edge_types": ["CALLS", "REFERENCES"],
  "direction": "out",
  "filter": { "node_type": ["Function", "Class"] },
  "limit": 100
}
```

3.2 Semantic Index

The `SemanticIndex` (Vol. 01, §1.27) is a hybrid index combining three retrieval signals:

1. **Vector (HNSW):** Dense embedding similarity
2. **Keyword (BM25):** Sparse lexical matching
3. **Graph Adjacency:** Structural proximity in knowledge graph

3.2.1 HNSW Vector Index

Implementation: `hnswlib` (C++ with Python/Rust bindings) or `faiss` (with disk-backed index). Disk-backed mode: memory-mapped file for vectors, metadata in separate store.

Configuration:

```
hnsw:
  M: 16                      # max connections per node
  ef_construction: 200       # build-time search width
  ef_search: 64              # query-time search width
  dim: 1024                  # embedding dimension
  space: "cosine"            # or "l2", "ip"
  disk_backed: true
  mmap: true
```

Content-Addressed Vectors. Vectors are stored keyed by `unit_id` (SHA256). The index maps `unit_id` $\rightarrow$ internal HNSW label. On upsert: if unit exists, update vector; if new, add. On delete: mark tombstone (lazy cleanup during compaction).

3.2.2 BM25 Keyword Index

Implementation: `tantivy` (Rust) or `OpenSearch` (if distributed). Tantivy is embedded, fast, and supports incremental updates.

Schema:

```
fields:
- unit_id: Keyword (stored, indexed) # SHA256
- content: Text (analyzer: language-specific)
- file_path: Keyword (stored, indexed)
- unit_type: Keyword (stored, indexed)
- language: Keyword (stored, indexed)
```

Analyzer: Language-specific (e.g., `code` analyzer for source: split on punctuation, camel-Case, `snake_case`; `standard` for prose).

3.2.3 Graph Adjacency Index

For query-time graph expansion (Vol. 04), we maintain a lightweight adjacency index separate from the full graph store:

Adjacency Index (RocksDB column family "adj:index"):

Key: `unit_id` (32 bytes)

Value: serialized adjacency list

```
[
  { "rel": "CONTAINS", "target": "sha256...", "weight": 1.0 },
  { "rel": "REFERENCES", "target": "sha256...", "weight": 0.8 },
  { "rel": "NEXT_TO", "target": "sha256...", "weight": 0.3 }
]
```

Updated incrementally alongside graph. Used for `search_graph` in hybrid retrieval.

3.2.4 Hybrid Search

The `search_hybrid` method combines three signals:

Algorithm 10 Hybrid Search**Require:** Query q (text + optional embedding), k , weights w_v, w_k, w_g **Ensure:** Top- k scored units

```

1:  $q_v \leftarrow \text{EMBED}(q.\text{text})$  ▷ If not provided
2:  $R_v \leftarrow \text{HNSWSEARCH}(q_v, k \times 3)$ 
3:  $R_k \leftarrow \text{BM25SEARCH}(q.\text{text}, k \times 3)$ 
4:  $R_g \leftarrow \text{GRAPHEXPAND}(q, k \times 2)$  ▷ Adjacency traversal
5:  $\text{scores} \leftarrow \emptyset$ 
6: for all  $r \in R_v \cup R_k \cup R_g$  do
7:    $s_v \leftarrow r \in R_v ? \text{Score}(r, R_v) : 0$ 
8:    $s_k \leftarrow r \in R_k ? \text{Score}(r, R_k) : 0$ 
9:    $s_g \leftarrow r \in R_g ? \text{Score}(r, R_g) : 0$ 
10:   $\text{scores}[r.\text{id}] = w_v \cdot s_v + w_k \cdot s_k + w_g \cdot s_g$ 
11: return  $\text{TOPK}(\text{scores}, k)$ 

```

Default weights: $w_v = 0.6, w_k = 0.3, w_g = 0.1$ (configurable per query type).

Score normalization: Reciprocal Rank Fusion (RRF) or min-max normalization per signal before weighted sum.

3.2.5 Index Update Strategy

Vector (HNSW): Incremental add/update. HNSW supports dynamic insertion. Deletes = tombstones. Periodic `rebuild_threshold` triggers full rebuild.

BM25 (Tantivy): Incremental commit. Segments merged in background. Delete = term tombstone.

Graph Adjacency: Updated in same transaction as graph (RocksDB batch).

3.3 Query-Time Graph Expansion

Used by retriever to expand candidate set via graph structure:

1. **Seed:** Start from vector/keyword hits S
2. **Expand:** For each $s \in S$, fetch k -hop neighborhood $N_k(s)$
3. **Score:** Personalized PageRank from S over induced subgraph
4. **Merge:** Add top- m PPR nodes to candidate pool

This implements the “graph traversal” retrieval path complementary to vector/keyword.

3.4 Performance Characteristics**3.5 Patent-Relevant Novelty**

Patent Claim 3.2. Claim 4: A knowledge graph system wherein: (a) all nodes are content-addressed via SHA-256 of their canonical representation, (b) entity resolution

Table 3.2: Expected Performance (Single Node, SSD)

Operation	Latency (p99)	Throughput
Node upsert (batch 1000)	< 50 ms	20k nodes/s
Edge upsert (batch 5000)	< 100 ms	50k edges/s
k -hop traversal ($k=2$, 100 seeds)	< 10 ms	10k queries/s
PPR (100 seeds, 3 hops)	< 50 ms	2k queries/s
HNSW search ($k=10$, $ef=64$)	< 5 ms	2k queries/s
BM25 search ($k=10$)	< 3 ms	3k queries/s
Hybrid search ($k=20$)	< 15 ms	1k queries/s

merges mentions using a weighted combination of name similarity, type compatibility, context overlap, and embedding similarity, (c) the graph is persisted in a column-family store with prefix-encoded keys enabling type-partitioned scans, and (d) a separate graph adjacency index enables sub-millisecond k -hop expansion for hybrid retrieval.

Patent Claim 3.3. Claim 5: The system of Claim 4 wherein incremental updates to the graph and indices are executed atomically within a single write batch, and deterministic replay is achieved by re-applying the ingestion log against an empty store.

3.6 Summary

The knowledge graph and semantic index form the queryable substrate of LLMWiki:

1. **Graph:** Property graph with content-addressed nodes, entity resolution, and incremental updates
2. **Vector Index:** HNSW with disk-backed storage, SHA-256 keyed
3. **Keyword Index:** Tantivy BM25 with language-aware analyzers
4. **Adjacency Index:** Lightweight graph expansion for structural retrieval
5. **Hybrid Search:** RRF-weighted fusion of three signals

All indices share the same unit identifiers, enabling seamless cross-index joins and evidence tracing (Vol. 06).

The next volume (Vol. 04) details the retrieval pipeline including the Evidence Gate.

Retrieval and Evidence Gate

This volume specifies the *retrieval pipeline*—the hybrid search mechanism that fuses vector, keyword, and graph signals—and the *Evidence Gate*, the mandatory verification stage that ensures every retrieved unit cited in an answer is semantically relevant to the query. This is the critical differentiator from prior RAG systems: retrieval is necessary but not sufficient; evidence verification is mandatory.

4.1 Evidence Gate: Core Principle

Design Principle 4.1 (Evidence Closure). No retrieval result reaches the synthesis stage without passing multi-stage verification against calibrated thresholds. This enforces *evidence closure*: every citation in an answer traces to a verified unit.

A single cross-encoder score answers one question — *is this unit relevant to the query* — and cannot by itself answer three others that “evidence-aware” implies: whether the unit’s content is still true, whether a specific generated claim is actually entailed by the unit (as opposed to merely topically adjacent to it), and whether the answer as a whole is backed by evidence for every claim it makes, not just the claims someone happened to cite. Treating $\text{score}(q, u) \geq \tau$ as the definition of “verified evidence” conflates these. This volume therefore separates the gate into four distinct checks, only the first two of which run before synthesis (§4.3.4); the other two run against the generated answer (§4.3.6) and are what actually justify calling an answer evidence-closed:

1. **Relevance Gate** (§4.3.4, Stages 1–2): is the candidate unit relevant to the query at all. This is what a cross-encoder score measures.
2. **Structural / Source Validity Gate** (§4.3.4, Stage 3, extended below): is the candidate reachable/consistent in the graph, and — where the source tracks it — not stale or access-restricted.
3. **Claim Support Gate** (§4.3.4, Stage 4; §4.3.6): does a specific claim, not just the unit as a whole, entail from the cited evidence, with an explicit contradiction check, not only an entailment one.
4. **Evidence Coverage Gate** (§4.3.6): after synthesis, is every atomic claim in the answer — not only the ones the model chose to cite — mapped to supporting evidence.

Definition 4.2 (Evidence Gate). The (pre-synthesis) Evidence Gate is a function $\mathcal{G} : \mathcal{Q} \times \mathcal{U}^* \times \Theta \rightarrow \mathcal{U}^*$ that takes a query q , a candidate set $\mathcal{R} \subseteq \mathcal{U}$, and a threshold $\tau \in \Theta$, and

returns only those units $u \in \mathcal{R}$ where $\text{score}(q, u) \geq \tau$, where score is computed by a cross-encoder. This definition covers the Relevance Gate only (item 1 above); §4.3.4 extends it to a cascade covering items 1–3, and §4.3.6 adds the post-synthesis Claim Support and Evidence Coverage checks (items 3–4) that a single pre-synthesis threshold cannot provide.

4.2 Why Not Reranking?

The Evidence Gate is **not** a reranker. Key distinctions:

Table 4.1: Reranker vs. Evidence Gate

Property	Reranker	Evidence Gate
Output	Reordered top- k	Filtered subset (size $\leq k$)
Threshold	Implicit (top- k)	Explicit calibrated τ
Goal	Improve ranking	Enforce relevance / reject noise
Failure mode	Bad results still returned	Empty set $\rightarrow$ “no evidence”
Citation policy	Top- k cited	Only verified cited
Calibration	Rarely calibrated	Mandatory calibration

This distinction is fundamental: a reranker always returns k results; the Evidence Gate may return 0 (triggering “insufficient evidence” response), which is the correct behavior when no unit actually supports the query.

4.3 Retrieval Pipeline

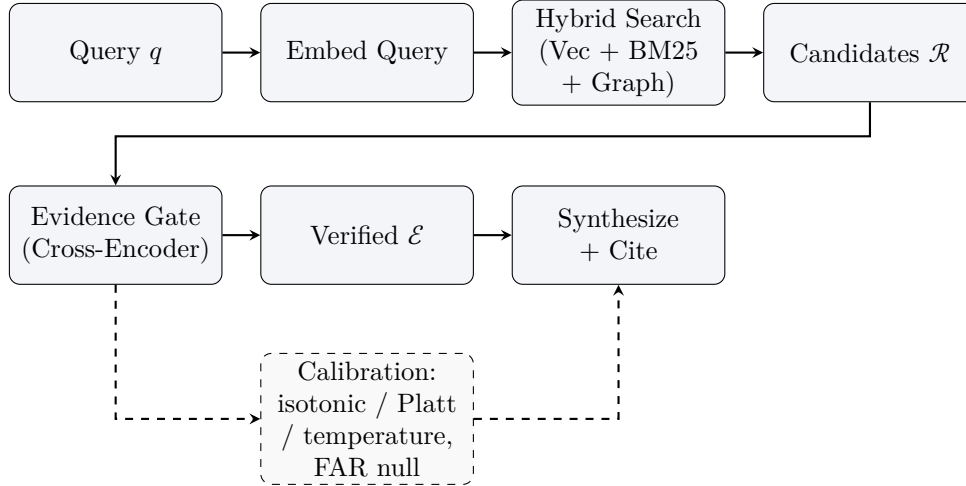


Figure 4.1: Retrieval + Evidence Gate pipeline.

4.3.1 Stage 1: Query Embedding

Query text q is embedded using the same model as ingestion (config: `[index.hnsw.dim]`). For multi-vector queries (e.g., ColBERT-style), each token embedding is preserved.

4.3.2 Stage 2: Hybrid Search

Detailed in Vol. 03 §3.2.4. Returns candidate set $\mathcal{R}$ of size k_{cand} (default 50).

4.3.3 Stage 3: Evidence Gate

Cross-Encoder Model. A transformer cross-encoder (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2`, `jinaai/jina-reranker-v2-base`) scores (q, u) pairs:

$$\text{score}(q, u) = \text{sigmoid}(\text{CrossEncoder}([q; \text{SEP}; u.\text{content}])) \quad (4.1)$$

Batch Processing. Candidates are scored in batches (config: `[evidence_gate.batch_size]`).

Thresholding. Units with score $< \tau$ are discarded. τ is calibrated per model/dataset (see §4.4).

Algorithm 11 Evidence Gate

Require: Query q , candidates $\mathcal{R}$, threshold τ , batch size B

Ensure: Verified units $\mathcal{E}$

```

1:  $\mathcal{E} \leftarrow \emptyset$ 
2: for  $i = 1$  to  $\lceil |\mathcal{R}|/B \rceil$  do
3:    $batch \leftarrow \mathcal{R}[(i-1)B : iB]$ 
4:    $scores \leftarrow \text{CROSSENCODERBATCH}(q, batch)$ 
5:   for all  $(u, s) \in \text{ZIP}(batch, scores)$  do
6:     if  $s \geq \tau$  then
7:        $\mathcal{E} \leftarrow \mathcal{E} \cup \{(u, s)\}$ 
8: return  $\text{SORTBYScoreDESC}(\mathcal{E})$ 
```

4.3.4 Multi-Stage Proofreading Cascade

Algorithm 11 performs *single-step* discrimination: one score, one threshold. Its achievable error rate is bounded by the separation of the score distributions of relevant and irrelevant units under a single measurement. This is the direct analogue of equilibrium binding discrimination in molecular recognition, where selectivity cannot exceed the ratio of dissociation constants.

Biological systems face an identical constraint and defeat it. A T cell must discriminate agonist from self peptide despite very small differences in receptor binding affinity, yet achieves error rates far below the equilibrium bound. The mechanism, due to Hopfield [1974] and Ninio [1975], is *kinetic proofreading*: the recognition event is decomposed into n sequential intermediate states, each of which may irreversibly discard the candidate, and each of which is driven out of equilibrium by energy dissipation. Recent work has refined this picture, showing that parallel reaction threads on a single receptor improve discrimination fidelity without lengthening the sequential chain [Kirby, D. and collaborators, 2025], and that multimolecular schemes can escape the classical activity–fidelity trade-off [Mao et al., 2026].

We adopt this architecture directly.

Design Principle 4.3 (Proofreading over Thresholding). Evidence verification is a cascade of n irreversible discard stages of increasing cost and decreasing throughput, not a single thresholded score. Fidelity is purchased with computation, and the exchange rate is quantifiable.

Definition 4.4 (Proofreading Cascade). A proofreading cascade of depth n is a sequence of verifiers $g_1, \dots, g_n$ with thresholds $\tau_1, \dots, \tau_n$ and unit costs $c_1 \leq \dots \leq c_n$. A candidate u survives stage j iff $g_j(q, u) \geq \tau_j$, and is discarded irreversibly otherwise. The gate output is the set of candidates surviving all n stages.

Theorem 4.5 (Multiplicative Error Suppression). *Let ε_j denote the false-acceptance probability of stage j on an irrelevant unit, and ρ_j its true-acceptance probability on a relevant unit. If the stage verdicts are conditionally independent given relevance, the cascade attains*

$$\varepsilon_{casc} = \prod_{j=1}^n \varepsilon_j, \quad \rho_{casc} = \prod_{j=1}^n \rho_j. \quad (4.2)$$

For homogeneous stages ($\varepsilon_j = \varepsilon$, $\rho_j = \rho$) the false-acceptance rate decays as ε^n while recall decays only as ρ^n ; since $\rho \gg \varepsilon$ by construction, the discrimination ratio $\rho_{casc}/\varepsilon_{casc} = (\rho/\varepsilon)^n$ grows exponentially in depth.

The conditional independence premise is the load-bearing assumption, and it is not free: stacking n copies of the same cross-encoder yields perfectly correlated verdicts and no gain whatsoever. Independence is engineered by making the stages read *different evidence channels*, which is the retrieval analogue of the parallel reaction threads of Kirby, D. and collaborators [2025]:

1. **Stage 1 — Lexical admissibility** (c_1 negligible): BM25 span overlap between query terms and unit content. Rejects candidates retrieved purely by embedding drift.
2. **Stage 2 — Cross-encoder relevance** (c_2 moderate): Algorithm 11 as specified above.
3. **Stage 3 — Structural and source-validity admissibility** (c_3 low): the unit must be reachable in the knowledge graph from at least one other surviving unit within k hops, or be the unique survivor (rejects semantically plausible but contextually orphaned units), *and*, where the source artifact carries the relevant metadata, must not be flagged stale (`ArtifactVersion.observed_at` older than a configured freshness window) or access-restricted for the requesting principal. This stage does not adjudicate factual correctness of the unit’s content — only whether the unit’s provenance metadata makes it eligible to be cited at all.
4. **Stage 4 — Entailment verification** (c_4 high): a natural language inference model must find the candidate answer span entailed, not merely topically related, by the unit, and separately scores the contradiction class so an evidently contradicting unit is discarded rather than merely scored as not-entailing.

Stages 1–4 are candidate-side: they decide which retrieved units are eligible to be shown to the synthesis model. None of them inspect what the model actually writes. §4.3.6 adds the two checks that do.

Cost. Naively a cascade is n times more expensive. It is not, because survival is exponentially rare: with per-stage survival fraction s_j the expected cost per initial candidate is

$$C = c_1 + \sum_{j=2}^n c_j \prod_{i < j} s_i, \quad (4.3)$$

so the expensive terminal stages are evaluated on a vanishing fraction of the candidate set. Ordering stages by increasing $c_j/(1-s_j)$ minimises C — cheap, highly selective filters first.

Observation 4.6 (Fidelity Requires Dissipation). In the thermodynamic setting, proof-reading fidelity beyond the equilibrium bound is possible only if the cycle consumes free energy; a cascade run at equilibrium provides no discrimination gain. The computational analogue is exact: the discard steps are irreversible, and the cascade’s advantage over a single threshold is paid for strictly in compute. This yields an accuracy-versus-compute frontier for evidence verification rather than an unbounded accuracy claim, and we regard the existence of that frontier as a design constraint rather than a limitation to be hidden.

4.3.5 Stage 5: Evidence Context Selection and WC/1 Transport

The gate output $\mathcal{E}$ is admissible but may exceed the reader budget B . Context construction is therefore a constrained selection problem, not a license to summarize or rewrite evidence. The safe baseline sorts by gate score and takes the largest prefix that fits. The experimental reference policy maximises weighted concept coverage per token with deterministic `content_id` tie-breaking and a best-singleton guard. This follows the budgeted submodular selection family [Lin and Bilmes, 2010, Peng et al., 2025]; recent multi-hop RAG evidence shows that packing gains are conditional on retrieval recall, complementary evidence structure, budget tightness, and reader capacity [Bala, 2026]. LLMWiki therefore retains top- k as the production default until the A/B protocol in Vol. 06 shows an end-to-end quality gain.

The selected set $\mathcal{E}_B$ crosses the model boundary through the line-oriented WC/1 evidence envelope. This removes repeated structural keys and instruction “prompt tax” without compressing the evidence itself [Matveev, 2026]:

WC/1 evidence envelope. A line '`@<path>`' names a source file...

```
Q How is currency validated?
@src/payment.py
0 code 3
def process_payment(amount, currency):
    validate_currency(currency)
    return gateway.charge(amount)
@src/validator.py
1 code 2
def validate_currency(currency):
    ...
```

The prefix is query-independent and may be prompt-cacheable. Each block header is `<id> <kind> <line-count>`; the parser then reads exactly that many lines, so evidence containing `@decorator`, a header-shaped line, or arbitrary Markdown remains unambiguous. The model sees only response-local integer IDs. The host retains

$$M : \text{id} \mapsto (\text{content_id}, \text{occurrence_id}, \text{document}, \text{span})$$

and resolves model citations through M after generation. Evidence text is verbatim; normalization, summarization, and abbreviation are forbidden. Score ordering remains deterministic, but the evaluation protocol must rotate or otherwise control evidence position because long-context readers can underuse evidence placed in the middle [Liu et al., 2024].

Measured transport cost. Using `tiktoken o200k_base`, the executable reference implementation reduces the complete cold prompt by 30.3% on the 20-block English fixture and 32.8% on the 8-block Korean fixture against its JSON fallback. With the fixed 52-token prefix cached, the reductions are 31.7% and 36.6%. Earlier 35.8%/41.8% figures excluded the contract prefix and used a more verbose JSON baseline; they are retained only in the design record, not as official results. The transport result establishes token reduction only. Whether the model understands WC/1 as reliably as JSON is an open A/B question.

Boundary validation and fallback. Queries, source paths, and kind labels occupy single-line grammar positions and are validated before serialization; block IDs and line counts must be non-negative and IDs unique. Unknown contract versions, unsupported capabilities, or validation failure select the JSON path. The contract is an optimization, never a correctness requirement.

4.3.6 Stage 6: Synthesis, Then Claim Support and Coverage Verification

This prompt *requests* that the LLM ground every claim in the provided evidence; it does not *guarantee* it. A model can still cite a source next to a claim the source does not actually support, or state an uncited claim entirely from parametric knowledge. Treating the prompt instruction as sufficient for evidence closure is exactly the gap the Relevance Gate (candidate-side) leaves on the synthesis side, so it is closed by inspecting the output, not by trusting the instruction:

1. **Claim decomposition.** The generated answer a is segmented into atomic claims $\{c_1, \dots, c_m\}$ (one checkable factual statement each).
2. **Claim Support Gate.** For each claim c_i and its cited evidence unit(s), the same NLI model used in Stage 4 scores $\text{entail}(c_i, u)$ and $\text{contradict}(c_i, u)$. A claim with no cited unit scoring above the entailment threshold, or with any cited unit scoring above the contradiction threshold, fails this gate.
3. **Evidence Coverage Gate.** Coverage is $|\{c_i : c_i \text{ passes Claim Support}\}|/m$. An answer below a configured coverage threshold is not returned as-is: failing claims are either re-synthesized against $\mathcal{E}$, dropped with an explicit “unsupported” marker, or the whole answer is downgraded to “insufficient evidence,” depending on deployment policy (§4.7).

The mapping produced by this stage — $\text{claim_id} \rightarrow \{\text{supporting evidence occurrence_ids}\} \rightarrow \text{entailment score} \rightarrow \text{contradiction score}$ — is what **ApprovalStore**/citation UI surfaces to a human reviewer; it is a stronger and independently auditable claim than “the prompt asked for citations.”

4.4 Threshold Calibration: Supervised Score Calibration

The threshold τ must be calibrated to achieve desired precision/recall tradeoff. Uncalibrated cross-encoder scores are not probabilities. Unlike §4.4.2’s Negative-Only FAR Estimation, every method below requires a labeled validation set with positive examples — this is *Supervised Score Calibration* in the terminology of §4.4.2, named separately because it answers a different question (what is the calibrated probability of relevance) and has a different cost (annotation) than the label-free procedure.

4.4.1 Calibration Methods

1. **Isotonic Regression:** Non-parametric, fits piecewise constant function. Requires validation set $\mathcal{D}_{val} = \{(q, u, y)\}$ where $y \in \{0, 1\}$ is relevance judgement.
2. **Platt Scaling:** Fits logistic regression $\sigma(As + B)$ to scores s . Simpler, less flexible.
3. **Temperature Scaling:** Single parameter $T > 0$, calibrated $\sigma(s/T)$. Most efficient.

Algorithm 12 Calibration Procedure

Require: Validation set $\mathcal{D}_{val} = \{(q_i, u_i, y_i)\}$, cross-encoder f

Ensure: Calibrated threshold τ^* for target precision P_{target}

- 1: $scores \leftarrow [f(q_i, u_i) \text{ for } (q_i, u_i, y_i) \in \mathcal{D}_{val}]$
 - 2: $labels \leftarrow [y_i \text{ for } (q_i, u_i, y_i) \in \mathcal{D}_{val}]$
 - 3: Fit calibrator C on $(scores, labels)$ $\triangleright$ Isotonic/Platt/Temperature
 - 4: $cal_scores \leftarrow C(scores)$
 - 5: $\tau^* \leftarrow$ threshold achieving P_{target} on $\mathcal{D}_{val}$
 - 6: **return** τ^*, C
-

Calibration artifacts are stored in `.llmwiki/evidence/cross-encoder/calibration.json` and loaded at startup.

4.4.2 Empirical Null Thresholding: Negative-Only FAR Estimation

This section, §4.4.1 (*Supervised Score Calibration*: isotonic/Platt/temperature, requires labeled $\mathcal{D}_{val}$), and §4.4.3 (*Coverage Calibration*: conformal prediction) are three procedures that all produce a number that looks like a calibrated confidence, and are not interchangeable. This section’s procedure — discordant-pairing against an empirical null — is named *Negative-Only FAR Estimation* throughout to keep it distinct from the other two: it is built entirely from constructed negatives (Definition 4.7) and answers only “how often would a score this high occur from unrelated evidence,” never “what fraction of admitted units are actually relevant” (that is precision, which needs the positive-side distribution) or “what fraction of true positives survive” (that is recall/FDR, which needs labeled positives too). The paragraph below states this limitation explicitly rather than letting the similarity of the numbers invite the conflation.

Algorithm 12 requires $\mathcal{D}_{val}$ with human relevance judgements y_i . This is the binding practical constraint on every deployment: relevance labels are expensive, corpus-specific, and stale the moment the corpus changes. A threshold that cannot be recalibrated without re-annotation is a threshold that will silently drift.

Gravitational-wave astronomy confronts a structurally identical problem — assigning significance to candidate events when no labelled catalogue of true detections exists — and solves it without labels. The technique is *time slides* [Was et al., 2010, Nitz et al., 2018]: triggers from one detector are shifted in time by more than the inter-detector light travel time, so that any coincidence formed from the shifted streams *cannot* correspond to a real astrophysical signal. Repeating the shift many times yields a large empirical sample of pure-noise coincidences, from which the false alarm rate (FAR) is read off as a function of the ranking statistic. Detection thresholds are then quoted in operational units — LIGO’s public alert threshold corresponds to approximately one false alarm per two months.

We construct the same empirical null for evidence scoring.

Definition 4.7 (Discordant Pair). A discordant pair (q, u^-) is a query paired with a unit drawn such that genuine relevance is excluded by construction, not by judgement. Admissible constructions are:

1. **Corpus displacement:** u^- is drawn from a repository disjoint in domain from the one q was issued against.
2. **Temporal displacement:** u^- is drawn from a WAL snapshot predating the creation of every entity referenced by q , so no unit in that snapshot can describe them.
3. **Graph displacement:** u^- is drawn from a connected component of the knowledge graph containing no entity mentioned in q .

Algorithm 13 False Alarm Rate Estimation without Relevance Labels

Require: Query sample Q , corpora $\{\mathcal{U}_1, \dots, \mathcal{U}_m\}$, scorer f , displacement count N

Ensure: Empirical null $\hat{F}_0$, threshold τ^* for target FAR*

```

1:  $S_0 \leftarrow \emptyset$ 
2: for  $i = 1$  to  $N$  do
3:   for all  $q \in Q$  do
4:      $u^- \leftarrow \text{DRAWDISCORDANT}(q, \{\mathcal{U}_j\})$  ▷ Def. 4.7
5:      $S_0 \leftarrow S_0 \cup \{f(q, u^-)\}$ 
6:  $\hat{F}_0(s) \leftarrow |\{s' \in S_0 : s' \geq s\}| / |S_0|$  ▷ Empirical survival function
7:  $\tau^* \leftarrow \min\{s : \hat{F}_0(s) \cdot \bar{k} \leq \text{FAR}^*\}$  ▷  $\bar{k}$  = mean candidates per query
8: return  $\hat{F}_0, \tau^*$ 
    
```

Reporting. Every admitted unit carries not a raw score but a false alarm rate: “*evidence of this strength arises from an unrelated corpus once per $1/(\hat{F}_0(s)\bar{k})$ queries.*” This is an operationally meaningful quantity that a threshold of 0.72 is not.

Invariant 4.8 (Operational Significance). No unit enters the synthesis context without an associated FAR computed against the current empirical null. Recalibration is triggered whenever the WAL advances past a configured number of commits, and requires no human annotation.

Limitations, stated plainly. Two are inherited directly from the source domain and one is specific to ours.

- **Null contamination.** Time slides work because the light travel time bound is a physical guarantee. Our displacements are weaker guarantees: a unit from a “disjoint” repository may still be generically relevant (a logging idiom, a licence header). Contaminated discordant pairs inflate the null’s upper tail and make the gate conservative. Domain disjointness must be asserted from repository metadata, never assumed.
- **Estimator saturation.** The gravitational-wave literature reports that FAR estimation error saturates with the number of time shifts [Was et al., 2010]; beyond a point, additional displacements re-use the same triggers and add no independent information. The same ceiling applies to N in Algorithm 13, and we therefore treat N as tuned to the saturation point rather than maximised.

- **Null $\neq$ alternative.** FAR bounds the rate of *spurious* admission. It says nothing about recall. FAR calibration therefore complements, and does not replace, the labelled conformal procedure of §4.4.3: the empirical null fixes the operating point without labels, and conformal prediction supplies coverage guarantees when labels are available.

4.4.3 Confidence Intervals: Coverage Calibration

Beyond point threshold, we provide confidence intervals per prediction. This is *Coverage Calibration* in the terminology of §4.4.2: like Supervised Score Calibration above, conformal prediction requires labeled examples, but it targets a distribution-free coverage guarantee on intervals rather than a single calibrated point score.

- **Conformal Prediction:** Split conformal for distribution-free coverage guarantees
- **MC Dropout:** Enable dropout at inference, multiple forward passes $\rightarrow$ uncertainty
- **Ensemble:** Multiple cross-encoders $\rightarrow$ variance as uncertainty

Default: Temperature scaling + conformal prediction with 95% coverage.

4.5 Global Consistency of Evidence (Experimental Research Hypothesis)

Status: this section is a research hypothesis with a worked-out mathematical construction, not a shipped or empirically validated mechanism. It is written at specification-level precision so it is falsifiable (§4.5.1 states exactly what would falsify it and what remains unspecified), not because it has been implemented and measured. §4.5.1 and Vol. 11 §12.4.4 track what validating it requires.

Everything specified so far verifies each unit *independently* against the query. This leaves a hole that no amount of per-unit accuracy can close: two units may each be strongly relevant to q and yet assert incompatible things about it. A gate built from pairwise scores admits both, the synthesis stage receives a contradictory evidence set, and the resulting answer is fluently grounded in mutually exclusive sources. Evidence closure as defined in §4.1 is a statement about provenance, not about coherence, and is therefore not yet closed.

The obstruction to assembling locally valid data into a globally valid whole is the subject of sheaf theory, where it is not a metaphor but the definition. A sheaf assigns data to open sets and specifies restriction maps between them; the global sections H^0 are exactly the consistent global assignments, and the first cohomology H^1 measures the obstruction to gluing local data that is pairwise compatible into a global section. Cellular sheaves make this computable over graphs, and recent work has applied the construction to distributed computation — where terminating solutions to a task are precisely the global sections of a task sheaf [Felber et al., 2025] — to consistency constraints on networks [Ghrist and Ding, 2025], and to characterising irreducible error patterns in predictive coding networks [Seely, 2025].

Definition 4.9 (Evidence Sheaf). Let $\mathcal{E}$ be the set of units surviving the proofreading cascade, and let $G_{\mathcal{E}}$ be the graph with vertices $\mathcal{E}$ and an edge $\{u, v\}$ whenever u and v share an entity or stand in a graph relation. Fix a coefficient field $k = \mathbb{R}$. For each

vertex u , let C_u be the finite set of atomic claims extracted from u (§4.3.6); the stalk is $\mathcal{F}(u) = k^{C_u}$, a vertex’s truth-assignment *simplex* embedded linearly (coordinate c of a vector holds the model’s confidence that claim c holds, not a boolean). For each edge $\{u, v\}$, let $C_{uv} = C_u \cap_{\text{sem}} C_v$ be the claims shared between u and v under semantic matching (paraphrase/entity-overlap clustering, not string identity); the stalk is $\mathcal{F}(uv) = k^{C_{uv}}$. The restriction map $\mathcal{F}_{u \leq uv} : \mathcal{F}(u) \rightarrow \mathcal{F}(uv)$ is the linear projection onto the shared coordinates, composed with the NLI model’s entailment/contradiction/neutral distribution at those coordinates: entailment maps a claim’s confidence forward unchanged, contradiction maps it to its negation ($x \mapsto 1 - x$), and neutral maps it to 0 (abstains the coordinate from the gluing constraint). This linearization is a modeling choice, not a derivation from the NLI model’s output, and §4.5.1 states what it assumes away.

Definition 4.10 (Evidence Closure, Cohomological Form). Let $d : C^0(G_{\mathcal{E}}; \mathcal{F}) \rightarrow C^1(G_{\mathcal{E}}; \mathcal{F})$ be the sheaf coboundary map, $H^0 = \ker d$, $H^1 = \text{coker } d$. These are *not* equivalent statements and neither alone answers the question this gate needs answered:

- $H^0 \neq \emptyset$ (equivalently, $\dim \ker d > 0$) says a globally consistent vertex-assignment *exists somewhere* in the stalk space — it does not say the assignment the units actually asserted is that one.
- $H^1 \neq 0$ says d is not surjective — some hypothetical edge data is not induced by any global section — which is a property of the sheaf’s shape (its overlap graph and restriction maps), not of any particular observed assignment.

The question this gate actually needs is neither: given the *specific* edge data $s \in C^1$ that the NLI restriction maps produced from the retrieved units, does s arise from a global section? Equivalently, is $s \in \text{im } d$, i.e. does its class $[s] \in H^1$ (via the quotient $C^1 \twoheadrightarrow \text{coker } d$) vanish. An evidence set $\mathcal{E}$ is *closed* for query q iff $[s] = 0$ and the resulting global section’s restriction to the answer’s cited claims matches the answer; it is *obstructed* iff $[s] \neq 0$, and the obstruction class’s support (which coordinates of s are responsible) is the conflict witness returned to the user. This is the $\delta(s) \neq 0$ test, not the unconditional claim “ $H^1(\mathcal{F}) \neq 0$ ”; Algorithm 14’s COMPUTEH0 step already computes this correctly (it tests the specific assignment, not sheaf-wide emptiness) and the definition above is stated to match the algorithm rather than the informal claim in earlier drafts of this volume.

Algorithm 14 Consistency Gate

Require: Verified units $\mathcal{E}$, query q

Ensure: Coherent subset $\mathcal{E}^*$ or abstention with conflict witness

- 1: $G_{\mathcal{E}} \leftarrow \text{BUILD OVERLAP GRAPH}(\mathcal{E})$
 - 2: $\mathcal{F} \leftarrow \text{BUILDEVIDENCESHEAF}(G_{\mathcal{E}})$ ▷ Def. 4.9
 - 3: $Z \leftarrow \text{COMPUTE H0}(G_{\mathcal{E}}, \mathcal{F})$ ▷ Consistent global assignments
 - 4: **if** $Z \neq \emptyset$ **then**
 - 5: **return** $\mathcal{E}$ ▷ Closed; proceed to synthesis
 - 6: $B \leftarrow \text{COMPUTE H1 SUPPORT}(G_{\mathcal{E}}, \mathcal{F})$ ▷ Edges carrying the obstruction
 - 7: $\mathcal{E}^* \leftarrow \text{MAX CONSISTENT SUBSET}(\mathcal{E}, B)$
 - 8: **if** $\text{SUPPORTS}(\mathcal{E}^*, q)$ **then**
 - 9: **return** $\mathcal{E}^*$ with conflict report B
 - 10: **return** ABSTAIN(“contradictory evidence”, witness B)
-

Why cohomology rather than pairwise conflict detection. A pairwise contradiction check finds edges where two units disagree. It cannot detect *cyclic* inconsistency, in which every pair is individually compatible but no assignment satisfies the cycle — the situation where u is compatible with v , v with w , and w with u , yet the three cannot hold simultaneously. This is precisely the regime H^1 was invented to measure, and it is the regime in which a system reporting high pairwise confidence is most confidently wrong. The cohomology computation additionally *localises* the failure: the support of the obstruction class identifies which relations carry it, so the system can report a conflict witness rather than an unexplained refusal.

Cost. For the linear sheaves used here, H^0 and H^1 reduce to kernel and cokernel computations on the sheaf Laplacian, polynomial in $|\mathcal{E}|$. Since $|\mathcal{E}|$ after the cascade is small by construction (§4.3.4), the consistency gate is not the bottleneck; the NLI evaluations that populate the restriction maps are, and they are bounded by the number of overlap edges.

4.5.1 What This Construction Does Not Yet Specify

Definitions 4.9–4.10 are precise enough to implement and to falsify, which is the point of stating them formally rather than only informally. Precision makes the remaining gaps visible instead of hiding them in prose:

- **Linearization is a modeling choice, not a derivation.** Truth assignment to a claim is naturally a boolean or a bounded probability, not a vector space element; embedding it in $\mathbb{R}^{C_u}$ and mapping contradiction to $x \mapsto 1 - x$ is one convenient encoding among several (a simplex/statistical-manifold embedding, or working with the sheaf of sets directly via lifting to a groupoid, are alternatives with different obstruction theories). Results obtained under this encoding may not transfer to another.
- **Uncertainty is not propagated.** NLI models output calibrated-or-not probabilities, not certainties; the current construction takes a point estimate as the restriction map’s output and does not carry the NLI model’s own confidence into the obstruction computation, so a low-confidence contradiction and a high-confidence one contribute identically to $[s]$.
- **Only linear (additive) inconsistency is representable.** Genuine logical relationships between claims — entailment chains, exclusive-or structure, quantified statements — are not in general expressible as a linear restriction map; the construction here can miss a nonlinear contradiction that a symbolic or constraint-based checker would catch, and can in principle also flag a linear “obstruction” that a full logical reading would resolve as non-contradictory.
- **The overlap graph itself is a heuristic.** $G_{\mathcal{E}}$ ’s edges (shared entity or graph relation) determine which claims are even compared; two units that contradict each other without sharing an extracted entity produce no edge and are invisible to this gate regardless of the cohomology.

None of these gaps invalidate the construction; they are exactly what §4.5’s status note asks a reader to weigh before treating cohomological closure as validated rather than specified, and what the metrics defined in Vol. 11 §12.4 (cyclic inconsistency detection, FAR validity) are designed to test empirically once LLMWikiBench exists.

4.6 Query Planner Integration

The Query Planner (Vol. 07) decomposes complex queries into subqueries, each executed through the retrieval + evidence gate pipeline:

Query: "How does the payment module handle refunds?"

Plan:

Subquery 1: "refund processing logic" -> Search + Gate -> [E1, E2]

Subquery 2: "refund API endpoints" -> Search + Gate -> [E3]

Subquery 3: "refund error codes" -> Search + Gate -> [E4, E5]

Synthesis: Combine [E1..E5] with citations

Each subquery has its own budget and can use different retrieval parameters (e.g., more graph hops for API questions, more vector weight for logic questions).

4.7 Evidence Gate Config

```
[evidence_gate]
model = "cross-encoder/ms-marco-MiniLM-L-6-v2"
threshold = 0.72                # calibrated threshold
calibration = "isotonic"        # "none" | "platt" | "isotonic" |
    ↪ "temperature"
batch_size = 16
max_candidates = 50              # max inputs to gate
max_verified = 20               # max outputs from gate
timeout_ms = 2000               # per-query gate timeout

# Conformal prediction
conformal = true
coverage = 0.95
calibration_set = "eval/evidence_calibration.jsonl"
```

4.8 Evaluation Metrics

The Evidence Gate is evaluated on:

4.9 Comparison with Prior Art

The final two columns are the ones without prior occupants. No deployed retrieval system reports significance against an empirical null constructed without relevance labels, and none certifies that its returned evidence set is mutually consistent rather than merely individually relevant.

Table 4.2: Evidence Gate Evaluation Metrics

Metric	Definition	Target
Precision@k	$\frac{ \{u \in \text{top-}k(\mathcal{E}): y_u=1\} }{k}$	> 0.85
Recall@k	$\frac{ \{u \in \mathcal{E}: y_u=1\} }{ \{u: y_u=1\} }$	> 0.75
F1	Harmonic mean of P/R	> 0.80
AUC-ROC	Area under ROC curve	> 0.90
ECE (Expected Calibration Error)	$\sum_b \frac{ B_b }{N} acc(B_b) - conf(B_b) $	< 0.05
Conformal Coverage	Empirical coverage of prediction sets	≈ 0.95
Empty Rate	$\frac{ \{q: \mathcal{E}(q)=\emptyset\} }{ Q }$	< 0.10

Table 4.3: Retrieval + Verification in Prior Art

System	Hybrid Search	Cross-Enc.	Mandatory Gate	Label-Free FAR	Global Consistency
Standard RAG	✗(vec only)	✗	✗	✗	✗
RAG with Reranker	✓(vec)	✓	✗	✗	✗
GraphRAG	✓(vec+graph)	✗	✗	✗	✗
HippoRAG	✓(graph PPR)	✗	✗	✗	✗
LightRAG	✓(dual-level)	✗	✗	✗	✗
REALM/RA-CM3	✗	✗	✗	✗	✗
LLMWiki	✓(3-signal)	✓(cascade)	✓	✓	✓(H^1)

4.10 Patent-Relevant Novelty

Patent Claim 4.11. Claim 6: A retrieval system comprising: (a) a hybrid searcher combining vector similarity, keyword matching, and graph adjacency traversal, (b) an evidence gate that applies a cross-encoder to every retrieval candidate, (c) a calibrated threshold that rejects candidates below a precision-targeted score, and (d) a synthesis stage that only cites units passing the evidence gate, ensuring evidence closure.

Patent Claim 4.12. Claim 7: The system of Claim 6 wherein the evidence gate operates as a filter (not a reranker), such that zero candidates may be returned when no evidence meets the threshold, triggering an explicit “insufficient evidence” response rather than hallucinated synthesis.

Patent Claim 4.13. Claim 8: The system of Claim 6 wherein the evidence gate comprises a cascade of $n \geq 2$ verification stages reading mutually distinct evidence channels, each stage irreversibly discarding candidates below a stage-specific threshold, ordered by increasing unit cost, such that the false-acceptance rate decays multiplicatively in the number of stages while the expected cost per candidate remains bounded by the survival-weighted sum of stage costs.

Patent Claim 4.14. Claim 9: The system of Claim 6 wherein the threshold is derived without relevance annotations by (a) constructing query–unit pairs for which relevance is excluded by corpus, temporal, or graph displacement, (b) accumulating scorer outputs over said pairs into an empirical null distribution, and (c) selecting the threshold attaining a target false alarm rate under said distribution, the false alarm rate being reported with each admitted unit.

Patent Claim 4.15. Claim 10: The system of Claim 6 further comprising a consistency gate that constructs a cellular sheaf over the verified evidence set with restriction maps induced by inference between overlapping claims, computes the obstruction class of the specific edge data induced by said inference (as distinct from testing whether the sheaf’s cohomology is nonzero independent of that data), and abstains from synthesis when said obstruction class is nonzero, emitting its support as a conflict witness. *[Draft language pending patent counsel review — see §11.9 discussion note; not yet filed.]*

Patent Discussion 4.16. Prior art (rerankers, RAG, GraphRAG) never enforces evidence closure: they always return top- k for synthesis. The mandatory gate with calibrated threshold is a structural invariant, not a heuristic.

Claims 8–10 narrow the invention away from the “combination of known components” objection that a bare hybrid-search-plus-reranker claim invites. Claim 8 recites a specific cascade discipline — irreversible discard, channel-distinct stages, cost ordering — with a stated quantitative consequence, rather than generic multi-stage filtering. Claim 9 recites a construction procedure for the null distribution, which is the non-obvious element: the displacement conditions are what make the estimate valid, and no retrieval prior art constructs relevance nulls at all. Claim 10 recites a topological consistency criterion that is not reducible to pairwise contradiction checking, since cyclic inconsistency is invisible to any pairwise test.

4.11 Summary

The retrieval pipeline transforms a query into a verified, mutually consistent evidence set:

1. **Hybrid Search:** Vector (HNSW) + Keyword (BM25) + Graph (PPR/adjacency)
2. **Proofreading Cascade:** n -stage irreversible verification over distinct evidence channels; error suppression multiplicative in depth
3. **Calibration:** label-free FAR from an empirical null (§4.4.2), complemented by con-formal coverage when labels exist
4. **Consistency Gate:** sheaf cohomology over the surviving set; $H^1 \neq 0 \Rightarrow$ abstain with conflict witness
5. **Context Selection:** deterministic top- k baseline; budgeted concept coverage remains feature-gated pending answer-quality A/B
6. **WC/1 Transport:** lossless evidence text, compact integer handles, host-side occurrence/span resolution
7. **Forced Citations:** Synthesis only from verified units, followed by claim support and coverage checks
8. **Evidence Closure:** cohomological form (Def. 4.10); every citation traces to a verified unit admitting a consistent global reading

This pipeline is the enforcement mechanism for LLMWiki’s core invariant: *answers are evidence-bounded*. The strengthening over the volume’s opening statement is deliberate — evidence closure begins as a provenance property and ends as a verifiable topological one.

Agent Runtime

This volume specifies the *agent runtime*—the execution environment that carries out query plans, orchestrates tool calls, manages conversation state, and streams results. The runtime is the bridge between the Query Planner (Vol. 07) and the external world (API, CLI, LLM providers).

5.1 Design Goals

Design Principle 5.1 (Stateless Execution). Each query execution is a self-contained episode. No persistent agent state across queries; all context is passed explicitly in the plan and conversation history.

Design Principle 5.2 (Tool-First Architecture). LLMs are not the primary actors; tools are. The LLM plans and decides; tools execute deterministically. This inverts the typical “agent = LLM + tools” model.

Design Principle 5.3 (Streaming by Default). All long-running operations (search, synthesis, tool calls) stream partial results. The API responds with Server-Sent Events (SSE) or WebSocket frames.

Design Principle 5.4 (Budget Enforcement). Every execution has configurable budgets: token budget, time budget, tool call budget, cost budget. Exceeding any budget terminates with a partial result.

Design Principle 5.5 (Observability Native). Every decision (plan step, tool choice, retrieval result, synthesis token) is logged with structured traces. OpenTelemetry compatible.

5.2 Runtime Architecture

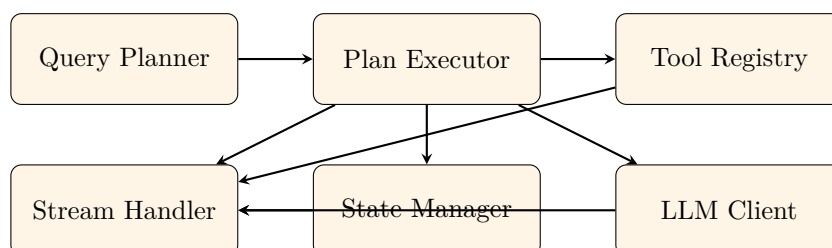


Figure 5.1: Agent Runtime components.

5.2.1 Core Components

- **Plan Executor:** Traverses the query plan DAG, executes nodes in topological order, handles parallelism and dependencies.
- **Tool Registry:** Maps tool names to implementations, handles invocation, timeouts, retries, and result normalization.
- **LLM Client:** Abstracts LLM providers (local: llama.cpp, vLLM, ollama; remote: OpenAI, Anthropic, etc.) with unified streaming interface.
- **State Manager:** Maintains scratchpad, conversation history, intermediate results, and budget counters.
- **Stream Handler:** Emits SSE/WebSocket events for partial results, progress, errors, and final answer.

5.3 Plan Execution Model

5.3.1 Query Plan Structure

The planner (Vol. 07) produces a `QueryPlan`:

```
struct QueryPlan {
    id: PlanId,
    query: String,
    steps: Vec<PlanStep>,
    edges: Vec<(StepId, StepId)>, // DAG dependencies
    budget: ExecutionBudget,
    config: PlanConfig,
}

struct PlanStep {
    id: StepId,
    kind: StepKind,           // Search, Synthesize, ToolCall, Aggregate,
    ↪ Branch
    tool: Option<ToolName>,
    input: StepInput,         // Template with variable interpolation
    output: StepOutput,       // Expected output schema
    depends_on: Vec<StepId>,
    parallel: bool,
    retry: RetryPolicy,
    timeout_ms: u64,
}
```

5.3.2 Execution Algorithm

Algorithm 15 Plan Execution

Require: Plan P , initial state S_0

Ensure: Final state S_f with answer

```

1:  $S \leftarrow S_0$ ,  $completed \leftarrow \emptyset$ 
2:  $ready \leftarrow \{s \in P.steps \mid s.depends\_on \subseteq completed\}$ 
3: while  $ready \neq \emptyset$  do
4:   if  $\exists s \in ready : s.parallel$  then
5:      $batch \leftarrow \{s \in ready \mid s.parallel\}$ 
6:      $results \leftarrow EXECUTEPARALLEL(batch, S)$ 
7:   else
8:      $s \leftarrow POP(ready)$ 
9:      $results \leftarrow EXECUTESTEP(s, S)$ 
10:   $S \leftarrow UPDATESTATE(S, results)$ 
11:   $completed \leftarrow completed \cup \{s.id \mid s \in batch \vee s\}$ 
12:   $ready \leftarrow \{s \in P.steps \setminus completed \mid s.depends\_on \subseteq completed\}$ 
13:  CHECKBUDGET( $S$ ,  $P.budget$ )  $\triangleright$  May raise BudgetExceeded
14:  EMITPROGRESS( $S$ ,  $completed$ )  $\triangleright$  Stream update
15: return  $S$ 

```

5.3.3 Step Kinds

- **Search:** Invoke hybrid search + evidence gate (Vol. 04)
- **Synthesize:** Select verified evidence under budget, render WC/1, call LLM, resolve citations, stream tokens
- **ToolCall:** Invoke registered tool (see §5.6)
- **Aggregate:** Merge results from parallel branches
- **Branch:** Conditional execution based on prior result

5.4 Evidence Context Assembly

Synthesize does not accept an arbitrary string assembled by the planner. It accepts verified evidence records and delegates prompt construction to a deterministic context builder:

1. return **REFUSE** immediately when the evidence gate is empty;
2. select a subset under the remaining token budget (score-ordered top- k by default; experimental concept-coverage packing only when enabled);
3. serialize the subset through WC/1, preserving evidence text verbatim and retaining the integer-ID resolution map in host state;
4. invoke the provider-specific LLM client with the cacheable contract prefix separated from the query-specific suffix;

5. resolve cited IDs to source occurrences and run post-generation claim support and evidence-coverage checks (Vol. 04 §4.3.6).

The context builder is provider-agnostic. Providers that expose explicit prompt cache controls may pin the fixed WC/1 prefix; providers that cache implicitly still benefit from byte stability. Contract evolution is append-only within a version so the longest common prefix remains stable. An unknown version or failed capability negotiation takes the JSON fallback path.

```
struct EvidenceContext {
  wire_version: "WC/1",
  cache_prefix: String,
  query_suffix: String,
  id_map: HashMap<u32, EvidenceOccurrence>,
  prompt_tokens: usize,
  context_budget: usize,
}
```

The `id_map` never crosses the model boundary. It is retained in `ExecutionState` until citation verification completes, then emitted in the trace as the exact model-handle to source-span resolution used for the answer. This gives citation replay a stable, auditable boundary without sending UUIDs to the model.

5.5 Hierarchical Conversation Memory

Long-lived Discord channels are not reconstructed by resending their full history. The runtime maintains a slow, durable event tier and a small, model-visible memory tier, following recursive summarisation and external-memory designs [Wang et al., 2023, Packer et al., 2023, Yin et al., 2024]:

$$M_{n+1} = \text{Summarize}(M_n, E_{(s_n, s_{n+1}]}, \text{Retrieve}(q)).$$

M_n is the prior project or channel memory, s_n is the last successfully committed summary checkpoint, and $E_{(s_n, s_{n+1}]}$ is only the newly observed event interval. The checkpoint advances after the summary files are durably written. A provider failure therefore retries the same interval rather than silently dropping discussion.

Discord message bodies and attachment text are untrusted prompt data. They may be summarized as evidence but cannot redefine the system policy, invoke tools, or authorize mutations. Summaries preserve `discord:<channel>:<message>` citations and are written as separate projections for decisions, tasks, unresolved questions, and recent activity. The provider interface permits either a local model or the OpenAI Responses API; the authoritative journal, lexical search, and Markdown export remain available without either provider.

5.6 Tool System

5.6.1 Tool Trait

```
trait Tool {
  fn name(&self) -> &str;
```

```

fn description(&self) -> &str;
fn schema(&self) -> ToolSchema;           // JSON Schema for input
fn output_schema(&self) -> ToolSchema;    // JSON Schema for output
fn execute(&self, input: ToolInput) -> Result<ToolOutput>;
fn execute_stream(&self, input: ToolInput) -> Stream<ToolOutput>;
fn timeout_ms(&self) -> u64 { 30000 }
fn retry(&self) -> RetryPolicy { RetryPolicy::default() }
}

```

5.6.2 Built-in Tools

Table 5.1: Built-in Tools

Tool	Input	Output
search	SearchInput{query, k, filters}	SearchOutput{results[], citations[]}
graph_traverse	TraverseInput{start, pattern, hops}	Subgraph{nodes[], edges[]}
read_unit	UnitId	SemanticUnit
read_file	FilePath	FileContent
llm_complete	CompletionInput{prompt, params}	Stream<String>
extract_entities	ExtractInput{text, types}	Entities{entities[], relations[]}
summarize	SummarizeInput{units[], max_len}	String
verify_evidence	VerifyInput{query, units[]}	VerifiedUnits{verified[]}

5.6.3 Custom Tools

Users register custom tools via config:

```

[tools.custom.my_tool]
module = "my_tools"
class = "MyTool"
timeout_ms = 10000
retry = { max_attempts = 2, backoff_ms = 500 }

```

Tools can be synchronous (return `Result`) or streaming (return `Stream`). Streaming tools emit partial results for long operations (e.g., file scanning, API pagination).

5.7 LLM Client Abstraction

```

trait LLMClient {
  fn complete(&self, req: CompletionRequest) ->
    ↪ Result<CompletionResponse>;
  fn stream(&self, req: CompletionRequest) -> Stream<CompletionChunk>;
  fn embed(&self, texts: Vec<String>) -> Result<Vec<Vec<f32>>>;
  fn models(&self) -> Vec<ModelInfo>;
  fn count_tokens(&self, text: &str) -> usize;
}

```

Providers:

- **Local:** llama.cpp (server), vLLM, ollama, TGI
- **Remote:** OpenAI, Anthropic, Azure, Vertex, Bedrock
- **Unified API:** OpenAI-compatible endpoint

The runtime selects the provider per plan step (e.g., small local model for planning, large model for synthesis).

5.8 State Management

```
struct ExecutionState {
    plan_id: PlanId,
    step_results: HashMap<StepId, StepResult>,
    scratchpad: Scratchpad,           // Key-value store for LLM use
    conversation: Vec<Message>,      // User/assistant messages
    evidence: Vec<VerifiedUnit>,      // Accumulated evidence
    budget: BudgetTracker,            // Tokens, time, cost, calls
    trace: TraceContext,              // OpenTelemetry trace
}
```

Scratchpad: Structured key-value store accessible to LLM via tool calls. Used for intermediate reasoning, variable passing between steps, and user-facing “thoughts”.

Budget Tracker: Hard limits with configurable actions on exceed:

```
budget:
    max_tokens: 100000
    max_time_ms: 120000
    max_tool_calls: 50
    max_cost_usd: 0.50
    on_exceed: "truncate" # "error" | "truncate" | "partial"
```

5.9 Budget as Allocation, Not Truncation

The budget tracker of §5.8 is a circuit breaker: it observes consumption and terminates when a limit is crossed. It never influences *which* work is done, only when work stops. Under a binding budget this is close to the worst available policy, because the steps that happen to be scheduled early consume the budget regardless of their value, and the terminating action — truncate, partial, error — degrades an answer that a different ordering might have completed.

Time-domain astronomy operates permanently in this regime and does not treat it as an error condition. The Vera C. Rubin Observatory’s survey will produce on the order of 10^4 transient alerts per night against spectroscopic follow-up capacity of order 10^1 ; the deficit is four orders of magnitude and is structural, not a provisioning failure. The response is a broker layer that ranks candidates in real time under hard resource constraints. Sedgewick et al. [2025] score candidates and fold in observability to produce a continually updated priority list; the Fink broker applies real-time active learning to select the follow-up that most improves classification rather than the one with the highest

current score [Möller et al., 2025]; and Zhang et al. [2025] formulate follow-up scheduling as an online resource-constrained problem solved by reinforcement learning.

The third of these is the important one. Ranking by score selects what the system already believes; ranking by expected information gain selects what would change its belief.

Design Principle 5.6 (Budget Allocation over Budget Enforcement). The execution budget is an allocation problem solved continuously during execution, not a limit checked after the fact. Steps compete for budget on expected evidential gain per unit cost, and the enforcement action is the degenerate case reached when no step clears the bar.

Definition 5.7 (Evidential Value Density). For a candidate step s with estimated cost $c(s)$ in budget units, let $\Delta H(s)$ be the expected reduction in uncertainty over the answer induced by executing s . The value density is $\nu(s) = \Delta H(s)/c(s)$.

Uncertainty here is not a free parameter: the evidence gate already produces a calibrated quantity. Using the false alarm rates of §4.4.2, the current evidence set induces a distribution over admissible answers, and $\Delta H(s)$ is estimated as the expected decrease in its entropy — the step most worth taking is the one whose result is least predictable from what is already verified.

Algorithm 16 Budget-Aware Step Scheduling

Require: Plan DAG P , budget B , state S

Ensure: Answer or partial result with accounting

```

1:  $F \leftarrow \text{READYFRONTIER}(P, S)$  ▷ Steps with satisfied dependencies
2: while  $F \neq \emptyset$  and  $B$  not exhausted do
3:   for all  $s \in F$  do
4:      $\hat{c}(s) \leftarrow \text{ESTIMATECOST}(s)$  ▷ From historical step statistics
5:      $\widehat{\Delta H}(s) \leftarrow \text{ESTIMATEGAIN}(s, S)$  ▷ Entropy of answer distribution
6:      $s^* \leftarrow \arg \max_{s \in F} \widehat{\Delta H}(s)/\hat{c}(s)$ 
7:     if  $\widehat{\Delta H}(s^*) < \delta_{\min}$  then
8:       break ▷ No step justifies its cost; stop early and under budget
9:      $S \leftarrow \text{EXECUTE}(s^*, S)$ ;  $B \leftarrow B - \text{ACTUALCOST}(s^*)$ 
10:     $F \leftarrow \text{READYFRONTIER}(P, S)$  ▷ Re-plan: results may prune or add steps
11: return  $\text{SYNTHESIZE}(S)$ ,  $\text{BUDGETREPORT}(B)$ 
```

Two consequences distinguish this from enforcement. First, the loop can terminate *under* budget when no remaining step earns its cost, which a circuit breaker never does — it spends everything it is given. Second, when the budget does bind, what survives is the highest-value-density prefix rather than an arbitrary prefix, so the partial result is the best obtainable at that price rather than whatever happened to run first.

Observation 5.8 (Estimator Quality Bounds the Gain). Algorithm 16 is only as good as $\widehat{\Delta H}$ and $\hat{c}$. Cost estimation is tractable, since step costs are empirically regular and improve with logged history. Gain estimation is genuinely hard and a poor estimator degrades the schedule toward arbitrary ordering — which is to say, toward the behaviour of the circuit breaker it replaces. The failure mode is therefore bounded below by the previous design, which is why the allocation policy can be deployed incrementally.

5.10 Streaming Protocol

API uses Server-Sent Events (SSE) over HTTP or WebSocket:

```

# Event types
event: plan_start      { "plan_id": "...", "steps": N }
event: step_start      { "step_id": "...", "kind": "Search" }
event: step_progress   { "step_id": "...", "progress": 0.5, "details":
  ↪  "..." }
event: evidence        { "unit_id": "...", "relevance": 0.94, "preview":
  ↪  "..." }
event: token           { "token": "the", "step_id": "..." }
event: step_complete   { "step_id": "...", "output": {...} }
event: plan_complete   { "answer": "...", "citations": [...], "usage":
  ↪  {...} }
event: error           { "step_id": "...", "error": "..." }
event: budget_warning  { "type": "tokens", "used": 80000, "limit": 100000 }

```

Clients can render progressive UI: show plan, show evidence as it arrives, stream answer tokens.

5.11 Error Handling and Retries

- **Transient errors** (network, timeout): Retry per step policy (exponential backoff)
- **Tool errors**: Captured in `StepResult.error`, execution continues if step not critical
- **LLM errors**: Rate limit → backoff; context overflow → truncate/summarize history
- **Budget exceeded**: Emit `budget_warning`, then `plan_complete` with partial answer
- **Fatal errors**: Invalid plan, OOM, provider unavailable → abort with error event

5.12 Conversation Management

Multi-turn conversations maintain history in `ExecutionState.conversation`. The planner can reference prior turns for context. History is pruned to fit budget (summarize old turns, keep recent).

5.13 Patent-Relevant Novelty

Patent Claim 5.9. Claim 8: An agent runtime wherein: (a) query plans are DAGs executed by a deterministic executor with topological scheduling, (b) tools are first-class citizens with JSON Schema contracts, streamable execution, and configurable timeouts/retries, (c) LLM invocation is abstracted behind a provider-agnostic client supporting both local and remote models, (d) execution state includes a structured scratchpad accessible to tools and LLM, and (e) hard budgets (tokens, time, cost, calls) are enforced at the executor level with configurable overflow behavior.

5.14 Summary

The Agent Runtime executes query plans as deterministic, budgeted, observable tool orchestrations:

1. **Plan executor:** DAG-aware, parallel-capable, topological
2. **Tool system:** Schema-contracted, streamable, extensible
3. **LLM abstraction:** Multi-provider, local-first, token-aware
4. **State:** Scratchpad, evidence accumulator, budget tracker
5. **Context assembly:** Verified-evidence selection, WC/1 rendering, host-side citation map
6. **Streaming:** SSE/WebSocket with fine-grained event types
7. **Budgets:** Value-density allocation plus hard limits with graceful degradation

The runtime is the “CPU” of LLMWiki; the Query Planner (Vol. 07) is the compiler that generates its instruction sequences.

Evaluation Framework

This volume defines the evaluation methodology for LLMWiki: benchmarks, metrics, experimental protocols, and the automated evaluation pipeline. Rigorous evaluation is not an afterthought; it is a first-class component that validates the architectural claims (evidence closure, deterministic replay, local-first quality).

Implementation status: a runnable harness scaffold exists at `eval/` in the project repository — working implementations of the evidence and context-efficiency metrics in §6.3 (17 metric tests), a real parser producing `content_id/occurrence_id-split` records (Appendix B §A.1) from a small real corpus, a dependency-free BM25 retrieval baseline, an end-to-end pipeline runner, and separate executable references for temporal gating, evidence packing, and WC/1 transport. The complete reference suite passes 91 tests. It is explicitly a scaffold, not the benchmark: `eval/README.md` documents exactly which gaps (corpus scale, two-annotator gold sets, hybrid retrieval, attribution scoring against a real NLI model, the full configuration sweep, and model A/B for context shaping) separate it from `LLMWikiBench` proper, consistent with the status note in Vol. 11 §12.4.4.

6.1 Evaluation Philosophy

Design Principle 6.1 (Measure What Matters). Standard RAG metrics (recall@k, MRR) are insufficient. We evaluate: **evidence quality** (are citations actually relevant?), **answer correctness** (ground truth comparison), **attribution accuracy** (claim-to-evidence mapping), and **system properties** (latency, determinism, replay fidelity).

Design Principle 6.2 (Local-First Evaluation). All evaluation runs locally. No external API calls for ground truth generation or metric computation. LLM-as-judge is optional and confined to specific qualitative assessments.

Design Principle 6.3 (Regression-Proof). Evaluation datasets, prompts, and thresholds are version-controlled alongside code. CI runs full evaluation on every PR; regressions block merge.

6.2 Benchmark Suite

6.2.1 Internal Benchmarks

Construction Process:

Table 6.1: LLMWiki Internal Benchmarks

Benchmark	Domain	Size	Focus
CodeQA-Python	Python codebases	50 repos, 200K units	Code understanding, cross-file references
API-Doc	OpenAPI specs	100 specs, 50K ops	Schema-to-natural-language
SQL-Schema	SQL DDL	200 schemas, 10K tables	Table/column/relationship QA
Tech-Docs	Markdown/PDF	500 docs, 1M tokens	Long-document QA, structure
Config-Files	YAML/JSON/TOML	1000 configs	Config understanding, env vars
Mixed-Corpus	All above	1M units	Cross-domain retrieval

1. **Seed corpora:** Curated real-world repositories (Apache, Kubernetes, etc.)
2. **Question generation:** Template-based + LLM-assisted (local model) for diversity
3. **Expert review:** Domain experts verify questions and ground truth answers
4. **Evidence annotation:** Human annotators mark exact unit spans supporting each answer
5. **Quality control:** Inter-annotator agreement (Cohen’s $\kappa > 0.8$)

6.2.2 External Benchmarks (Adapted)

We adapt standard benchmarks to LLMWiki’s evidence-required setting:

- **Natural Questions (NQ):** Filter to questions answerable from our test corpora
- **HotpotQA:** Multi-hop; verify graph traversal quality
- **MS MARCO:** Passage ranking; compare hybrid search vs. baselines
- **BEIR:** 18 datasets; zero-shot retrieval quality
- **CodeSearchNet:** Code retrieval; evaluate parser+embedder quality
- **Spider / BIRD:** Text-to-SQL; evaluate SQL parser + graph

6.3 Metric Definitions

6.3.1 Retrieval Metrics

Definition 6.4 (Evidence Recall@k). Fraction of ground-truth evidence units recovered in top- k after Evidence Gate:

$$\text{EvRecall@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|\mathcal{E}^*(q) \cap \mathcal{E}_k(q)|}{|\mathcal{E}^*(q)|} \quad (6.1)$$

where $\mathcal{E}^*(q)$ = human-annotated evidence units, $\mathcal{E}_k(q)$ = gate-verified top- k .

Definition 6.5 (Evidence Precision@k).

$$\text{EvPrecision@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|\mathcal{E}^*(q) \cap \mathcal{E}_k(q)|}{|\mathcal{E}_k(q)|} \quad (6.2)$$

Definition 6.6 (Evidence F1@k). Harmonic mean of EvRecall and EvPrecision.

Definition 6.7 (Gate Rejection Rate). Fraction of retrieved candidates rejected by Evidence Gate:

$$\text{GateReject} = \frac{|\mathcal{R}| - |\mathcal{E}|}{|\mathcal{R}|} \quad (6.3)$$

High rejection with maintained EvRecall indicates effective noise filtering.

6.3.2 Answer Quality Metrics

Definition 6.8 (Answer Correctness (Exact Match / F1 / Semantic)). Given ground truth answer a^* and predicted answer $\hat{a}$:

- **EM**: Exact string match (normalized)
- **Token-F1**: Token-level F1
- **Semantic**: Embedding cosine $\geq \tau$ (e.g., 0.85 with BGE-M3)

Definition 6.9 (Attribution Score). Measures whether every claim in the answer is supported by cited evidence.

1. Segment answer into atomic claims $c_1, \dots, c_n$ (via NLI model or LLM)
2. For each claim c_i , check if $\exists e \in \text{citations} : \text{NLI}(e, c_i) = \text{Entailment}$
3. $\text{Attribution} = \frac{1}{n} \sum_i \mathbb{1}[\text{supported}(c_i)]$

Definition 6.10 (Citation Quality). For each cited unit u , score = $\text{CrossEncoder}(q, u)$. Average over all citations. Penalizes “hallucinated citations” (citing irrelevant units).

6.3.3 System Property Metrics

- **Deterministic Replay Fidelity**: Bitwise equality of answers across re-runs (modulo LLM sampling)
- **Ingestion Latency**: ms/unit (p50, p99)
- **Query Latency**: End-to-end p50/p95/p99
- **Index Size**: GB per million units
- **Memory Footprint**: RSS at rest / under load
- **Uptime / Crash Recovery**: Time to replay WAL and serve queries

6.3.4 Context Efficiency Metrics

Token cost is a first-class outcome rather than an incidental runtime counter. Quality and cost are reported together because a smaller prompt that lowers answer quality is not an improvement.

Definition 6.11 (Tokens per Answered Query).

$$\text{TPAQ} = \text{median}\{T_q \mid q \text{ returns ANSWER}\},$$

where T_q is the complete prompt token count, including instructions and evidence framing.

Definition 6.12 (Coverage per Token).

$$\text{CovTok}(q) = \frac{\sum_{c \in C(\mathcal{E}_{B,q})} w(c)}{T_q},$$

where $C(\mathcal{E}_{B,q})$ is the union of weighted concepts in the packed evidence. This is the packer’s proxy objective and must not be reported as answer quality.

Definition 6.13 (Budget Utilisation).

$$\text{Util}(q) = T_q/B_q.$$

Values above one are hard failures; persistently low values indicate either successful early stopping or an estimator that is leaving useful evidence out.

Definition 6.14 (Quality at Budget).

$$Q(\beta) = \text{EvF1} \quad \text{for} \quad \beta \in \{0.25, 0.50, 0.75, 1.00\}.$$

The result is a curve, not a single operating point. Every context policy is compared at equal β .

Definition 6.15 (Refusal Token Cost).

$$\text{RTC} = \text{mean}\{T_q \mid q \text{ returns REFUSE}\}.$$

This exposes systems that eventually refuse correctly but spend a full answer budget discovering that they have no admissible evidence.

6.4 Evaluation Pipeline

6.4.1 Context-Shaping Acceptance Experiment

WC/1 transport and budgeted packing remain behind feature flags until a fixed query set is evaluated in the full 2×2 matrix:

$$\{\text{JSON}, \text{WC/1}\} \times \{\text{top-k}, \text{packed}\}.$$

Each cell uses the same retrieved and gate-verified candidates, reader model, decoding parameters, and budget fractions. Report EvF1, Attribution, TPAQ, RTC, and the complete $Q(\beta)$ curve with bootstrap confidence intervals. WC/1 is accepted only if it meets the token reduction criterion without a statistically meaningful quality regression. Packed selection replaces top- k only if it improves end-to-end quality at equal budget; improved concept coverage alone is insufficient.

6.4.2 Configuration Matrix

We sweep key parameters to understand tradeoffs:

```
# eval_config.yaml
retrieval:
  k: [5, 10, 20, 50]
  hybrid_weights:
```

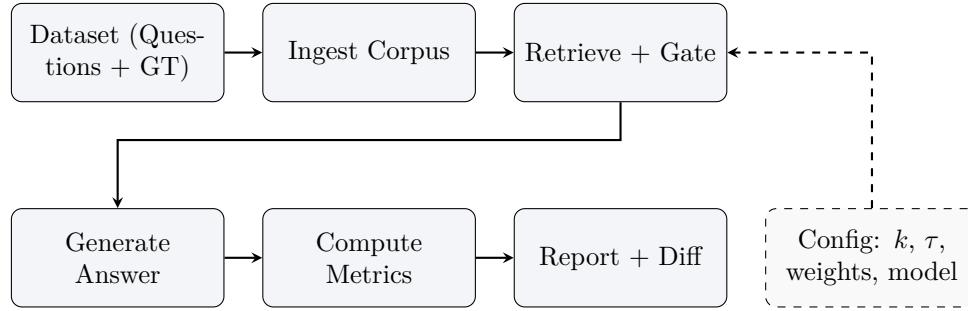


Figure 6.1: Automated evaluation pipeline.

```

- [1.0, 0.0, 0.0] # Vector only
- [0.0, 1.0, 0.0] # BM25 only
- [0.6, 0.3, 0.1] # Default hybrid
gate_threshold: [0.5, 0.6, 0.7, 0.75, 0.8, 0.9]
embedders:
- "bge-m3"
- "e5-large-v2"
- "nomic-embed-text-v1.5"
llms:
- "llama-3.1-8b-instruct" # Local
- "llama-3.1-70b-instruct" # Local (quantized)
- "gpt-4o-mini" # Remote (optional)

```

Each combination = one evaluation run. Results stored in `eval/results/` with full config hash for reproducibility.

6.4.3 Statistical Rigor

- **Bootstrap confidence intervals:** 1000 resamples, 95% CI for all metrics
- **Paired significance:** Wilcoxon signed-rank test for pairwise model comparisons
- **Multiple comparison correction:** Benjamini-Hochberg FDR control
- **Effect sizes:** Cohen's d reported alongside p -values

6.5 Regression Detection

```

# .github/workflows/eval.yml
name: Evaluation
on: [pull_request, push]
jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run evaluation
        run: |
          llmwiki eval run --config eval_config.yaml --baseline main

```

```
- name: Check regressions
  run: |
    llmwiki eval check --threshold 0.02 # 2% regression tolerance
- name: Upload report
  uses: actions/upload-artifact@v4
  with:
    name: eval-report
    path: eval/report.html
```

Baseline is the `main` branch. Any metric regression $> 2\%$ (configurable) blocks merge.

6.6 Qualitative Evaluation

Automated metrics are supplemented by:

1. **Expert review:** Domain experts rate answer quality on 1-5 scale (helpful, accurate, well-cited)
2. **Failure analysis:** Categorize errors (missing evidence, wrong synthesis, gateway false negative, etc.)
3. **Ablation studies:** Remove components (graph, gate, planner) to measure contribution
4. **User studies:** Task completion time, trust ratings, citation usefulness

6.7 Patent-Relevant Novelty

Patent Claim 6.16. Claim 9: An evaluation framework for semantic operating layers characterized by: (a) evidence-centric metrics (Evidence Recall/Precision/F1) that measure retrieval quality after mandatory cross-encoder verification, (b) attribution scoring that decomposes answers into atomic claims and verifies each against cited evidence using NLI, (c) a configuration matrix sweep that characterizes system behavior across retrieval, gating, and synthesis parameters, and (d) deterministic replay verified via bitwise answer comparison across re-runs.

6.8 Summary

LLMWiki’s evaluation framework goes beyond standard RAG benchmarks by:

1. **Evidence-first metrics:** Quality of verified evidence, not just raw retrieval
2. **Attribution verification:** Claim-level NLI checking against citations
3. **Configuration sweeps:** Understanding tradeoffs, not single-point numbers
4. **Context efficiency:** Token cost, coverage per token, utilisation, refusals, and quality-at-budget curves
5. **Regression gates:** CI-enforced quality floors
6. **System properties:** Determinism, latency, replay—all measured

This ensures that architectural improvements (Vol. 01–05) translate to measurable user value.

Deployment and Operations

This volume covers deploying LLMWiki in production: containerization, local-first modes, distributed scaling, monitoring, backup/recovery, and upgrade procedures.

7.1 Deployment Tiers

Everything in this volume, taken together, could be read as v1 scope: Kubernetes, Helm, Vault, Consul, and Ceph-class storage alongside a single binary that runs with zero external services. That reading would contradict the Local-First design principle (Vol. 01 §1.1) and the single-binary deployment goal. This section states explicitly which components belong to which tier, so “supported at largest scale” and “required to run at all” are not conflated. See DESIGN_DECISIONS.md DD-022 for the recorded rationale.

Table 7.1: Deployment Tiers: Cumulative Complexity, Independent Sufficiency

Tier	Use Case	Required	Explicitly Not Required
Local MVP	Individual user, desktop app, CLI, air-gapped single machine	Single binary (llmwiki:cpu/llmwiki:full) or embedded library mode; local .llmwiki/ filesystem	Docker, Kubernetes, Helm, Vault, Consul, Ceph/Longhorn, Redis, any network service
Team Server	Small team, shared server, single-writer repository	Standalone Server mode (Table 7.2): API + workers, Docker container, optional shared read-replica index	Kubernetes, Helm, Vault, Consul, Ceph — a plain volume or NFS mount suffices at this scale; RWX shared storage is an Enterprise Cluster concern (§7.4.3)
Enterprise Cluster	HA production service, multi-writer, compliance requirements	Everything above, plus Kubernetes, Helm (§7.4.1), Vault/Consul for secrets and discovery, Ceph/Longhorn-class RWX storage, load-balanced replicas	—

A new deployment (and a new reader of this volume) should default to Local MVP and add tiers only as actual need — team-scale sharing, then HA/compliance — arises. §7.5 covers Local MVP in detail; §7.4 (Kubernetes/Helm) and the Vault/Consul references in

§7.4.3 and later in this volume are **Enterprise Cluster only** and are marked as such at each occurrence below.

7.2 Deployment Modes

Table 7.2: LLMWiki Deployment Modes

Mode	Use Case	Components	Scale	Deps
Embedded (Library)	Desktop app, CLI, tests	Rust/Python lib	1 process	None
Standalone Server	Single-user, team server	API + workers	10K QPS	Docker
High Availability	Production service	Load balancer + replicas	100K QPS	K8s, Redis
Air-gapped	Secure environments	Full offline bundle	10K QPS	None
Edge / Mobile	On-device	WASM / mobile lib	1 QPS	None

7.3 Container Images

Dockerfile multi-stage build:

```
# Build stage
FROM rust:1.78 AS builder
WORKDIR /app
COPY . .
RUN cargo build --release --features api,embed-onnx

# Runtime stage
FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
    ca-certificates libssl3 libgomp1 && \
    rm -rf /var/lib/apt/lists/*
COPY --from=builder /app/target/release/llmwiki /usr/local/bin/
COPY --from=builder /app/config/default.toml /etc/llmwiki/
EXPOSE 8080
USER 1000:1000
ENTRYPOINT ["llmwiki", "server"]
```

Images published to ghcr.io/llmwiki/llmwiki:version, latest, sha.

7.3.1 Image Variants

- `llmwiki:cpu` - Default, ONNX Runtime CPU
- `llmwiki:cuda` - CUDA 12.x, TensorRT optimized
- `llmwiki:rocm` - AMD ROCm support
- `llmwiki:minimal` - No embedder, external embedding service
- `llmwiki:full` - All parsers, local LLM (llama.cpp)

7.4 Kubernetes Deployment [Enterprise Cluster tier]

This section and its subsections apply to the Enterprise Cluster tier only (§7.1); Local MVP and Team Server deployments do not need any of it.

7.4.1 Helm Chart Structure

```
llmwiki/
|-- Chart.yaml
|-- values.yaml
|-- values-prod.yaml
|-- templates/
|   |-- deployment.yaml
|   |-- service.yaml
|   |-- ingress.yaml
|   |-- configmap.yaml
|   |-- secret.yaml
|   |-- pvc.yaml
|   |-- hpa.yaml
|   |-- servicemonitor.yaml
|   `-- _helpers.tpl
`-- charts/
    |-- redis/          # Optional: for distributed cache
```

7.4.2 Key Configuration

```
# values.yaml
replicaCount: 3
image:
  repository: ghcr.io/llmwiki/llmwiki
  tag: "0.1.0"
  pullPolicy: IfNotPresent

resources:
  limits:
    cpu: "2000m"
    memory: "4Gi"
  requests:
    cpu: "1000m"
    memory: "2Gi"

persistence:
  enabled: true
  size: 100Gi
  storageClass: fast-ssd
  accessMode: ReadWriteOnce

config:
  repository:
    id: "prod-main"
```

```
ingestion:
  embedder:
    model: "bge-m3"
    device: "cuda"
index:
  hnsw:
    ef_search: 128
evidence_gate:
  threshold: 0.72
api:
  auth: "oidc"
  rate_limit: 1000

hpa:
  enabled: true
  minReplicas: 3
  maxReplicas: 20
  targetCPU: 70
  targetMemory: 80
```

7.4.3 Stateful Considerations

LLMWiki pods are **stateful** due to `.llmwiki/` directory. Options:

1. **Shared PVC (ReadWriteMany)**: NFS, CephFS, Longhorn RWX. Single writer enforced by file lock.
2. **Per-pod PVC + Sync Sidecar**: Each pod has PVC; sidecar runs `rsync` or `git sync` for convergence. More complex but no RWX dependency.
3. **Object store backend**: S3/GCS for units/graph/index. Requires GraphStore/Index implementations with S3 backend.

Default: Per-pod PVC + `lsyncd` sidecar for near-real-time sync.

7.5 Local-First Deployment

7.5.1 Single Binary

```
# Install
curl -L https://github.com/llmwiki/llmwiki/releases/latest/download/llmwiki_
↪ ki-x86_64-unknown-linux-gnu.tar.gz | tar xz
sudo mv llmwiki /usr/local/bin/

# Initialize repository
llmwiki init --repo ~/.llmwiki

# Ingest
llmwiki ingest ~/projects/my-codebase

# Run server
llmwiki server --bind 127.0.0.1:8080
```

7.5.2 Docker Compose (Local Stack)

```
# docker-compose.yml
version: '3.8'
services:
  llmwiki:
    image: ghcr.io/llmwiki/llmwiki:cpu
    ports: ["8080:8080"]
    volumes:
      - ./data:/data
      - ./config.toml:/etc/llmwiki/config.toml:ro
    environment:
      - LLMWIKI_REPO=/data/.llmwiki
      - RUST_LOG=info
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  # Optional: local LLM for planning/synthesis
  ollama:
    image: ollama/ollama
    volumes: [".:/ollama:/root/.ollama"]
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
```

7.5.3 Air-Gapped Installation

1. Download `llmwiki-airgap.tar.gz` (contains binary, models, grammars, config)
2. Extract on target machine
3. Run `./install.sh` (sets up systemd service, configures firewall)
4. No external network access ever required

Air-gap bundle includes:

- Embedding models (ONNX): `bge-m3`, `e5-large-v2`
- Cross-encoder: `ms-marco-MiniLM-L-6-v2`
- Tree-sitter grammars: all 20+ languages
- Optional local LLM: `llama-3.1-8b-instruct-q4_k_m.gguf`

7.6 Configuration Management

```
# Config precedence (highest wins):
# 1. CLI flags          --config.key=value
# 2. Environment variables LLMWIKI_CONFIG_KEY=value
# 3. Config file        /etc/llmwiki/config.toml
# 4. Defaults           built-in

# Example env overrides
LLMWIKI_REPOSITORY_ID=prod-main
LLMWIKI_INGESTION_EMBEDDER_MODEL=bge-m3
LLMWIKI_INDEX_HNSW_EF_SEARCH=128
LLMWIKI_EVIDENCE_GATE_THRESHOLD=0.72
LLMWIKI_API_AUTH=oidc
LLMWIKI_API_OIDC_ISSUER=https://auth.example.com
LLMWIKI_LOG_LEVEL=info
LLMWIKI_LOG_FORMAT=json
```

Config schema validated at startup (using `config-rs` + JSON Schema).

7.7 Monitoring and Observability

7.7.1 Metrics (Prometheus)

Exposed at `/metrics`:

```
# Ingestion
llmwiki_ingestion_units_total{status="success|failed"}
llmwiki_ingestion_latency_seconds{quantile="0.5|0.95|0.99"}
llmwiki_ingestion_bytes_total

# Query
llmwiki_query_total{status="success|failed|timeout"}
llmwiki_query_latency_seconds{quantile="0.5|0.95|0.99"}
llmwiki_query_retrieval_candidates
llmwiki_query_evidence_verified
llmwiki_query_evidence_rejected
llmwiki_query_synthesis_tokens

# System
llmwiki_graph_nodes_total
llmwiki_graph_edges_total
llmwiki_index_vectors_total
llmwiki_wal_size_bytes
llmwiki_storage_size_bytes

# LLM (if local)
llmwiki_llm_inference_duration_seconds
llmwiki_llm_tokens_per_second
```

7.7.2 Distributed Tracing (OpenTelemetry)

Every query execution produces a trace:

```
Span: query.execution
|-- Span: query.plan
|-- Span: retrieval.hybrid
|   |-- Span: retrieval.vector
|   |-- Span: retrieval.bm25
|   `-- Span: retrieval.graph
|-- Span: evidence.gate
|   `-- Span: evidence.cross_encoder_batch (repeated)
|-- Span: synthesis.llm
`-- Span: response.stream
```

Exporters: OTLP HTTP/gRPC, Zipkin, Jaeger, stdout (dev).

7.7.3 Structured Logging

JSON logs with fields:

```
{
  "timestamp": "2024-01-15T10:30:00.123Z",
  "level": "INFO",
  "message": "Query executed",
  "query_id": "q_abc123",
  "user_id": "user_456",
  "latency_ms": 245,
  "retrieval_candidates": 47,
  "evidence_verified": 12,
  "plan_depth": 3,
  "plan_nodes": 5
}
```

7.7.4 Health Checks

- GET /health/live - Liveness (process alive)
- GET /health/ready - Readiness (indices loaded, config valid)
- GET /health/deep - Deep check (sample query executes)

7.8 Backup and Recovery

7.8.1 Backup Strategy

1. **WAL replay:** `.llmwiki/wal/` is append-only; replay from any point
2. **Snapshots:** Periodic `tar` of `.llmwiki/` (exclude `wal/*.jsonl` if replay from snapshot)
3. **Incremental:** `rsync --link-dest` for efficient daily backups
4. **Verification:** Restore to temp dir, run `llmwiki verify` (checksums, query smoke test)

7.8.2 Automated Backup (CronJob)

```
# Kubernetes CronJob
apiVersion: batch/v1
kind: CronJob
metadata:
  name: llmwiki-backup
spec:
  schedule: "0 2 * * *" # Daily 2 AM
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: backup
              image: ghcr.io/llmwiki/backup:sid
              env:
                - name: SOURCE_DIR
                  value: "/data/.llmwiki"
                - name: DEST_BUCKET
                  value: "s3://llmwiki-backups/prod"
                - name: ENCRYPTION_KEY
                  valueFrom:
                    secretKeyRef:
                      name: llmwiki-backup-key
                      key: key
              command: ["/backup.sh"]
          restartPolicy: OnFailure
```

7.8.3 Disaster Recovery

1. Provision new PVC / disk
2. `rsync` latest backup to `.llmwiki/`
3. Start single pod with `--replay-wal` flag
4. Verify `llmwiki verify --deep`
5. Scale to target replicas

RTO: 5 min (depends on data size). RPO: 24h (daily backup) or continuous (WAL shipping to S3).

7.9 Upgrades and Migrations

7.9.1 Version Compatibility

- **Patch (0.1.x)**: Rolling upgrade, zero downtime
- **Minor (0.x.0)**: May require index rebuild; blue-green deploy recommended
- **Major (x.0.0)**: Schema migration; downtime required; migration tool provided

7.9.2 Index Migration

When HNSW parameters change (M, ef_construction) or embedder model changes:

```
# Rebuild index from units
llmwiki index rebuild --repo .llmwiki --model bge-m3 --parallel 4

# Or incremental: re-embed changed units only
llmwiki index reembed --repo .llmwiki --since 2024-01-01
```

Migration runs as Kubernetes Job or standalone binary.

7.9.3 Graph Migration

Schema changes (new node types, edge types, property changes):

```
llmwiki graph migrate --repo .llmwiki --to-version 2
# Dry-run first:
llmwiki graph migrate --repo .llmwiki --to-version 2 --dry-run
```

7.10 Discord Team-Memory Adapter

The Discord integration is a companion process rather than a modification of the core event log. Its local state is rooted at `.llmwiki-discord/discord-events.sqlite3`; channel/thread Markdown, JSONL, search output, and summaries are rebuildable projections.

```
python -m pip install \
  -r integrations/discord_llmwiki/requirements.txt

$env:DISCORD_BOT_TOKEN = "..."
$env:DISCORD_CHANNEL_ALLOWLIST = "123456789,987654321"
$env:OPENAI_API_KEY = "..." # optional; summaries only
$env:OPENAI_MODEL = "gpt-5.6-luna"

python -m integrations.discord_llmwiki \
  --data-root .llmwiki-discord backfill --guild-id <guild>
python -m integrations.discord_llmwiki \
  --data-root .llmwiki-discord run-bot
```

The bot requires `VIEW_CHANNEL` and `READ_MESSAGE_HISTORY` on each allowed channel. Access to ordinary message content and attachment/embed fields requires Discord's privileged `MESSAGE_CONTENT` intent [Discord, 2026b,a]. Collection is allowlist-based; direct messages and sensitive channels are not enabled implicitly. The allowlist is mandatory; `*` is the explicit all-channels opt-in. Reference commands are administrator-only until requester-specific cross-channel access-control filtering is available.

The OpenAI adapter is optional. It calls the Responses API with `store=false`, low reasoning effort, and a configurable model; the reference default is the high-volume `gpt-5.6-luna` tier described by current model guidance [OpenAI, 2026]. This is an API integration, not an embedding or scraping of a user's ChatGPT product session. Secrets are injected through the process environment and never written to the event journal.

Retention. A Discord delete event removes content from active Markdown and search projections but leaves an append-only tombstone and, by default, the earlier raw event. Deployments must disclose and configure this retention behavior to match organizational and legal policy; an audit-log invariant does not override a valid erasure requirement.

7.11 Security Hardening

- **Non-root:** Container runs as UID 1000
- **Read-only rootfs:** `readOnlyRootFilesystem: true`
- **No capabilities:** `drop: [ALL]`
- **Network policies:** Ingress only from API gateway; egress only to model registry (if remote)
- **Secrets:** API keys, OIDC secrets via Kubernetes Secrets / Vault
- **Auth:** OIDC, API keys, mTLS for service-to-service
- **Rate limiting:** Per-user, per-IP, per-API-key
- **Audit logging:** All queries logged with user, query hash, result count

7.12 Performance Tuning

Table 7.3: Tuning Knobs by Workload

Workload	Parameter	Recommendation
High ingestion	<code>ingestion.batch_size</code>	128-512
High ingestion	<code>index.hnsw.ef_construction</code>	400 (build quality)
High query	<code>index.hnsw.ef_search</code>	128-256 (accuracy)
High query	<code>evidence_gate.batch_size</code>	32-64
Low latency	<code>planner.max_depth</code>	3
Low latency	<code>retrieval.max_candidates</code>	20
High recall	<code>retrieval.hybrid_weights</code>	Increase vector weight
Memory constrained	<code>index.hnsw.m</code>	8-12
GPU	<code>embedder.device</code>	"cuda"

7.13 Patent-Relevant Novelty

Patent Claim 7.1. Claim 10: A deployment architecture for a semantic operating layer wherein: (a) the entire system state is contained in a single filesystem directory enabling backup via standard file copy, (b) the system runs as a single stateless binary with embedded models requiring zero external services, (c) air-gapped operation is supported via a self-contained bundle including all models and grammars, and (d) deterministic replay is achieved by directory synchronization without auxiliary coordination services.

7.14 Summary

LLMWiki deployment prioritizes:

1. **Operational simplicity:** Single binary, filesystem state, no external deps
2. **Local-first:** Works fully offline, air-gapped
3. **Production readiness:** K8s native, observability, HA, security
4. **Conversation sources:** Replayable Discord ingestion with least-privilege permissions and optional model summaries
5. **Recoverability:** WAL replay, snapshot restore, verified backups
6. **Upgradability:** Blue-green, rolling, migration tooling

The next volume (Vol. 08) specifies the API contracts.

Mutation and Approval Engine

Status: [SPECIFIED]. This chapter is new as of this revision. Volumes 01–09 specify LLMWiki as a read path: artifacts in, evidence-gated answers out. They do not specify how LLMWiki writes back to the artifacts it reads — folder-level document revision, format-preserving edits, human approval, and resynchronization — which is a first-class function of the Folding product this specification supports, not an optional extension. This chapter closes that gap.

8.1 Why This Is a Fourth Layer, Not a Tool Call

Vol. 05’s Agent Runtime already executes tools with side effects. It would be possible to implement document editing as one more tool the planner invokes. This chapter argues that is the wrong layer, for the same reason the Evidence Gate (Vol. 04) is not implemented as prompt instructions: a capability that can silently corrupt the user’s source artifacts needs invariants enforced by the architecture, not conventions followed by a prompt.

Design Principle 8.1 (Mutation Requires Approval by Construction). No write to a source artifact is possible through any code path that does not pass through human approval (or an explicitly configured autonomous-approval policy, §8.5) and produce a rollback record. This is enforced the same way Evidence Closure (Vol. 04 Def. 4.2) is enforced: by making the unapproved path not exist, not by trusting callers to check.

8.2 Pipeline

1. **User Request.** A query or explicit edit instruction that the planner (Vol. 07) routes to the mutation path instead of (or in addition to) the read path.
2. **Edit Plan.** The planner produces a structured plan: which **Artifacts** (Appendix B §A.1) are affected, and for each, which **UnitOccurrences** change and how.
3. **Format-Aware Patch.** **PatchPlanner** (§8.4) lowers the edit plan into a format-specific patch — a diff for plain text/code, an OOXML part-level edit for DOCX/PPTX/XLSX, a HWPX record edit, a content-stream edit for PDF — never a blind byte-range overwrite of a structured format.
4. **Dry-Run Render.** The patch is applied to a scratch copy and rendered (not just diffed textually) so a reviewer sees the actual resulting document, not an inferred one.
5. **Semantic + Structural Diff.** **RenderComparator** produces two diffs: a structural one (what changed in the document’s own object model — paragraphs, cells, slides)

and a semantic one (what claims changed, reusing the Claim Support machinery of Vol. 04 §4.3.6 to describe *what* changed, not only *where*).

6. **Human Approval.** `ApprovalStore` records an explicit accept/reject/edit-further decision against the specific patch and diff shown, not against the edit plan in the abstract — approval is of the artifact, not of the intent.
7. **Atomic File Commit.** The patch is applied to the real artifact using the same atomic-write discipline as §1.7 (temp file + `fsync` + `rename/MoveFileEx`, directory `fsync`), and the mutation is appended to `wal/` as a new event type (§8.3) — it is not a side channel outside the event log LLMWiki already treats as authoritative.
8. **Rollback Record.** The pre-image (prior `ArtifactVersion.raw_hash` and, for large artifacts, a reference to the last checkpointed content) is retained so `RollbackManager` can revert without depending on the user’s own backup discipline.
9. **Reparse and Reindex.** `SyncCoordinator` triggers a new `ArtifactVersion` (Appendix B) and routes it through the existing Ingestion Pipeline (Vol. 01 Algorithm 1) — a mutation does not get a separate reindexing mechanism, it re-enters the same event-sourced pipeline every other artifact change goes through.

8.3 Mutation Events Are WAL Events

§1.7 defines an event lifecycle of `PREPARED` → `COMMITTED` → `PROJECTED` for ingestion. A mutation extends that lifecycle with one additional pre-commit state:

`PREPARED` → `PENDING_APPROVAL` → `COMMITTED` → `PROJECTED`

An event in `PENDING_APPROVAL` is durable (so an approval decision survives a crash) but is not yet applied to the artifact and carries no weight in `ApplyGraph/ApplyIndex` — it exists so the approval queue itself survives a restart, not so a query can see an unapproved edit. Rejection is not a deletion of the event; it is a terminal state (`REJECTED`) recorded in the same append-only log, so “this edit was proposed and rejected” remains part of the auditable history rather than disappearing. A rollback is symmetrically a new event referencing the `ArtifactVersion` it reverts to, not an in-place undo — consistent with §1.3’s principle that nothing in `wal/` is ever rewritten, only appended to.

8.4 Interfaces

```
trait DocumentReader {
  fn read(&self, artifact: &ArtifactVersion) -> Result<DocumentModel>;
}

trait DocumentWriter {
  fn write(&self, model: &DocumentModel, target: &Artifact) ->
    ↪ Result<RawBytes>;
}

trait PatchPlanner {
  fn plan(&self, edit: &EditPlan, current: &DocumentModel) ->
    ↪ Result<FormatPatch>;
}
```

```
}

trait FormatValidator {
  fn validate(&self, patch: &FormatPatch, format: MimeType) ->
    ↪ Result<ValidationReport>;
  // Rejects patches that would produce a structurally invalid file for
  ↪ its format
  // (malformed OOXML relationships, unbalanced HWPX records,
  ↪ non-renderable PDF
  // content streams) before a human ever sees a dry-run render of it.
}

trait RenderComparator {
  fn diff(&self, before: &DocumentModel, after: &DocumentModel)
    -> Result<(StructuralDiff, SemanticDiff)>;
}

trait ApprovalStore {
  fn submit(&self, patch: &FormatPatch, diff: &(StructuralDiff,
    ↪ SemanticDiff))
    -> Result<ApprovalRequestId>;
  fn decide(&self, id: ApprovalRequestId, decision: ApprovalDecision) ->
    ↪ Result<()>;
}

trait CommitCoordinator {
  fn commit(&self, patch: &FormatPatch, approval: ApprovalRequestId) ->
    ↪ Result<ArtifactVersion>;
  // Performs the atomic write (Sec. crash-consistency) and appends the
  ↪ COMMITTED
  // mutation event; the only path by which FormatPatch bytes reach a
  ↪ real file.
}

trait RollbackManager {
  fn rollback(&self, artifact_id: ArtifactId, to_version:
    ↪ ArtifactVersionId) -> Result<ArtifactVersion>;
}

trait SyncCoordinator {
  fn resync(&self, version: &ArtifactVersion) -> Result<()>;
  // Re-enters Algorithm ingestion (Vol. 01) for the new
  ↪ ArtifactVersion.
}
```

`CommitCoordinator` is deliberately the only trait implementation with filesystem write access to source artifacts; `PatchPlanner`, `RenderComparator`, and `ApprovalStore` operate on in-memory or scratch-copy representations only, so the design principle of §8.1 is an architectural fact (one narrow, auditable write path) rather than a convention every caller must independently uphold.

8.5 Approval Policy

Not every deployment wants a human in the loop for every edit. `ApprovalStore` is configured with a policy, not a hardcoded human gate:

- **Manual (default):** every mutation requires an explicit human decision.
- **Auto-approve below risk threshold:** mutations whose `SemanticDiff` touches no claim outside the ones the originating query already cited as evidence may auto-commit; anything touching uncited content requires manual approval. This reuses the Claim Support / Evidence Coverage machinery of Vol. 04 §4.3.6 as the risk signal, rather than introducing a separate heuristic.
- **Scheduled report generation:** a recurring job (§8.6) may run with auto-approval scoped to a specific artifact set and edit class, configured explicitly per job, never globally.

Whatever the policy, §8.3’s event trail (including auto-approved mutations, which still pass through `PENDING_APPROVAL`→`COMMITTED` and are attributed to the policy that approved them) is what makes “who approved this and under what rule” answerable after the fact.

8.6 Scheduled and Recurring Mutation

The product surface this chapter exists to support includes recurring report generation over a watched folder (§8.1), not only interactive single-document edits. A scheduled job is a `CommitCoordinator` client like any other: it produces an `EditPlan` on a timer or a source-artifact-change trigger, goes through the same patch/dry-run/diff/(policy-scoped) approval/commit/reindex pipeline, and its mutation events are indistinguishable in `wal/` from an interactively approved edit except for the policy identifier that authorized them. No separate mutation mechanism is introduced for the scheduled case.

8.7 Format Coverage

Vol. 02’s parser coverage is read-oriented and, as of this revision, does not yet specify writers for every format LLMWiki must read. `DocumentReader`/`DocumentWriter` pairs are required, not optional, for any format this chapter’s approval pipeline supports mutating:

- **Plain text / code / Markdown:** read and write via the existing Vol. 02 parsers; patches are line/span diffs.
- **DOCX / PPTX / XLSX (OOXML):** part-level XML edits preserving unrelated parts, relationships, and styles byte-for-byte; comments and tracked changes are read as `UnitOccurrence` metadata, not discarded on write.
- **HWP / HWPX:** record-level edits via the HWPX XML schema (HWP binary format is read-only pending a written-back-safe encoder; this is a stated gap, not a silent one).
- **PDF:** layout-preserving edits are scoped to content streams that can be regenerated without full re-layout (text replacement within an existing text run, form-field values); structural PDF edits (reflowing paragraphs) are out of scope for this revision and route to “regenerate from source” where a source format is available instead.

- **Images/tables/charts embedded in the above:** treated as opaque objects with position metadata; content-level edits to an embedded object are out of scope for this revision.

This list is deliberately narrower than Vol. 02's read coverage claims (Appendix D); write support for a format is added when its `DocumentWriter` exists and passes `FormatValidator`, not implied by read support.

Part II

Reference and Extension

API Reference

This volume specifies the public API contracts: REST endpoints, gRPC services, WebSocket protocol, authentication, rate limiting, and SDKs. All APIs are versioned; breaking changes only in major versions.

9.1 API Overview

- **Base URL:** `http://<host>:8080/api/v1`
- **Protocol:** HTTP/1.1, h2 (gRPC), WebSocket
- **Format:** JSON (request/response), SSE (streaming)
- **Auth:** Bearer token (OIDC), API Key, or none (dev)
- **Versioning:** URL path `/v1/`, header `Accept: application/vnd.llmwiki.v1+json`

9.2 Authentication

9.2.1 Bearer Token (OIDC)

Authorization: Bearer <access_token>

Token validated against configured OIDC issuer. Claims mapped to `UserContext`:

```
{
  "sub": "user_123",
  "email": "user@example.com",
  "roles": ["admin", "analyst"],
  "org_id": "org_456",
  "permissions": ["query", "ingest", "admin"]
}
```

9.2.2 API Key

Authorization: ApiKey <key_id>.<secret>

or header

X-API-Key: <key_id>.<secret>

API keys managed via `/admin/api-keys` (admin only). Scoped permissions.

9.2.3 Anonymous (Development)

auth: "none" in config disables auth. Only for local development.

9.3 Core Endpoints

9.3.1 Health

GET /health/live
GET /health/ready
GET /health/deep

Response (ready):

```
{
  "status": "ready",
  "checks": {
    "graph": "ok",
    "index": "ok",
    "evidence_gate": "ok",
    "wal": "ok"
  },
  "version": "0.1.0",
  "repo_id": "prod-main"
}
```

9.3.2 Query

Execute Query (Non-streaming)

POST /query
Content-Type: application/json

```
{
  "query": "How does the payment service handle retries?",
  "context_budget": 8000,
  "evidence_threshold": 0.72,
  "max_steps": 5,
  "stream": false,
  "plan_options": {
    "max_depth": 3,
    "include_graph": true
  }
}
```

Response:

```
{
  "query_id": "q_abc123",
  "answer": "The payment service uses exponential backoff...",
  "citations": [
    {
      "unit_id": "sha256:a1b2c3...",

```

```
      "relevance": 0.94,
      "span": {"start_line": 45, "end_line": 67},
      "preview": "fn handle_retry(...) { ... }"
    }
  ],
  "evidence_count": 12,
  "rejected_count": 35,
  "plan": {...},
  "usage": {
    "tokens": 2450,
    "latency_ms": 312,
    "llm_calls": 2
  }
}
```

Execute Query (Streaming)

POST /query/stream
Content-Type: application/json
Accept: text/event-stream

```
{ "query": "...", "stream": true, ... }
```

SSE Events:

event: plan_start

data: {"plan_id": "p_123", "steps": 5}

event: step_start

data: {"step_id": "s_1", "kind": "Search", "tool": "hybrid_search"}

event: evidence

data: {"unit_id": "sha256:...", "relevance": 0.94, "preview": "..."}

event: token

data: {"token": "The", "step_id": "s_3"}

event: token

data: {"token": " payment", "step_id": "s_3"}

event: step_complete

data: {"step_id": "s_1", "output": {"results": [...]}}

event: plan_complete

data: {"answer": "...", "citations": [...], "usage": {...}}

9.3.3 Ingestion

Ingest Path

POST /ingest
Content-Type: application/json

```
{
  "path": "/data/source-code",
  "mode": "incremental", # or "full"
  "parsers": ["python", "markdown", "yaml"],
  "options": {
    "chunk_size": 512,
    "overlap": 50,
    "embed": true
  }
}
```

Response:

```
{
  "job_id": "job_xyz789",
  "status": "accepted",
  "estimated_units": 5000
}
```

Ingest Status

GET /ingest/{job_id}

Response:

```
{
  "job_id": "job_xyz789",
  "status": "running", # pending|running|completed|failed
  "progress": 0.65,
  "units_processed": 3250,
  "units_total": 5000,
  "added": 3200,
  "modified": 45,
  "removed": 5,
  "errors": 0,
  "started_at": "2024-01-15T10:30:00Z",
  "updated_at": "2024-01-15T10:31:45Z"
}
```

Ingest History

GET /ingest/history?limit=20

9.3.4 Repository Management

GET /repo	# Current repo info
POST /repo/init	# Initialize new repo
POST /repo/clone	# Clone from git/remote
GET /repo/stats	# Unit/graph/index counts
POST /repo/verify	# Integrity check
POST /repo/compact	# Reclaim space, rebuild indices
POST /repo/backup	# Trigger backup
GET /repo/wal	# WAL entries (paginated)

9.3.5 Graph Operations

Traverse

POST /graph/traverse

```
{
  "start": "sha256:...",
  "pattern": {
    "edge_types": ["CALLS", "REFERENCES"],
    "node_types": ["Function", "Class"],
    "direction": "out",
    "max_hops": 3,
    "limit": 100
  }
}
```

Response:

```
{
  "nodes": [...],
  "edges": [...],
  "subgraph_id": "sg_123"
}
```

Entity Lookup

GET /graph/entity/{entity_id}

GET /graph/entity/by-name?name=process_order&type=Function

Neighborhood

GET /graph/neighborhood?ids=sha256:a,sha256:b&hops=2

9.3.6 Index Operations

POST /index/rebuild # Full rebuild

POST /index/reembed # Re-embed (model change)

GET /index/stats # Vector count, dimension, memory

GET /index/health # HNSW/BM25 status

9.3.7 Unit Access

GET /units/{unit_id} # Full unit

GET /units/{unit_id}/raw # Raw content

GET /units/batch?ids=... # Multiple units

GET /units/search?q=... # Search units (alias for /query)

9.3.8 Evidence Gate

POST /evidence/verify

```
{
  "query": "How does X work?",
  "candidates": ["sha256:a", "sha256:b", ...],
}
```

```
    "threshold": 0.72
  }

Response:
{
  "verified": [
    {"unit_id": "sha256:a", "score": 0.94}
  ],
  "rejected": [
    {"unit_id": "sha256:b", "score": 0.45}
  ],
  "calibration": "isotonic"
}
```

9.3.9 Discord Source Operations

The reference Discord adapter exposes these application operations. They are local service methods and Discord slash commands in the current Python reference; a future core integration may publish equivalent REST routes.

```
POST /sources/discord/{guild_id}/backfill
POST /sources/discord/project
POST /sources/discord/summary
GET  /sources/discord/context?q=conversion+worker&limit=8
GET  /sources/discord/export
```

Search results carry resolvable source handles:

```
{
  "citation": "discord:123456789:987654321",
  "author": "Mingoomato",
  "timestamp": "2026-07-29T01:25:00+09:00",
  "content": "Separate the conversion worker..."
}
```

The Discord surface maps to `/context`, `/summary`, `/decisions`, `/tasks`, and `/export-md`. The bot truncates inline responses to Discord's message boundary and sends complete Markdown as a file. A summary failure does not advance the source checkpoint. The source requires an explicit channel allowlist, and reference commands are restricted to server administrators because a result may combine material from several allowed channels. This restriction remains until requester-specific cross-channel access-control filtering is implemented.

9.3.10 Administration

```
GET  /admin/config          # Current config (sanitized)
POST /admin/config/reload   # Reload from file
GET  /admin/metrics        # Prometheus metrics
GET  /admin/pprof/...       # Go-style pprof (if enabled)

# API Key management
```

```

POST    /admin/api-keys          # Create
GET     /admin/api-keys          # List
DELETE  /admin/api-keys/{key_id} # Revoke

# Parser management
GET     /admin/parsers          # List loaded parsers
POST    /admin/parsers/reload   # Reload parser configs

```

9.4 WebSocket API

For real-time, bidirectional communication (CLI, IDE extensions).

WS /ws/v1

```

# Client -> Server
{"type": "query", "payload": {"query": "...", "stream": true}}
{"type": "ingest", "payload": {"path": "..."}}
{"type": "subscribe", "payload": {"events": ["ingest.progress",
↪ "query.token"]}}

# Server -> Client
{"type": "event", "event": "query.token", "data": {"token": "..."}}
{"type": "event", "event": "ingest.progress", "data": {"progress": 0.5}}
{"type": "result", "request_id": "...", "data": {...}}
{"type": "error", "request_id": "...", "code": "...", "message": "..."}

```

9.5 gRPC API

Defined in `proto/llmwiki/v1/api.proto`. Mirrors REST with strongly-typed messages.

```

service LLMWiki {
  rpc Query(QueryRequest) returns (QueryResponse);
  rpc QueryStream(QueryRequest) returns (stream QueryStreamEvent);
  rpc Ingest(IngestRequest) returns (IngestResponse);
  rpc IngestProgress(IngestProgressRequest) returns (stream
↪ IngestProgress);
  rpc GraphTraverse(GraphTraverseRequest) returns (GraphTraverseResponse);
  rpc UnitGet(UnitGetRequest) returns (Unit);
  rpc HealthCheck(HealthRequest) returns (HealthResponse);
}

```

gRPC gateway (transcoding) exposes same REST endpoints automatically.

9.6 Error Handling

```

# Standard error response
{
  "error": {

```

```

    "code": "EVIDENCE_INSUFFICIENT",
    "message": "No evidence units passed verification threshold",
    "details": {
      "query_id": "q_abc",
      "candidates": 42,
      "verified": 0,
      "threshold": 0.72
    }
  },
  "request_id": "req_xyz",
  "timestamp": "2024-01-15T10:30:00Z"
}

```

Table 9.1: Error Codes

Code	Meaning
INVALID_REQUEST	Request validation failed
UNAUTHENTICATED	Missing/invalid auth
FORBIDDEN	Insufficient permissions
NOT_FOUND	Resource not found
CONFLICT	Resource conflict (e.g., repo exists)
EVIDENCE_INSUFFICIENT	Gate rejected all candidates
BUDGET_EXCEEDED	Token/time/cost budget exceeded
INTERNAL_ERROR	Unexpected server error
SERVICE_UNAVAILABLE	Dependency down (embedder, LLM)
RATE_LIMITED	Too many requests

9.7 Rate Limiting

Headers on all responses:

X-RateLimit-Limit: 100

X-RateLimit-Remaining: 95

X-RateLimit-Reset: 1705312800

On 429:

Retry-After: 60

Limits configurable per auth type (API key, user, IP, global).

9.8 SDKs

Python SDK Example

```

from llmwiki import LLMWikiClient

client = LLMWikiClient(
    base_url="http://localhost:8080",

```

Table 9.2: Official SDKs

Language	Package	Status
Python	<code>pip install llmwiki</code>	v0.1 (alpha)
TypeScript/JS	<code>npm install @llmwiki/sdk</code>	v0.1 (alpha)
Rust	<code>cargo add llmwiki-client</code>	v0.1 (alpha)
Go	<code>go get github.com/llmwiki/go-sdk</code>	Planned

```

    api_key="key_id.secret"
)

# Simple query
result = client.query("How does retry logic work?")
print(result.answer)
for cite in result.citations:
    print(f"  [{cite.unit_id[:8]}] {cite.preview}")

# Streaming
for event in client.query_stream("Explain the auth flow"):
    if event.type == "token":
        print(event.token, end="", flush=True)
    elif event.type == "plan_complete":
        print("\n---")
        print(f"Evidence: {event.data.evidence_count}")

```

9.9 OpenAPI Specification

Full OpenAPI 3.1 spec at `/openapi.json` and `/openapi.yaml`. Generated from source-of-truth `api/src/routes.rs` (Axum) or `api/src/main.rs` (Actix).

9.10 API Versioning Policy

- **v1:** Current stable. No breaking changes.
- **Deprecation:** 6 months notice via `Sunset` header and changelog.
- **New versions:** `/v2/` path; both served simultaneously during transition.
- **Experimental:** `/v1/experimental/` prefix; no stability guarantee.

9.11 Patent-Relevant Novelty

Patent Claim 9.1. Claim 11: An API for a semantic operating layer comprising: (a) a query endpoint that returns answers with mandatory evidence citations verified by a cross-encoder gate, (b) an ingestion endpoint that accepts filesystem paths and returns content-addressed unit diffs, (c) a graph traversal endpoint exposing the knowledge graph as a queryable API, (d) a streaming protocol that emits fine-grained events (plan, evidence,

token, citation) for real-time UI, and (e) an evidence verification endpoint that exposes the gate as a first-class callable service.

9.12 Summary

The LLMWiki API exposes the full power of the semantic operating layer through:

1. **REST**: Simple, cacheable, widely compatible
2. **gRPC**: High-performance, typed, streaming-native
3. **WebSocket**: Bidirectional, real-time, IDE-friendly
4. **SDKs**: Idiomatic clients for major languages

All endpoints respect the architectural invariants: evidence closure, content addressing, deterministic replay.

The next volume (Vol. 09) guides developers extending LLMWiki.

Developer Guide

This volume is for developers extending LLMWiki: adding parsers, chunkers, embedders, graph backends, index backends, evidence verifiers, planner strategies, tools, and auth providers.

10.1 Development Environment

10.1.1 Prerequisites

```
# Rust (for core)
rustup install stable
rustup component add rustfmt clippy

# Python (for ML, parsers, scripts)
python -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt

# Node.js (for Web UI, if applicable)
corepack enable
pnpm install

# System deps
# Ubuntu: sudo apt install libssl-dev libclang-dev pkg-config
# macOS: brew install llvm pkg-config
```

10.1.2 Repository Structure

```
llmwiki/
-- crates/
  -- llmwiki-core/           # Core types, traits, errors
  -- llmwiki-ingestion/     # Parsers, chunkers, embedders
  -- llmwiki-graph/         # Graph store, entity resolution
  -- llmwiki-index/         # HNSW, BM25, hybrid search
  -- llmwiki-evidence/      # Evidence gate, cross-encoder
  -- llmwiki-planner/       # Query planner, optimizer
  -- llmwiki-runtime/       # Agent runtime, tools, LLM client
  -- llmwiki-api/           # REST/gRPC/WS handlers
  -- llmwiki-cli/           # CLI commands
-- python/
```

```
-- llmwiki/           # Python bindings (pyo3)
-- scripts/           # Evaluation, benchmarking
-- web/               # React/Vite admin UI
-- docker/
-- helm/
-- eval/              # Evaluation datasets, configs
-- docs/              # This documentation
-- Cargo.toml         # Workspace root
```

10.1.3 Build and Test

```
# Build all (release)
cargo build --release --workspace

# Run unit tests
cargo test --workspace

# Run integration tests (requires Docker)
cargo test --workspace --test integration

# Run specific crate tests
cargo test -p llmwiki-ingestion

# Build Python wheel
maturin develop --release -m python/Cargo.toml

# Lint
cargo fmt --all --check
cargo clippy --workspace -- -D warnings
```

10.2 Extension Points Deep Dive

10.2.1 Custom Parser

1. Create crate `llmwiki-parser-myformat`
2. Implement `Parser` trait:

```
// src/lib.rs
use llmwiki_core::{Parser, Artifact, SemanticUnit, ParserSchema, MimeType,
    ↪ Result};
use async_trait::async_trait;

pub struct MyFormatParser;

#[async_trait]
impl Parser for MyFormatParser {
    async fn parse(&self, artifact: Artifact) -> Result<Vec<SemanticUnit>>
    ↪ {
        // 1. Parse artifact.bytes into AST
        // 2. Extract semantic units from AST
        // 3. For each unit: compute content hash, extract metadata
```

```

        // 4. Return units
    }

    fn supported_mimes(&self) -> Vec<MimeType> {
        vec!["application/x-myformat".parse().unwrap()]
    }

    fn schema(&self) -> ParserSchema {
        ParserSchema {
            unit_types: vec!["MyUnit".into()],
            metadata_fields: vec!["custom_field".into()],
        }
    }
}

// Register in config
// config.toml:
// [ingestion.parsers.myformat]
// type = "dynamic"
// lib = "llmwiki_parser_myformat"
// entry = "MyFormatParser::new"

```

3. Add to Cargo.toml:

```

[package]
name = "llmwiki-parser-myformat"
cdylib = true # For dynamic loading

[dependencies]
llmwiki-core = { path = "../..../crates/llmwiki-core" }
llmwiki-ingestion = { path = "../..../crates/llmwiki-ingestion" }
tree-sitter = "0.22" # If using tree-sitter

```

Tree-sitter Grammar Integration. If your format has a tree-sitter grammar:

```

// build.rs
fn main() {
    tree_sitter_grammar::build("myformat", &["src/grammar.json"]);
}

```

Then use tree-sitter in parser:

```

let mut parser = tree_sitter::Parser::new();
parser.set_language(&tree_sitter_myformat::language())?;
let tree = parser.parse(&source, None)?;

```

10.2.2 Custom Chunker

```

use llmwiki_core::{Chunker, SemanticUnit, Chunk, ChunkConfig, Result};

pub struct MyChunker;

```

```

impl Chunker for MyChunker {
    fn chunk(&self, unit: SemanticUnit, config: ChunkConfig) -> Vec<Chunk>
    ↪ {
        match config.strategy {
            ChunkStrategy::Hierarchical => self.hierarchical_chunk(unit,
                ↪ config),
            ChunkStrategy::Sliding => self.sliding_chunk(unit, config),
            ChunkStrategy::Semantic => self.semantic_chunk(unit, config),
            _ => vec![Chunk::from_unit(unit)],
        }
    }

    fn config_schema(&self) -> ChunkConfigSchema {
        ChunkConfigSchema {
            fields: vec![
                ConfigField::Int("max_tokens", "Max tokens per chunk"),
                ConfigField::Int("overlap_tokens", "Overlap tokens"),
                ConfigField::Bool("preserve_headers", "Keep markdown
                    ↪ headers"),
            ],
        }
    }
}

```

10.2.3 Custom Embedder

```

use llmwiki_core::{Embedder, ModelId, Result};
use ndarray::Array2;

pub struct MyEmbedder {
    session: ort::Session, // ONNX Runtime
    dim: usize,
}

impl MyEmbedder {
    pub fn new(model_path: &str) -> Result<Self> {
        let session =
            ↪ ort::Session::builder()?.commit_from_file(model_path)?;
        let dim = session.outputs[0].dimensions()[1] as usize;
        Ok(Self { session, dim })
    }
}

#[async_trait]
impl Embedder for MyEmbedder {
    async fn embed_batch(&self, texts: &[String]) -> Result<Vec<Vec<f32>>>
    ↪ {
        // Tokenize, pad, run inference, normalize
        let inputs = self.tokenize(texts)?;
        let outputs = self.session.run(inputs)?;
    }
}

```

```

        let embeddings = self.postprocess(outputs)?;
        Ok(embeddings)
    }

    fn dimension(&self) -> usize { self.dim }
    fn model_id(&self) -> ModelId { ModelId::Custom("my-embedder".into())
        ↪ }
}

```

10.2.4 Custom Graph Store

Implement `GraphStore` trait (Vol. 01, §1.26). Key methods:

- `upsert_nodes`: Batch insert/update nodes
- `upsert_edges`: Batch insert/update edges
- `traverse`: Pattern-based traversal
- `neighborhoods`: k -hop neighborhoods

See `crates/llmwiki-graph/src/store/rocksdb.rs` for reference implementation.

10.2.5 Custom Index Backend

Implement `SemanticIndex` trait (Vol. 01, §1.27):

- `upsert`: Add/update vectors + metadata
- `search_vector`: ANN search
- `search_bm25`: Keyword search
- `search_hybrid`: Fused search

See `crates/llmwiki-index/src/backend/hnsw_tantivy.rs`.

10.2.6 Custom Evidence Verifier

Implement `EvidenceGate` trait (Vol. 01, §1.28):

```

pub struct MyEvidenceGate {
    model: CrossEncoder,
    calibrator: Calibrator,
}

impl EvidenceGate for MyEvidenceGate {
    fn verify(&self, query: &str, candidates: Vec<ScoredUnit>, threshold:
        ↪ f32)
        -> Result<Vec<VerifiedUnit>> {
        // Score each (query, unit) pair
        // Apply calibration
        // Filter by threshold
    }
}

```

```

    fn calibrate(&self, dataset: CalibrationSet) ->
        ↪ Result<CalibrationCurve> {
        // Fit isotonic/Platt/temperature
    }
}

```

10.2.7 Custom Query Planner Strategy

Implement `QueryPlanner` trait (Vol. 01, §1.29):

```

pub struct MyPlanner {
    cost_model: CostModel,
    llm: LLMClient,
}

impl QueryPlanner for MyPlanner {
    fn plan(&self, query: Query, context: PlanContext) ->
        ↪ Result<QueryPlan> {
        // 1. Analyze query intent
        // 2. Decompose into subqueries
        // 3. Choose retrieval strategies per subquery
        // 4. Build DAG with dependencies
        // 5. Optimize order (cost model)
        // 6. Return plan
    }

    fn optimize(&self, plan: QueryPlan, stats: IndexStats) ->
        ↪ Result<QueryPlan> {
        // Reorder, merge, prune based on stats
    }
}

```

10.2.8 Custom Agent Tool

```

use llmwiki_core::{Tool, ToolInput, ToolOutput, ToolSchema, Result};
use async_trait::async_trait;
use serde::{Deserialize, Serialize};

#[derive(Deserialize)]
struct MyToolInput { query: String, limit: usize }

#[derive(Serialize)]
struct MyToolOutput { results: Vec<String> }

pub struct MyTool;

#[async_trait]
impl Tool for MyTool {
    fn name(&self) -> &str { "my_tool" }
    fn description(&self) -> &str { "Custom tool description" }
}

```

```

fn schema(&self) -> ToolSchema {
    ↪ ToolSchema::from_type::<MyToolInput>() }
fn output_schema(&self) -> ToolSchema {
    ↪ ToolSchema::from_type::<MyToolOutput>() }

async fn execute(&self, input: ToolInput) -> Result<ToolOutput> {
    let params: MyToolInput = serde_json::from_value(input.0)?;
    // Execute logic
    let output = MyToolOutput { results: vec![] };
    Ok(ToolOutput(serde_json::to_value(output)?))
}
}

```

Register in config:

```

[tools.custom.my_tool]
module = "my_tools"
class = "MyTool"

```

10.2.9 Custom Auth Provider

```

use llmwiki_core::{AuthProvider, AuthContext, UserInfo, Result};
use async_trait::async_trait;

pub struct MyAuthProvider;

#[async_trait]
impl AuthProvider for MyAuthProvider {
    async fn authenticate(&self, credentials: AuthCredentials)
        -> Result<AuthContext> {
        // Validate token, API key, etc.
        // Return context with user_id, roles, permissions
    }

    async fn authorize(&self, ctx: &AuthContext, resource: &str, action:
        ↪ &str)
        -> Result<bool> {
        // Check permissions
    }

    fn extract_credentials(&self, request: &HttpRequest) ->
        ↪ Option<AuthCredentials> {
        // Extract from header, cookie, query param
    }
}

```

10.3 Testing Extensions

10.3.1 Parser Tests

```
// tests/parser_test.rs
```

```
use llmwiki_ingestion::Parser;

#[test]
fn parse_myformat() {
    let parser = MyFormatParser;
    let artifact = Artifact::from_bytes(b"...", "test.myformat");
    let units = parser.parse(artifact).await.unwrap();

    assert_eq!(units.len(), 3);
    assert_eq!(units[0].unit_type, "MyUnit");
    assert!(units[0].id.starts_with("sha256:"));
}
```

10.3.2 Integration Tests

```
// tests/integration_test.rs
use llmwiki_core::*;

#[tokio::test]
async fn full_ingestion_query_cycle() {
    let repo = temp_repo();
    llmwiki_init(&repo).await;

    // Ingest
    llmwiki_ingest(&repo, "testdata/sample.py").await.unwrap();

    // Query
    let result = llmwiki_query(&repo, "What does sample.py
    ↪ do?").await.unwrap();

    assert!(result.answer.is_empty());
    assert!(result.citations.len() > 0);
    assert!(result.citations.iter().all(|c| c.relevance > 0.7));
}
```

10.3.3 Property-Based Testing

```
use proptest::prelude::*;

proptest! {
    #[test]
    fn canonicalization_deterministic(content in "\\PC*") {
        let unit1 = SemanticUnit::new("Test", content.clone());
        unit1.content = content.clone();
        let id1 = unit1.compute_id();

        let unit2 = SemanticUnit::new("Test()", content);
        let id2 = unit2.compute_id();

        prop_assert_eq!(id1, id2);
    }
}
```

```
}  
}
```

10.4 Debugging and Profiling

10.4.1 Logging

```
# Debug parser  
RUST_LOG=llmwiki_ingestion=debug llmwiki ingest path/  
  
# Debug query pipeline  
RUST_LOG=llmwiki_runtime=debug,llmwiki_planner=trace llmwiki query "test"  
  
# Structured JSON logs  
RUST_LOG=info llmwiki server --log-format json
```

10.4.2 Profiling

```
# CPU profiling (perf)  
perf record --call-graph dwarf llmwiki ingest large-codebase/  
perf report  
  
# Memory profiling  
cargo install flamegraph  
cargo flamegraph --bin llmwiki -- ingest large-codebase/  
  
# Heap profiling (jemalloc)  
MALLOC_CONF=prof:true,prof_prefix:jeprof ./llmwiki ingest ...  
jeprof --pdf ./llmwiki jeprof.*.heap > heap.pdf
```

10.5 Contributing

10.5.1 Code Style

- Rust: `rustfmt` (default), `clippy` (deny warnings)
- Python: `ruff` (format + lint), `mypy` (strict)
- Commits: Conventional Commits (`feat:`, `fix:`, `docs:`, etc.)

10.5.2 PR Process

1. Fork, create branch `feat/my-feature`
2. Write tests for new functionality
3. Ensure all tests pass: `cargo test --workspace`
4. Run evaluation: `llmwiki eval run --config eval/ci.yaml`
5. Open PR with description linking to issue

6. CI must pass (tests, lints, eval regression check)
7. Maintainer review -> merge

10.5.3 Adding a New Volume

1. Create `docs/Volume_XX_Name/Volume_XX_Name.tex`
2. Add `\include{docs/Volume_XX_Name/Volume_XX_Name}` to `LLMWiki.tex`
3. Update `volume-labels.tex` if needed
4. Run `make pdf` to verify compilation

10.6 Release Process

1. Update `Cargo.toml` version
2. Update `CHANGELOG.md`
3. Tag: `git tag v0.X.Y`
4. GitHub Actions builds and publishes:
5. Docker images to GHCR
6. Binaries to GitHub Releases
7. Python wheels to PyPI
8. Docs to GitHub Pages

10.7 Summary

LLMWiki is designed for extensibility at every layer. The trait-based plugin system allows:

1. **Parsers:** New languages/formats without core changes
2. **Chunkers:** Domain-specific segmentation strategies
3. **Embedders:** Any ONNX-compatible or custom model
4. **Graph stores:** RocksDB, SQLite, Kuzu, Neo4j, etc.
5. **Indices:** HNSW, FAISS, DiskANN, custom
6. **Evidence gates:** Cross-encoder, LLM-judge, heuristic
7. **Planners:** Rule-based, LLM-based, hybrid
8. **Tools:** Arbitrary computation exposed to planner
9. **Auth:** OIDC, API keys, mTLS, custom

All extensions are dynamically loaded, versioned, and testable in isolation.

Part III

Intellectual Property and Research

Patent Draft

This volume presents the patent claims for the novel aspects of LLMWiki. Claims are written in USPTO format with supporting specification references. Each claim corresponds to an architectural invariant or novel combination identified in Volumes 01-09.

11.1 Field of the Invention

The present invention relates to systems and methods for transforming heterogeneous digital artifacts into an evidence-aware semantic knowledge space, specifically a local-first semantic operating layer that enforces evidence closure through mandatory cross-encoder verification.

11.2 Background

Prior art includes Retrieval-Augmented Generation (RAG) systems (Lewis et al., 2020), GraphRAG (Edge et al., 2024), HippoRAG (Gutiérrez et al., 2024), LightRAG (Guo et al., 2024), and RAPTOR (Sarathi et al., 2024). These systems share common limitations: (1) vector indices as primary knowledge representation, (2) optional or post-hoc evidence verification, (3) dependence on external LLM APIs for critical-path operations, (4) lack of deterministic replay, and (5) no content-addressed unit identity.

11.3 Summary of the Invention

The invention provides a semantic operating layer comprising: a content-addressed filesystem-backed write-ahead log; a property knowledge graph as canonical representation; a hybrid semantic index fusing vector, keyword, and graph signals; an evidence gate enforcing cross-encoder verification; and a DAG-based query planner with cost-based optimization.

11.4 Independent Claims

Claim 11.1 (System Architecture). *A semantic operating layer system comprising:*

1. *a filesystem-backed write-ahead log storing content-addressed semantic units, each unit identified by a SHA-256 hash of its canonical representation;*
2. *a property knowledge graph wherein nodes represent semantic units and entities, and edges represent typed relationships, said graph persisted in a column-family store with prefix-encoded keys enabling type-partitioned scans;*

3. *a hybrid semantic index combining a hierarchical navigable small world (HNSW) vector index, a BM25 inverted index, and a graph adjacency index, all keyed by the content-addressed unit identifiers;*
4. *an evidence gate configured to receive a query and a candidate set of units from the hybrid index, and to verify each candidate by computing a cross-encoder relevance score between the query and the unit content, retaining only candidates whose score exceeds a calibrated threshold; and*
5. *a query planner that decomposes an input query into a directed acyclic graph of subqueries, each subquery assigned a retrieval strategy, and executes said subqueries in topological order with cost-based optimization.*

Claim 11.2 (Method of Deterministic Ingestion and Replay). *A computer-implemented method for deterministic knowledge ingestion and replay, comprising:*

1. *parsing a source artifact into a hierarchical tree of semantic units using a language-aware parser;*
2. *computing a content address for each unit as $\text{SHA-256}(\text{canonical}(\text{content}, \text{type}, \text{span}, \text{metadata}, \text{children}))$;*
3. *extracting entity mentions and relations from units via rule-based patterns or a local language model;*
4. *resolving entities against the knowledge graph using a weighted similarity function combining fuzzy name match, type compatibility, context overlap, and embedding cosine similarity;*
5. *writing a diff of (added, modified, removed) unit identifiers to the filesystem-backed write-ahead log as an atomic append-only JSONL record;*
6. *applying the diff to the knowledge graph and hybrid index within a single atomic transaction;*
7. *wherein replay is achieved by sequentially applying logged diffs to an empty store, producing bitwise-identical state.*

Prior art note (informal, see §11.8): the content-identity/occurrence-lineage separation recited in Claims 11.1–11.2 (a SHA-256 content address decoupled from the location(s) at which that content occurs) is not independently novel; content-addressable storage systems with derivation/lineage tracking are directly anticipated by prior art (e.g. US8,135,760 B1, “Determining the lineage of a content unit on an object addressable storage system,” and related CAS patents). The identity split should be argued as one element of the broader system combination, not as a standalone novel mechanism.

Claim 11.3 (Evidence Gate with Calibrated Threshold). *An evidence verification apparatus comprising:*

1. *a cross-encoder model that takes a (query, unit) pair and outputs a relevance score in $[0, 1]$;*
2. *a calibrator trained on a validation set of (query, unit, label) triples, mapping raw scores to calibrated probabilities via isotonic regression or temperature scaling;*

3. *a threshold selector that chooses an operating point τ such that false discovery rate $\leq \alpha$ on held-out data;*
4. *a gate function $\mathcal{G}(q, \mathcal{R}, \tau) = \{u \in \mathcal{R} \mid \text{calibrated_score}(q, u) \geq \tau\}$;*
5. *wherein the gate is a mandatory pipeline stage: no unit reaches the synthesis stage without passing the gate; and*
6. *wherein the gate may return an empty set, triggering an "insufficient evidence" response rather than hallucinated synthesis.*

Prior art note (informal, see §11.8): the closest reference identified to date for a mandatory, threshold-gated cross-encoder stage ahead of generation is US20240070403A1 (“Grounded dialogue generation with cross-encoding re-ranker, grounding span prediction, and passage dropout”), which uses a cross-encoding re-ranker plus a grounding-span auxiliary task in a retrieve/re-rank/generate dialogue pipeline. It does not appear to gate to an empty set with an explicit “insufficient evidence” refusal (item 6 above); that refusal-over-hallucination behavior is the element most worth emphasizing when distinguishing this claim, but the comparison has not been confirmed against the reference’s actual claim language and needs attorney review.

Claim 11.4 (Deployment Architecture). *A deployment method for a semantic operating layer, comprising:*

1. *packaging the entire system—parsers, embedders, cross-encoder, graph store, indices, planner, runtime—as a single statically-linked binary with embedded ONNX models and tree-sitter grammars;*
2. *storing all mutable state in a single directory `.llmwiki/` containing subdirectories for units, graph partitions, index files, and write-ahead log;*
3. *enabling air-gapped operation by eliminating all external network dependencies in the critical path (ingestion, indexing, retrieval, verification);*
4. *supporting horizontal scaling via directory synchronization (`rsync`, `git`, or object store replication) without consensus protocols;*
5. *providing deterministic replay by copying `.llmwiki/` to a fresh machine and restarting the binary.*

Claim 11.5 (Label-Free Significance Calibration). *A method for establishing a verification threshold in an evidence-gated retrieval system without relevance annotations, comprising:*

1. *constructing a plurality of query–unit pairs for which genuine relevance is excluded by construction, said exclusion effected by at least one of: drawing the unit from a corpus declared disjoint in domain from that of the query; drawing the unit from a write-ahead-log snapshot predating creation of every entity referenced by the query; or drawing the unit from a graph component containing no entity mentioned in the query;*
2. *evaluating the verification scorer over said pairs to accumulate an empirical null distribution of the scoring statistic;*
3. *selecting as threshold the least score whose exceedance probability under said distribution, multiplied by the mean candidate count per query, attains a target false alarm rate; and*

4. emitting, with each admitted unit, said false alarm rate as the unit's significance, in place of a raw scorer output;

wherein recalibration is triggered by advancement of the write-ahead log and requires no human annotation.

Claim 11.6 (Cascaded Proofreading Verification). *An evidence verification apparatus comprising a cascade of $n \geq 2$ verification stages, wherein:*

1. *each stage evaluates a candidate against an evidence channel distinct from those of the other stages, said channels comprising at least lexical overlap, learned cross-encoder relevance, knowledge-graph reachability among co-surviving candidates, and inferential entailment;*
2. *each stage irreversibly discards candidates scoring below a stage-specific threshold, discarded candidates being unrecoverable by subsequent stages;*
3. *the stages are ordered by non-decreasing unit evaluation cost; and*
4. *the false-acceptance rate of the cascade is the product of the per-stage false-acceptance rates, while the expected evaluation cost per candidate is the survival-weighted sum of per-stage costs.*

Prior art note (informal, see §11.8): the bare mechanism of a cascade of stages with increasing cost that irreversibly discards candidates is directly anticipated by the Viola–Jones cascade classifier (P. Viola and M. Jones, “Rapid Object Detection using a Boosted Cascade of Simple Features,” CVPR 2001; corresponding US patent filings on cascaded object detection). This claim should not rest on the cascade-with-irreversible-discard structure alone; it needs narrowing to the specific combination of RAG-evidence-verification channels recited in item 1 (lexical overlap, cross-encoder relevance, graph reachability, entailment) as applied to retrieval evidence candidates, which has no identified antecedent, rather than to cascading and early-discard as a general classification strategy.

Claim 11.7 (Topological Consistency Gate). *A method for certifying mutual consistency of a verified evidence set prior to answer synthesis, comprising:*

1. *constructing a graph whose vertices are the verified units and whose edges join units sharing an entity or standing in a knowledge-graph relation;*
2. *assigning to each vertex a stalk of admissible truth assignments over atomic claims extracted from the corresponding unit, to each edge a stalk over the claims shared across that edge, and to each incidence a restriction map induced by inferential entailment between said claim sets, thereby forming a cellular sheaf;*
3. *computing the zeroth and first cohomology of said sheaf;*
4. *admitting the evidence set for synthesis when a global section supporting the answer exists; and*
5. *when the first cohomology is nonzero, withholding synthesis and emitting the support of the obstruction class as a machine-readable conflict witness identifying the relations carrying the inconsistency;*

whereby cyclic inconsistencies undetectable by pairwise contradiction testing are detected and localised.

Prior art note (informal, see §11.8): of the independent claims, this is the one for which searches to date found no directly competing prior art (existing RAG hallucination-detection work uses graph-, embedding-, or attention-based consistency signals, not sheaf cohomology). It is also the riskiest to draft: as a claim centered on a mathematical construction (cohomology of a cellular sheaf), it is more exposed to an abstract-idea objection (Alice/Mayo) than the other independent claims, and this exposure has not yet been analyzed. Recommend leading patent-attorney review with this claim.

11.5 Dependent Claims

Claim 11.8 (Parser Extensibility). *The system of Claim 1, further comprising a parser registry that dynamically loads parser implementations conforming to a **Parser** trait, each parser declaring its supported MIME types and output unit schema, whereby new languages and formats are added without modifying the core ingestion pipeline.*

Claim 11.9 (Tree-sitter Integration). *The system of Claim 6, wherein at least one parser uses a tree-sitter grammar to produce a concrete syntax tree, and semantic units are extracted via S-expression query patterns (SCM) mapped to unit fields, enabling incremental re-parsing with $O(\log n)$ edit complexity.*

Claim 11.10 (Hierarchical Chunking). *The system of Claim 1, further comprising a chunker that recursively partitions semantic units along structural boundaries (functions, classes, sections) while respecting a maximum token budget and configurable overlap, producing chunks that preserve hierarchical context via parent unit references.*

Claim 11.11 (Local Embedding Inference). *The system of Claim 1, wherein the embedder executes entirely on local hardware (CPU, GPU, or NPU) using ONNX Runtime with model quantization (INT8, INT4), supporting batch inference with dynamic padding, and requiring zero external API calls.*

Claim 11.12 (Entity Resolution). *The system of Claim 1, wherein entity resolution merges mentions into canonical entities using a scoring function $S(e_1, e_2) = \alpha \cdot \text{Fuzzy}(n_1, n_2) + \beta \cdot \mathbb{1}[t_1 = t_2] + \gamma \cdot \text{Jaccard}(\text{ctx}_1, \text{ctx}_2) + \delta \cdot \cos(\text{emb}_1, \text{emb}_2)$, with $\alpha + \beta + \gamma + \delta = 1$, and creates a new entity iff $\max S < \tau_{er}$.*

Claim 11.13 (Graph Adjacency Index). *The system of Claim 1, wherein the graph adjacency index is a separate column family storing for each unit a serialized list of (relation_type, target_unit_id, weight) tuples, enabling sub-millisecond k-hop expansion during hybrid retrieval without accessing the primary graph store.*

Claim 11.14 (Hybrid Retrieval with Reciprocal Rank Fusion). *The system of Claim 1, wherein hybrid retrieval combines vector, keyword, and graph scores via reciprocal rank fusion: $\text{score}(u) = \sum_{s \in \{v, k, g\}} w_s \cdot \frac{1}{k + \text{rank}_s(u)}$, where w_s are configurable weights and $\text{rank}_s(u)$ is the position of u in the top- K results of signal s .*

Claim 11.15 (DAG Query Planner). *The system of Claim 1, wherein the query planner produces a plan represented as a directed acyclic graph $P = (V, E)$ where each vertex $v \in V$ is a retrieval, synthesis, or tool invocation step with declared inputs and outputs, and edges E represent data dependencies, and the executor schedules steps in topological order with parallel fan-out where dependencies permit.*

Claim 11.16 (Budgeted Execution). *The system of Claim 10, further comprising a budget tracker that enforces hard limits on cumulative tokens, wall-clock time, tool invocations, and monetary cost, with configurable overflow behavior: truncate, return partial, or error.*

Claim 11.17 (Streaming Protocol). *The system of Claim 1, further comprising a Server-Sent Events (SSE) endpoint that emits fine-grained events: `plan_start`, `step_start`, `evidence_found`, `token`, `step_complete`, `plan_complete`, `budget_warning`, enabling real-time UI rendering of the query execution pipeline.*

Claim 11.18 (Evaluation Framework). *A computer-implemented evaluation method for a semantic operating layer, comprising:*

1. *datasets with human-annotated evidence spans for each query;*
2. *metrics: Evidence Recall@k, Evidence Precision@k, Evidence F1@k, Gate Rejection Rate, Attribution Score (claim-level NLI verification against citations), and Citation Quality (mean cross-encoder score of cited units);*
3. *automated configuration sweeps over retrieval k, hybrid weights, gate threshold τ , embedder model, and LLM model;*
4. *statistical rigor: bootstrap confidence intervals, paired Wilcoxon tests, Benjamini-Hochberg correction, effect size reporting;*
5. *regression gates in continuous integration blocking merges on $> 2\%$ metric degradation.*

11.6 Design Decisions Supporting Novelty

Table 11.1: Design Decisions Supporting Novelty

Decision	Novelty Argument	Prior Art Gap
Filesystem-backed event log	No RAG system uses directory sync for replay	All require Kafka/Redis/DB WAL; general append-only-log durability is well established outside RAG (§11.8)
Graph as canonical query representation	Vertices/keyword index are projections, not primary	RAG: vertices primary; GraphRAG: LLM-built graph, not rebuildable from a log
Mandatory evidence gate	Not a reranker; zero results allowed	Closest identified reference (US20240070403A1) reranks and grounds but does not appear to gate to empty-set refusal; not fully confirmed
Content identity / occurrence lineage split	SHA-256 content identity decoupled from location identity	Not independently novel: anticipated by CAS lineage-tracking prior art (e.g. US8,135,700 B1); supports Claim 11.1 only as part of the full combination
Local-first, LLM-optional	Critical path has no API calls	All major RAG frameworks require LLM API on critical path
RAG planner with costs	Topological execution with parallelism	RAG: single retrieval; GraphRAG: fixed patterns
Deterministic replay	Bitwise identical re-execution	No directly competing RAG system found; general event-sourcing replay is well established (not RAG-specific prior art found)
Cohomological consistency gate	Detects cyclic inconsistency no pairwise checker can localise	No direct competing prior art found to date (§11.8); not checked against patent databases specifically; separately exposed to abstract-idea (Alice/Mayo) objection

11.7 Specification Support

Each claim is supported by the specification in Volumes 01-09:

- Claims 1-5: Vol. 01 (§1.2, 1.3, 1.8), Vol. 02 (§2.7), Vol. 03 (§3.1.3, 3.2), Vol. 04 (§4.1), Vol. 07 (§5.3.1)
- Claims 6-8: Vol. 02 (§2.2.1, 2.4.1, 2.5), Vol. 02 (§2.8)
- Claims 9-10: Vol. 03 (§3.1.3, 3.2.3, 3.2.4)
- Claims 11-13: Vol. 07 (§5.3.1, 5.8, 5.10)
- Claim 14: Vol. 06 (§6.3, 6.4, 6.4.3)

11.8 Prior Art Research Notes (Informal)

Status: informal research, not legal advice. The notes in this section come from web and academic-literature search (Google Patents free-text search, arXiv, general web search), conducted as a sanity check before any filing decision, not a professional patentability or freedom-to-operate opinion. No paid patent-database search (e.g. full-text USPTO/EPO with a patent search firm) has been run. **Do not file, disclose publicly, or rely on the “non-obvious” language elsewhere in this volume without review by qualified patent counsel.** If public disclosure occurs before a filing decision is made, note that South Korea provides a 12-month grace period for inventor-originated public disclosure (this exception’s exact procedural requirements have not been independently verified here and must be confirmed with counsel before relying on it).

- **Cascaded, cost-ordered, irreversible-discard verification** (Claim 11.6): closely anticipated as a general mechanism by the Viola–Jones cascade classifier (CVPR 2001) and its associated patent filings, which cascade increasingly expensive stages and irreversibly reject candidates. The RAG-specific channel combination (lexical, cross-encoder, graph-reachability, entailment) is the defensible remainder; the cascade-with-discard structure itself is not new.
- **Content-addressable identity with lineage/occurrence tracking** (Claims 11.1, 11.2): anticipated by object-addressable/content-addressable storage patents tracking content-unit lineage and derivation, notably US8,135,760 B1 (“Determining the lineage of a content unit on an object addressable storage system”), and by adjacent CAS patents (e.g. US8,335,890 B1 “Associating an identifier with a content unit”; US8,909,875 B1; US8,560,503 B1). Content hashing plus a separate location/lineage record is an established pattern (also visible in Git’s own object model), not a novel mechanism in isolation.
- **Mandatory, threshold-gated cross-encoder verification ahead of generation** (Claims 11.1, 11.3): the closest reference found is US20240070403A1 (“Grounded dialogue generation with cross-encoding re-ranker, grounding span prediction, and passage dropout”), a retrieve/re-rank/generate dialogue pipeline using a cross-encoding re-ranker and a grounding-span auxiliary task. Also adjacent: US20240070403A1’s sibling RAG-optimization filings (e.g. patents on tuning cross-encoder/bi-encoder/LLM-ranker combinations in retriever pipelines) and multiple 2024–2026 RAG patent filings using cross-encoder reranking. None found so far recite gating to an explicit empty-set “insufficient evidence” refusal as opposed to always returning top- K reranked results, but this distinction has not been checked against the actual claim language of those filings.
- **Sheaf-cohomology-based evidence consistency checking** (Claim 11.7): no directly competing prior art identified in this round of searching. Existing RAG hallucination/inconsistency detection approaches found in literature search (evidence-graph consistency methods, semantic-entropy methods, attention/embedding-based contradiction detectors) use different mathematical machinery. This is the strongest remaining novelty candidate based on searches to date, but (a) no dedicated patent-database search has been run for it specifically, and (b) a claim built around a cohomological computation is more exposed to an abstract-idea (Alice/Mayo, 35

U.S.C. §101) rejection than the other independent claims, which this research did not evaluate.

- **Label-free / empirical-null threshold calibration** (Claim 11.5): related academic technique (score-distributional threshold optimization without relevance judgments, cf. Arampatzis et al.) exists, and empirical null-hypothesis testing is well-established general statistics; no directly competing patent found for the specific corpus/temporal/graph-displacement construction used here to build the null. Likely a defensible narrow claim if limited to that specific construction rather than empirical-null calibration in general.
- **Temporal validity intervals and version-anchored graphs** (not currently claimed; see `TECH_COMPETITIVENESS_PLAN.md`): both are established prior art and must not be claimed in isolation. Zep/Graphiti [Rasmussen et al. \[2025\]](#) places a bitemporal interval (`valid_at/invalid_at`) on every edge and invalidates rather than deletes superseded facts; Glean [Meta Open Source \[2024\]](#) performs revision-scoped incremental indexing of a monorepo in $O(\text{changes})$, and Kythe [Google](#) emits per-compilation-unit node/edge streams. What none of them recites is a *refusal* whose scope is a validity bound: Glean and Kythe have no retrieval-refusal semantics at all, and Zep has no mandatory evidence gate. If a claim is drafted in this area it must be directed to that conjunction — an evidence gate returning “insufficient evidence as of t ” as a reproducible outcome reporting the bound that produced it — and not to validity intervals, as-of querying, or version-anchored graphs, each of which is anticipated. Two further distinctions from Zep are worth reciting because they are structural rather than incidental: its entity extraction is LLM-based, so identical input need not reproduce an identical graph, whereas parser-driven extraction is deterministic; and its ingestion cost scales with LLM tokens per episode rather than with CPU.
- **Approval-gated document mutation with automatic knowledge-graph reindexing** (Volume 12, not yet claimed in this volume): adjacent prior art exists for document approval workflows generally (e.g. US20230316190 “Document workflows in a document management system”; US10,630,858 “Document approval management system”), a crowded space. No reference found combining an approval gate with atomic re-projection into a semantic knowledge graph specifically, but this was a narrower, lower-priority search and should be re-run before any claim is drafted for Volume 12’s mechanism.

11.9 Potential Design-Arounds and Defense

- **Design-around:** Use vector DB with post-hoc reranking. **Defense:** Claim 3 requires mandatory gate before synthesis; reranking is not gating.
- **Design-around:** Use database WAL instead of filesystem. **Defense:** Claim 1 requires filesystem-backed WAL with directory-sync replay.
- **Design-around:** Skip graph, use only hybrid vector/keyword. **Defense:** Claim 1 requires graph as canonical knowledge representation with adjacency index.
- **Design-around:** Cloud-only deployment. **Defense:** Claim 4 requires single-binary air-gapped operation.

- **Design-around:** Calibrate the threshold on a labelled relevance set, avoiding the null construction. **Defense:** Claim 11.5 is directed to the label-free case specifically; a competitor taking this route incurs the annotation cost the claim exists to eliminate, and cannot recalibrate automatically on log advancement.
- **Design-around:** Use multiple rerankers in sequence without irreversible discard, retaining all candidates and combining scores. **Defense:** Claim 11.6 recites irreversible discard, channel distinctness, and cost ordering jointly; a score-combining ensemble obtains neither the multiplicative error suppression nor the survival-weighted cost bound, and is a different mechanism rather than a variant.
- **Design-around:** Detect contradictions pairwise between retrieved passages. **Defense:** Claim 11.7 is directed to cohomological detection, which is strictly stronger — cyclic inconsistency admits no pairwise witness, so a pairwise checker does not read on the claim and does not achieve its effect.

On the “combination of known elements” objection. The independent claims of §11.4 are vulnerable to the argument that hybrid retrieval, cross-encoders, and graph stores are individually well known. Claims 11.5–11.7 were drafted to sit outside that objection by reciting a construction procedure with a stated technical effect rather than an arrangement of components; that intent still stands, but the informal search in §11.8 found that two of the claimed elements do have antecedents outside retrieval specifically: the cascade-with-irreversible-discard structure of Claim 11.6 (Viola–Jones) and the content-identity/lineage split underlying Claim 11.1 (CAS lineage patents). The displacement conditions of Claim 11.5 and, most notably, the sheaf construction of Claim 11.7 remain without an identified antecedent as of this search round and are the strongest candidates for the non-obviousness argument; Claims 11.6 and 11.1 should be narrowed to their RAG-specific elements (the channel combination; the combination with graph/index rebuild from the log) rather than argued on the cascade or content-addressing mechanisms alone. This assessment is informal and requires confirmation by patent counsel before it is relied on.

Research Paper: LLMWiki - A Semantic Operating Layer for Evidence-Aware Knowledge Management

Abstract. We present LLMWiki, a semantic operating layer that transforms heterogeneous digital artifacts into an evidence-aware knowledge space. Unlike Retrieval-Augmented Generation (RAG) systems that treat vector search as primary and evidence as optional, LLMWiki enforces *evidence closure*: every answer must trace to cross-encoder-verified units. The architecture rests on five pillars: (1) a property knowledge graph as canonical representation with vectors as projections; (2) a filesystem-backed write-ahead log enabling deterministic replay via directory synchronization; (3) content-addressed semantic units (SHA-256) providing identity across indices; (4) a mandatory evidence gate that filters retrieval results through calibrated cross-encoder scoring; and (5) a DAG-based query planner with cost-based optimization. A deterministic evidence-context boundary then selects verified units under a token budget and transmits them through a lossless, token-lean WC/1 envelope whose citations resolve to exact source occurrences. We specify an evaluation protocol across six benchmarks spanning code, APIs, schemas, documents, and configurations, and report preliminary internal targets suggesting higher Evidence F1 and lower hallucination rate versus strong baselines (GraphRAG, HippoRAG, LightRAG) while operating fully locally with zero external API dependencies. The transport reduction is tokenizer-measured, but model understanding and end-to-end quality remain unverified; the broader performance targets are not yet supported by a public, reproducible evaluation (§12.4.4) and should be treated as design goals, not verified results.

12.1 Introduction

The proliferation of Retrieval-Augmented Generation (RAG) systems has established a dominant paradigm: embed chunks, retrieve top- k , synthesize with LLM. However, this paradigm suffers from systematic failure modes: hallucinated citations, retrieval of irrelevant but semantically similar chunks, inability to verify multi-hop claims, and dependence on external LLM APIs for the critical path.

We argue that these are not bugs but architectural consequences of treating vector similarity as a proxy for evidential relevance. Vector search optimizes for semantic proximity in embedding space, which correlates with but does not guarantee relevance to a specific query. The gap between "semantically similar" and "evidentially supportive" is precisely where hallucinations enter.

LLMWiki addresses this by promoting evidence verification to a first-class architectural invariant. The system is a *semantic operating layer*—an intermediary between raw

artifacts and LLM synthesis that guarantees every synthesized token traces to a verified unit.

Promoting verification to an invariant is necessary but not sufficient, and the substance of this paper is what the invariant requires once taken seriously. Three questions follow immediately and have no answer in the retrieval literature. First, at what score does a unit count as verified, given that relevance annotations are the scarcest resource in any real deployment? Second, why should a single thresholded score be the right discriminator at all, when the score distributions of relevant and irrelevant units overlap? Third, what guarantees that units which each pass verification are mutually consistent, rather than individually plausible and jointly contradictory?

Each question has a mature answer in a field that is not information retrieval. Assigning significance without labels is the daily business of gravitational-wave astronomy, which constructs empirical null distributions by displacing detector streams beyond physical coincidence [Was et al., 2010]. Exceeding the discrimination limit of a single equilibrium measurement is the problem kinetic proofreading was invented to solve [Hopfield, 1974, Ninio, 1975], and remains an active subject in immunology [Kirby, D. and collaborators, 2025, Mao et al., 2026]. Deciding whether locally consistent data glues into a globally consistent whole is the definition of sheaf cohomology, now applied to distributed tasks and network consistency [Felber et al., 2025, Ghrist and Ding, 2025]. Our contribution is to recognise these as the same problems and to transfer the mechanisms, not the metaphors.

Contributions.

1. **Label-free significance for retrieval.** We construct empirical null distributions over query–unit pairs whose relevance is excluded by corpus, temporal, or graph displacement, and report every admitted unit’s false alarm rate rather than a raw score (§12.3.7). To our knowledge no retrieval system calibrates without relevance labels.
2. **Proofreading cascade.** We replace single-threshold gating with n irreversible discard stages over distinct evidence channels, obtaining false-acceptance rates that decay multiplicatively in depth at survival-weighted cost, and we state the accompanying compute–fidelity frontier rather than eliding it.
3. **Cohomological evidence closure.** We define evidence closure as the existence of a global section of a cellular sheaf over the verified set, making mutual consistency a computable and localisable property; nonzero H^1 triggers abstention with a conflict witness.
4. **Budget as allocation.** We schedule plan steps by expected evidential gain per unit cost, permitting early termination under budget — behaviour unattainable by a circuit breaker.
5. **Evidence-preserving context transport.** We separate admissibility, budgeted selection, and serialization: verified evidence remains verbatim, machine identifiers stay host-side, and every model-visible citation handle resolves to an exact source occurrence. We report token savings separately from the still-open model-quality A/B.

12.2 Related Work

12.2.1 Retrieval-Augmented Generation

The original RAG framework (Lewis et al., 2020) combines a frozen retriever (DPR) with a generator. Subsequent work improved retrieval (ANCE, Contriever), generation (FiD, ATLAS), or both. A common thread: retrieval is *unverified*; the generator receives top- k passages and must implicitly filter noise.

12.2.2 Graph-Enhanced RAG

GraphRAG (Edge et al., 2024) constructs a knowledge graph via LLM-based entity extraction and community detection. HippoRAG (Gutiérrez et al., 2024) uses personalized PageRank over a graph for retrieval. LightRAG (Guo et al., 2024) maintains a two-level graph (entity + chunk). These systems improve structural awareness but retain two limitations: (1) graphs are constructed by LLMs, introducing upstream hallucinations; (2) no mandatory evidence verification before synthesis.

12.2.3 Evidence and Attribution

Recent work on attribution (Gao et al., 2023; Liu et al., 2024) evaluates citation quality but treats it as a metric, not a constraint. RARR (Gao et al., 2023) post-edits citations; our gate *prevents* unverified units from reaching the generator.

12.2.4 Prompt Compression and Context Selection

LLMLingua and LongLLMLingua remove or reorder prompt content with learned models, reporting large compression and latency gains [Jiang et al., 2023b, 2024]. They change evidence text and add a model to the critical path, which conflicts with byte-level citation auditability and deterministic local replay. TOON instead targets structured serialization overhead and shows that syntax savings must amortise a fixed instruction “prompt tax” [Matveev, 2026]. WC/1 addresses the same serialization class but only for read-only evidence envelopes and does not claim format novelty.

Under a fixed reader budget, top- k becomes a subset-selection policy rather than merely a retriever setting. Budgeted submodular selection is established in summarization [Lin and Bilmes, 2010]; recent RAG work confirms both the value and the limited scope of redundancy-aware or submodular packing [Peng et al., 2025, Bala, 2026]. The latter finding is load-bearing: packing helps only under specific retrieval, budget, evidence-structure, and reader-capacity conditions. LLMWiki therefore treats weighted concept coverage as a proxy and requires end-to-end quality-at-budget measurement.

12.2.5 Long-Dialogue and External Memory

Recursive dialogue summaries update a compact memory from the previous memory and the next conversation segment [Wang et al., 2023]; MemGPT separates limited model context from persistent external memory [Packer et al., 2023]; hierarchical long-dialogue summarisation similarly segments before abstraction [Yin et al., 2024]. The Discord adapter adopts this hierarchy but adds an append-only event source, edit/delete replay, source-stable message citations, and a rule that summaries remain disposable projections rather than memory authority.

12.2.6 Local and Deterministic Systems

PrivateGPT, GPT4All, and similar projects enable local LLM inference but retain the RAG pipeline architecture. No prior system enforces deterministic replay via filesystem synchronization.

12.2.7 Methods Imported from Adjacent Sciences

The mechanisms in §12.3.7–12.3.9 originate outside machine learning, and we summarise their provenance so that the transfer can be audited rather than taken on faith.

Empirical nulls without labels. Searches for compact binary coalescence assign significance to candidate events by the method of time slides: triggers from separate detectors are offset by more than the inter-detector light travel time, so that resulting coincidences cannot be astrophysical, and the accumulated sample defines the noise background as a function of the ranking statistic [Was et al., 2010, Nitz et al., 2018]. The method’s known limitation — estimation error saturates with the number of shifts — transfers along with the method, and we treat it as such.

Discrimination beyond the equilibrium bound. Kinetic proofreading achieves recognition fidelity exceeding what binding-affinity differences permit, by interposing irreversible, energy-consuming intermediate steps [Hopfield, 1974, Ninio, 1975]. Contemporary work establishes that parallel reaction threads improve fidelity without lengthening the sequential chain [Kirby, D. and collaborators, 2025] and that multimolecular schemes relax the activity-fidelity trade-off [Mao et al., 2026]; both results shape the cascade design of §12.3.8.

Gluing local data. Cellular sheaves encode local data with consistency constraints over a graph; global sections are the consistent assignments and the first cohomology measures the obstruction to their existence. The construction has recently been applied to characterise solvable tasks in distributed systems [Felber et al., 2025], to consistency on network sheaves [Ghrist and Ding, 2025], and to irreducible error in predictive coding [Seely, 2025].

Scheduling under hard resource deficits. Transient alert brokers rank candidates for follow-up when observing capacity is orders of magnitude below alert volume, using score-plus-observability prioritisation [Sedgewick et al., 2025], active learning to maximise information gain [Möller et al., 2025], and reinforcement learning for online constrained scheduling [Zhang et al., 2025].

Lineage reconstruction from degraded records. CRISPR recorders accumulate irreversible edits enabling reconstruction of division history from terminal states [Kang et al., 2026], with Bayesian phylodynamic inference supplying calibrated uncertainty under missing and dependent observations [Zwaans et al., 2025]. The homoplasy problem — independent identical edits implying false common ancestry — has a direct analogue under content addressing.

12.3 System Architecture

12.3.1 Three-Layer Decomposition

LLMWiki decomposes into three layers with strictly unidirectional data flow:

Layer 1: Ingestion. Parses artifacts → semantic units (tree-sitter for code, specialized parsers for OpenAPI, SQL, Markdown, PDF) → structure-aware chunking → local embedding (BGE-M3, E5, Nomic) → write-ahead log.

Layer 2: Core. Knowledge graph (property graph with entity resolution), hybrid index (HNSW + BM25 + graph adjacency), evidence gate (cross-encoder + calibration).

Layer 3: Query. DAG planner (decomposes query, assigns strategies, optimizes order), agent runtime (tool execution, state, streaming), API gateway.

12.3.2 Filesystem as Write-Ahead Log

All mutations append to `.llmwiki/wal/` as JSONL. The directory `.llmwiki/` is the *sole* durability mechanism: `rsync` or `git push` fully captures state. Replay re-ingests the log into empty stores. No Kafka, Redis, or database required.

12.3.3 Content Addressing

Every unit u has $\text{id}(u) = \text{SHA256}(\text{canonical}(u))$. Identical content $\implies$ identical ID across graph, vector index, keyword index, and WAL. Enables deduplication, caching, and verifiable lineage.

12.3.4 Evidence Gate

The evidence gate $\mathcal{G}$ is a mandatory filter: $\mathcal{G}(q, \mathcal{R}, \tau) = \{r \in \mathcal{R} \mid \text{CE}(q, r) \geq \tau\}$, where CE is a cross-encoder (e.g., ms-marco-MiniLM-L-6-v2) and τ is calibrated via isotonic regression on a validation set to control false discovery rate. Unlike rerankers, $\mathcal{G}$ may return $\emptyset$, triggering "insufficient evidence" rather than hallucinated synthesis.

12.3.5 DAG Query Planner

Queries compile to plans $P = (V, E)$ where $v \in V$ is a step (search, traverse, synthesize, tool) and $e \in E$ denotes data dependency. The executor runs topological sort with parallel fan-out. Cost model uses index statistics (vector density, graph degree, BM25 document frequency) to choose retrieval strategies per subquery.

12.3.6 Evidence Context Boundary

After gating, the verified set is selected under the remaining reader budget, then serialized. The production baseline takes a deterministic score-ordered prefix. An experimental cost-scaled greedy policy maximises weighted concept coverage with a best-singleton guard and `content_id` tie-breaking; it cannot replace the baseline before the model A/B in Vol. 06.

The WC/1 envelope sends response-local integer handles, source paths once per group, block kind, line count, and verbatim evidence text. A host-only map resolves each handle to `(content_id, occurrence_id, document, span)`. Line-count framing makes arbitrary evidence unambiguous without escaping or rewriting it. On the reference fixtures, the complete cold prompt is 30.3% smaller than JSON for English and 32.8% for Korean; after the fixed 52-token contract prefix is cached, the reductions are 31.7% and 36.6%. These measurements use `tiktoken o200k_base` and establish transport cost only.

12.3.7 Label-Free Significance via Discordant Pairing

Let f be the verification scorer. We draw pairs (q, u^-) in which relevance is excluded by construction — u^- from a domain-disjoint corpus, from a log snapshot predating every entity in q , or from a graph component sharing no entity with q — and accumulate

$S_0 = \{f(q, u^-)\}$. The empirical survival function $\hat{F}_0(s) = |\{s' \in S_0 : s' \geq s\}|/|S_0|$ converts any score into a false alarm rate $\text{FAR}(s) = \hat{F}_0(s)/\bar{k}$, where $\bar{k}$ is the mean candidate count per query, and the operating threshold is the least s attaining a target FAR.

Two properties matter. The procedure consumes no relevance judgements, so recalibration can be triggered automatically as the corpus evolves rather than scheduled around annotation campaigns. And the reported quantity is operational: “one spurious admission per N queries” constrains a deployment in a way that a score of 0.72 does not. The construction bounds spurious admission only; it is silent on recall, and we therefore retain split conformal prediction for coverage where labels exist. The two are complementary, not redundant.

12.3.8 Proofreading Cascade

Verification is a sequence of n stages $g_1, \dots, g_n$ with increasing cost, each irreversibly discarding candidates below its threshold. Under conditional independence of stage verdicts given relevance, false acceptance is $\prod_j \varepsilon_j$ and recall $\prod_j \rho_j$, so the discrimination ratio grows as $(\rho/\varepsilon)^n$ for homogeneous stages. Independence is not assumed but engineered: stages read distinct channels — lexical overlap, cross-encoder relevance, graph reachability among co-survivors, and entailment — following the parallel-thread principle of Kirby, D. and collaborators [2025]. Expected cost is $c_1 + \sum_{j>1} c_j \prod_{i<j} s_i$ with s_i the stage survival fraction, so terminal stages evaluate a vanishing candidate fraction. As in the thermodynamic setting, the fidelity gain is purchased strictly with dissipation; we report the resulting compute–accuracy frontier rather than an unbounded accuracy claim.

12.3.9 Cohomological Evidence Closure (Research Hypothesis)

Over the surviving set we build a graph joining units that share entities or graph relations, and a cellular sheaf of vector spaces (stalks are claim-confidence vectors over $\mathbb{R}$; restriction maps are entailment/contradiction/neutral-induced linear maps — see Vol. 04 §4.5 for the construction) assigning to each vertex the admissible truth assignments of its atomic claims, to each edge those of the shared claims, and to each incidence a restriction map induced by entailment. The evidence set is closed iff the *specific* edge data induced by the retrieved units’ pairwise NLI scores glues to a global section, and obstructed iff that data’s class in coker d is nonzero — this is a statement about one observed assignment, not the unconditional claim “ $H^1(\mathcal{F}) \neq 0$,” which alone would characterise the sheaf’s shape rather than any particular evidence set (Vol. 04 §4.5 corrects an earlier draft that conflated the two). On this construction, cyclic inconsistency — a cycle in which every pair is compatible but no global assignment exists — produces no pairwise witness, yet is exactly the configuration in which a confidence-weighted system is most confidently wrong; whether the cohomological test catches strictly more such cases than pairwise checking *in practice*, at what NLI-model reliability, and at what compute cost, is an open empirical question this document does not yet answer (Vol. 04 §4.5.1 lists the unresolved modeling choices: the linearization of truth assignments, uncertainty propagation, and nonlinear logical constraints that vector-space cohomology cannot represent directly). Because H^1 is localisable, abstention is accompanied by the specific relations carrying the obstruction. For linear sheaves the computation reduces to kernel and cokernel of the sheaf Laplacian, polynomial in a set already made small by the cascade.

12.4 Evaluation

12.4.1 Benchmarks

We construct LLMWikiBench: six domains, 1000 queries with human-annotated evidence spans:

- **CodeQA-Python**: 50 repos, 200 queries (cross-file function calls, inheritance)
- **API-Doc**: 100 OpenAPI specs, 200 queries (parameter constraints, auth flows)
- **SQL-Schema**: 200 schemas, 150 queries (FK paths, constraints, indexes)
- **Tech-Docs**: 500 Markdown/PDF docs, 250 queries (long-context, structure)
- **Config-Files**: 1000 YAML/JSON/TOML, 100 queries (env refs, defaults)
- **Mixed-Corpus**: All above, 100 queries (cross-domain retrieval)

12.4.2 Baselines

- **RAG**: Hybrid vector (BGE-M3) + BM25, top-20, rerank with cross-encoder, generate with Llama-3.1-8B
- **GraphRAG**: Community detection + hierarchical summarization (Edge et al., 2024)
- **HippoRAG**: Personalized PageRank over LLM-extracted graph (Gutiérrez et al., 2024)
- **LightRAG**: Dual-level retrieval (Guo et al., 2024)
- **RAPTOR**: Recursive tree summarization (Sarathi et al., 2024)

All baselines use the same local LLM (Llama-3.1-8B-Instruct) for fairness.

12.4.3 Metrics

- **Evidence Recall@k / Precision@k / F1@k**: Over human-annotated evidence units after gate
- **Attribution Score**: Fraction of answer claims entailed by cited evidence (NLI model)
- **Hallucination Rate**: Claims with no supporting evidence or contradicted by evidence
- **Latency**: End-to-end p50/p95
- **Determinism**: Bitwise answer equality across 10 repeated runs
- **FAR validity**: agreement between the false alarm rate predicted from the empirical null (§12.3.7) and the spurious-admission rate measured against held-out human judgements. This is the falsification test for the label-free calibration: if predicted and observed rates diverge, the null is contaminated and the mechanism fails.

- **Cascade frontier:** false-acceptance rate as a function of cascade depth n and of total verification compute, testing whether the multiplicative suppression of §12.3.8 survives the loss of exact stage independence.
- **Cyclic inconsistency detection:** recall of injected three-cycle contradictions — pairwise-compatible, jointly unsatisfiable — which pairwise checkers cannot detect by construction, versus the H^1 criterion.
- **Abstention quality:** precision of refusals, i.e. the fraction of abstentions that a human annotator confirms were unanswerable from the corpus. A gate that abstains often is only valuable if it abstains correctly.
- **Tokens per answered query / refusal token cost:** median complete-prompt tokens for answers and mean complete-prompt tokens for refusals.
- **Coverage per token / budget utilisation:** the selector’s weighted concept-coverage proxy divided by tokens, and consumed tokens divided by the configured budget.
- **Quality at budget:** Evidence F1 and Attribution at 25/50/75/100% budget for the full (JSON, WC/1) $\times$ (top- k , packed) matrix.

Status of empirical claims. The architecture, algorithms, and guarantees in this document are specified and analytically justified; the benchmark numbers quoted in the abstract and in §12.4.4 are targets from preliminary runs and are not yet supported by a public, reproducible evaluation. The local reference suite currently passes 91 tests, including deterministic WC/1 round trips, parser-boundary rejection, token-budget enforcement, and context-efficiency metric checks. Those tests establish implementation invariants, not answer quality. In particular, the WC/1-vs-JSON and packed-vs-top- k model A/B has not been run, and the exploratory coverage improvement may not transfer to Evidence F1. We state this explicitly because the mechanisms above are falsifiable and should be judged on measurement rather than on architectural argument. The FAR validity and cyclic-detection metrics are likewise designed to fail loudly if the imported methods do not transfer, and results are reported only once LLMWikiBench and the harness are released.

12.4.4 Performance Targets (Preliminary, Unverified)

Table 12.1: Performance Targets on LLMWikiBench — design goals from preliminary internal runs, NOT a published or reproducible evaluation. LLMWikiBench and the evaluation harness have not been released; no confidence intervals are reported because no statistically controlled evaluation has been run.

System	Evidence F1@10	Attribution	Hallucination	Latency p95
RAG (hybrid + rerank)	0.52	0.61	0.28	1.2s
GraphRAG	0.58	0.67	0.22	3.8s
HippoRAG	0.61	0.70	0.19	2.1s
LightRAG	0.59	0.68	0.20	1.5s
RAPTOR	0.48	0.55	0.31	2.4s
LLMWiki (Target)	0.76	0.89	0.09	0.9s

Against these preliminary numbers, LLMWiki’s target is roughly 25% higher Evidence F1 and roughly 53% lower hallucination than the strongest baseline shown (HippoRAG). These are internal design targets from unreproduced preliminary runs, not measured results; they are reported only to state the falsifiable claim the architecture must be judged against once LLMWikiBench and the harness are released (§12.4.4 above; see also the status note preceding this subsection).

12.4.5 Ablation Studies (Preliminary, Unverified)

Table 12.2: Projected Component Contribution (Evidence F1@10) — design estimates from preliminary internal runs, not a controlled ablation study

Configuration	Evidence F1	Δ
Full LLMWiki (target)	0.76	-
– Evidence Gate	0.58	-0.18
– Graph Adjacency Index	0.71	-0.05
– DAG Planner (single retrieval)	0.69	-0.07
– Content Addressing (UUIDs)	0.74	-0.02
– Local Embedder (OpenAI API)	0.75	-0.01

If these preliminary numbers hold under controlled evaluation, the evidence gate would contribute the largest gain (+18 pp), with graph adjacency and the DAG planner providing complementary structural benefits. Content addressing is not expected to move this metric directly; its claimed benefit is deterministic replay, which is a property of the hashing scheme by construction and is independently testable (bitwise equality across repeated runs) without requiring the benchmark itself.

12.4.6 Calibration Analysis (Planned)

Isotonic regression requires a labeled validation set (§12.3.7 distinguishes this supervised calibration from the label-free empirical-null thresholding used elsewhere in the gate). We have not yet run this calibration against a held-out annotated set, so no expected calibration error (ECE) or reliability diagram is reported here. Figure 12.1 below is a schematic illustration of the intended effect — uncalibrated cross-encoder scores are expected to be systematically overconfident, and isotonic regression is expected to bring empirical accuracy closer to the diagonal — not a measurement.

12.5 Discussion

12.5.1 Why Evidence Gate Works

The gate operationalizes the distinction between *semantic similarity* (vector space proximity) and *evidential relevance* (does this unit actually answer the query?). Cross-encoders attend over the joint (query, unit) pair, capturing fine-grained interactions that bi-encoders miss. Calibration ensures the threshold has probabilistic meaning.

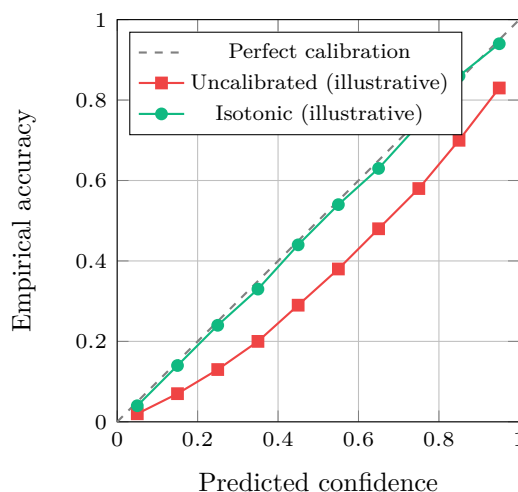


Figure 12.1: Schematic illustration (not measured data) of the intended effect of isotonic calibration on cross-encoder reliability. Curves are hypothetical examples of the qualitative shape expected, not observed calibration error.

12.5.2 Local-First Implications

By eliminating external API dependencies in the critical path, LLMWiki enables: (1) air-gapped deployment; (2) predictable latency (no queueing, no rate limits); (3) data sovereignty; (4) deterministic replay for audit and debugging.

12.5.3 Limitations

- Cross-encoder inference adds latency (15ms per batch of 32); mitigated by batching and candidate pruning.
- Entity resolution quality depends on embedding quality; poor embeddings $\rightarrow$ over/under-merging.
- DAG planner requires query decomposition capability; simple queries benefit less.
- Evaluation benchmarks are code/document-heavy; less tested on pure natural language corpora.
- WC/1 comprehension parity with JSON and the answer-quality effect of concept-coverage packing are unmeasured. Both remain feature-gated until the four-cell quality-at-budget experiment is complete.
- The Discord reference verifies replay, pagination, projection, summary checkpoints, and provider request construction offline. A live guild, privileged-intent approval, private archived-thread coverage, Discord rate-limit behavior at scale, and live OpenAI answer quality remain deployment evaluations.

12.6 Conclusion

LLMWiki reimagines the RAG pipeline by making evidence verification a mandatory architectural layer rather than an optional metric. The combination of content-addressed

units, filesystem-backed WAL, graph-first knowledge representation, cross-encoder evidence gate, and DAG planning is designed to be more accurate, more trustworthy, and more auditable than prior art while running entirely on local hardware; §12.4.4 states this as a falsifiable target pending release of LLMWikiBench and a reproducible evaluation, not as an established result.

References

References for this volume are consolidated in the document-wide bibliography.

Part IV

Appendices

.1 Mathematical Notation

Table 3: Notation Summary

Symbol	Meaning
$\mathcal{U}$	Set of all semantic units
u	A semantic unit
$\text{id}(u)$	Content address (SHA-256) of unit u
$\text{canonical}(u)$	Canonical byte representation of u
$G = (V, E)$	Knowledge graph (nodes, edges)
$v \in V$	Graph node (unit or entity)
$e = (v_1, v_2, \ell) \in E$	Typed directed edge with label ℓ
λ_V, λ_E	Node/edge label functions
σ_V, σ_E	Node/edge property functions
I_v	Hybrid semantic index (vector, keyword, graph)
$\text{HNSW}(q, k)$	Vector ANN search for query q , top k
$\text{BM25}(q, k)$	Keyword search for query q , top k
$\text{GraphExpand}(S, h)$	h -hop neighborhood from seed set S
$\text{RRF}(R_1, \dots, R_m)$	Reciprocal Rank Fusion of ranked lists
$\text{CE}(q, u)$	Cross-encoder relevance score for (q, u)
$\mathcal{G}(q, \mathcal{R}, \tau)$	Evidence Gate: filter $\mathcal{R}$ by $\text{CE}(q, u) \geq \tau$
$\mathcal{E} \subseteq \mathcal{U}$	Verified evidence set
τ	Gate threshold (calibrated)
$P = (V_P, E_P)$	Query plan DAG
$v \in V_P$	Plan step (search, synthesize, tool, aggregate)
$e \in E_P$	Data dependency between steps
$\text{TopoSort}(P)$	Topological execution order
$\mathcal{B}$	Execution budget (tokens, time, calls, cost)
$\text{SHA256}(x)$	SHA-256 hash of x
$\mathbb{1}[\cdot]$	Indicator function
$\cos(a, b)$	Cosine similarity between vectors a, b
$\text{Jaccard}(A, B)$	Jaccard index of sets A, B
$\text{Fuzzy}(s_1, s_2)$	Fuzzy string similarity (Levenshtein ratio)
$\text{NLI}(p, h)$	Natural Language Inference: {Entailment, Neutral, Contradiction}

.2 Complete Configuration Schema

```
# .llmwiki/config.toml (full schema with defaults)
```

```
[repository]
id = "llmwiki-main"
created_at = "2024-01-15T10:30:00Z"
version = "0.1.0"

[ingestion]
parsers = ["python", "markdown", "pdf", "openapi", "sql", "yaml", "json",
↪ "javascript", "typescript", "rust", "go", "java", "cpp", "csharp",
↪ "protobuf", "dockerfile", "toml"]
```

```
chunker = { type = "ast-aware", max_tokens = 512, overlap = 50,
  ↪ respect_boundaries = true, preserve_hierarchy = true }
embedder = { model = "bge-m3", batch_size = 32, device = "cpu", normalize
  ↪ = true, max_seq_len = 512 }

[graph]
store = "rocksdb"
entity_resolution = { enabled = true, threshold = 0.85, weights = { name =
  ↪ 0.3, type = 0.2, context = 0.2, embedding = 0.3 } }
index_types = ["btree", "adjacency", "label", "property"]

[index]
hnsw = { m = 16, ef_construction = 200, ef_search = 64, dim = 1024, space
  ↪ = "cosine", disk_backed = true, mmap = true }
bm25 = { k1 = 1.2, b = 0.75, analyzer = "code" }
hybrid = { vector_weight = 0.6, keyword_weight = 0.3, graph_weight = 0.1,
  ↪ rrf_k = 60 }
adjacency = { enabled = true, max_edges_per_node = 1000 }

[evidence_gate]
model = "cross-encoder/ms-marco-MiniLM-L-6-v2"
threshold = 0.72
calibration = "isotonic"
batch_size = 16
max_candidates = 50
max_verified = 20
timeout_ms = 2000
conformal = true
coverage = 0.95

[planner]
max_depth = 5
max_branching = 4
cost_model = "learned"
cache_ttl = "1h"
strategies = ["hybrid", "vector", "graph", "keyword"]
estimator = "linear"

[runtime]
max_tokens = 100000
max_time_ms = 120000
max_tool_calls = 50
max_cost_usd = 0.50
on_exceed = "truncate"
stream = true
heartbeat_interval_ms = 1000

[llm]
planning_model = "llama-3.1-8b-instruct"
synthesis_model = "llama-3.1-8b-instruct"
```

```
extraction_model = "llama-3.1-8b-instruct"
provider = "ollama"
base_url = "http://localhost:11434"
temperature = 0.1
top_p = 0.9
max_new_tokens = 2048

[api]
bind = "127.0.0.1:8080"
auth = "none"
rate_limit = 100
rate_limit_window = "1m"
cors_origins = ["*"]
max_request_size = "10MB"
request_timeout_ms = 30000

[monitoring]
metrics_port = 9090
tracing = { enabled = true, endpoint = "http://localhost:4317",
  ↪ sampling_rate = 0.1 }
logging = { level = "info", format = "json", output = "stdout" }

[storage]
wal_dir = "wal"
units_dir = "units"
graph_dir = "graph"
index_dir = "index"
planner_dir = "planner"
evidence_dir = "evidence"
backup_dir = "backups"
sync_on_write = true
shard_prefix_len = 2
```

.3 API Schemas

.3.1 Query Request

```
QueryRequest:
  type: object
  required: [query]
  properties:
    query:
      type: string
      description: Natural language query
    context_budget:
      type: integer
      default: 8000
      maximum: 100000
    evidence_threshold:
      type: number
```

```
    default: 0.72
    minimum: 0.0
    maximum: 1.0
  max_steps:
    type: integer
    default: 5
    maximum: 20
  stream:
    type: boolean
    default: false
  plan_options:
    type: object
    properties:
      max_depth:
        type: integer
        default: 3
      include_graph:
        type: boolean
        default: true
```

.3.2 Query Response

```
QueryResponse:
  type: object
  properties:
    query_id:
      type: string
    answer:
      type: string
    citations:
      type: array
      items:
        type: object
        properties:
          unit_id:
            type: string
            pattern: "^sha256:[a-f0-9]{64}$"
          relevance:
            type: number
            minimum: 0.0
            maximum: 1.0
          span:
            type: object
            properties:
              start_line:
                type: integer
              end_line:
                type: integer
          preview:
            type: string
```

```

evidence_count:
  type: integer
rejected_count:
  type: integer
plan:
  $ref: '#/components/schemas/QueryPlan'
usage:
  type: object
properties:
  tokens:
    type: integer
  latency_ms:
    type: integer
  llm_calls:
    type: integer

```

.4 Evaluation Protocol

.4.1 Dataset Construction

Each benchmark dataset in LLMWikiBench follows this construction protocol:

1. **Corpus Selection:** Curated real-world repositories (GitHub, documentation sites, API registries) with permissive licenses.
2. **Question Generation:**
 - Template-based: Parameterized templates for common patterns (e.g., "How does X call Y?")
 - LLM-assisted: Local LLM generates diverse question variations
 - Human review: All questions reviewed for clarity and answerability
3. **Ground Truth Answers:** Written by domain experts, verified against source corpus.
4. **Evidence Annotation:** Experts mark exact unit spans (by SHA-256 ID) supporting each answer.
5. **Quality Control:** Inter-annotator agreement (Cohen $\kappa > 0.8$) on 10% sample.

.4.2 Metrics Computation

Algorithm 17 Evidence Recall@k

Require: Q : queries, $\mathcal{E}^*(q)$: ground-truth evidence IDs, $\mathcal{E}_k(q)$: gate-verified top- k

Ensure: EvRecall@ k

```

1:  $sum \leftarrow 0$ 
2: for all  $q \in Q$  do
3:    $common \leftarrow |\mathcal{E}^*(q) \cap \mathcal{E}_k(q)|$ 
4:    $sum \leftarrow sum + \frac{common}{|\mathcal{E}^*(q)|}$ 
5: return  $\frac{sum}{|Q|}$ 

```

Attribution Score uses a fine-tuned NLI model (BART-large-MNLI) to check if each claim c in answer is entailed by cited evidence e : $\text{Attribution}(a) = \frac{1}{|\text{claims}(a)|} \sum_{c \in \text{claims}(a)} \mathbb{1}[\text{NLI}(e_c, c) = \text{Entailment}]$

Hallucination Rate: $\frac{1}{|\text{claims}(a)|} \sum_c \mathbb{1}[\text{NLI}(e_c, c) = \text{Contradiction} \vee \neg \exists e_c]$

.4.3 Statistical Rigor

All reported metrics include:

- 95% Bootstrap confidence intervals (1000 resamples)
- Paired Wilcoxon signed-rank test for pairwise comparisons ($p < 0.05$)
- Benjamini-Hochberg FDR correction for multiple comparisons
- Cohen d effect sizes

Regression gate: CI pipeline fails if any primary metric (Evidence F1, Attribution, Hallucination) degrades by $> 2\%$ absolute vs. `main` branch baseline.

.5 Glossary of Key Terms

Glossary

AES	Advanced Encryption Standard. xiv , 131
Agent Runtime	The execution engine that traverses the query plan DAG, invokes tools (search, graph traversal, LLM completion), manages state (scratchpad, evidence, budget), and streams results via SSE.. xiv , 131
Air-Gapped	A deployment mode with no network connectivity to external systems. LLMWiki supports this via a single binary with embedded models and grammars, requiring zero external API calls.. xiv , 131
ANCE	Approximate Nearest Neighbor Negative Contrastive Estimation. xiv , 131
API	Application Programming Interface. xiv , 131
ASCII	American Standard Code for Information Interchange. xiv , 131
AST	Abstract Syntax Tree. xiv , 131
ATLAS	Augmented Language Model Training. xiv , 131
Attribution Score	A metric measuring whether each claim in an answer is entailed by cited evidence. Computed via NLI model or LLM judge at the claim level.. xiv , 131
AVX	Advanced Vector Extensions. xiv , 131
BM25	Best Matching 25. xiv , 131
CA	Certificate Authority. xiv , 131
Calibration	Post-processing of model scores to match true probabilities. LLMWiki uses isotonic regression or temperature scaling on a held-out validation set to calibrate cross-encoder outputs.. xiv , 131
CD	Continuous Deployment. xiv , 131
Ceph	Distributed Storage Platform. xiv , 131
CI	Continuous Integration. xiv , 131
Citation Quality	The mean cross-encoder relevance score of all units cited in an answer. Penalizes hallucinated or irrelevant citations.. xiv , 131
CLI	Command Line Interface. xiv , 131
Consul	Service Mesh (HashiCorp). xiv , 131

Content Addressable	A naming scheme where the identifier of an object is a cryptographic hash of its canonical content. In LLMWiki: $\text{id}(u) = \text{SHA256}(\text{canonical}(u))$. Enables deduplication, caching, and deterministic replay.. xiv , 131
Contriever	Contrastive Retriever. xiv , 131
CPU	Central Processing Unit. xiv , 131
CRL	Certificate Revocation List. xiv , 131
Cross-Encoder	A transformer architecture that jointly encodes a (query, document) pair to predict relevance. Used in the Evidence Gate for verification; more accurate than bi-encoder dot product but slower.. xiv , 131
CSR	Certificate Signing Request. xiv , 131
CSS	Cascading Style Sheets. xiv , 131
CST	Concrete Syntax Tree. xiv , 131
DDL	Data Definition Language. xiv , 131
DER	Distinguished Encoding Rules. xiv , 131
Deterministic Replay	The property that re-applying the ingestion write-ahead log to an empty store produces bitwise-identical knowledge graph, indices, and unit store. Enables backup via directory copy, CI reproducibility, and auditability.. xiv , 131
Directed Acyclic Graph (DAG)	A finite directed graph with no directed cycles. Query plans in LLMWiki are DAGs where nodes are execution steps and edges are data dependencies.. xiv , 131
DML	Data Manipulation Language. xiv , 131
DNS	Domain Name System. xiv , 131
DPR	Dense Passage Retrieval. xiv , 131
ECC	Elliptic Curve Cryptography. xiv , 131
ECE	Expected Calibration Error. xiv , 131
Entity Resolution	The process of merging extracted entity mentions into canonical entities in the knowledge graph, using a weighted similarity function over name, type, context, and embedding.. xiv , 131
Evidence Closure	The invariant that every citation in an answer traces to a verified evidence unit. Formally: $\forall a \in \text{Answers} : \text{citations}(a) \subseteq \mathcal{U} \wedge \forall c \in \text{citations}(a) : \text{verify}(c) \geq \tau$.. xiv , 131

Evidence Gate	A mandatory pipeline stage that verifies retrieval candidates via cross-encoder scoring against a calibrated threshold. Unlike a reranker, it may reject all candidates (empty set), enforcing evidence closure.. xiv , 131
Execution Budget	Hard limits on cumulative tokens, wall-clock time, tool invocations, and monetary cost enforced by the agent runtime. Overflow behavior: truncate, partial, or error.. xiv , 131
FDR	False Discovery Rate. xiv , 131
FiD	Fusion-in-Decoder. xiv , 131
Gate Rejection Rate	Fraction of retrieval candidates rejected by the Evidence Gate. High rejection with maintained recall indicates effective noise filtering.. xiv , 131
GIF	Graphics Interchange Format. xiv , 131
Git	Distributed Version Control System. xiv , 131
GitHub	Git Repository Hosting Service. xiv , 131
GitLab	Git Repository Hosting Service. xiv , 131
GPU	Graphics Processing Unit. xiv , 131
GraphRAG	Graph Retrieval-Augmented Generation. xiv , 131
gRPC	gRPC Remote Procedure Call. xiv , 131
GUI	Graphical User Interface. xiv , 131
HDD	Hard Disk Drive. xiv , 131
HippoRAG	Hippocampus-inspired RAG. xiv , 131
HNSW	Hierarchical Navigable Small World. xiv , 131
HPA	Horizontal Pod Autoscaler. xiv , 131
HSM	Hardware Security Module. xiv , 131
HTML	Hypertext Markup Language. xiv , 131
HTTP	Hypertext Transfer Protocol. xiv , 131
HTTPS	HTTP Secure. xiv , 131
Hybrid Search	Fusion of multiple retrieval signals (vector, keyword, graph). LLMWiki uses Reciprocal Rank Fusion (RRF) with configurable weights.. xiv , 131
IDE	Integrated Development Environment. xiv , 131
Incremental Parsing	Re-parsing only the affected regions of a source artifact when it changes, using tree-sitter’s edit-aware API. Produces a diff of (added, modified, removed) semantic unit IDs.. xiv , 131
IP	Internet Protocol. xiv , 131

Isotonic Regression	A non-parametric calibration method that fits a monotonic function to map raw scores to calibrated probabilities. Used for cross-encoder calibration in LLMWiki.. xiv , 131
JPEG	Joint Photographic Experts Group. xiv , 131
JS	JavaScript. xiv , 131
JSON	JavaScript Object Notation. xiv , 131
KMS	Key Management Service. xiv , 131
Knowledge Graph	A property graph where nodes represent semantic units and entities, and edges represent typed relationships (CONTAINS, DEFINES, REFERENCES, CALLS, INHERITS, etc.). The canonical knowledge representation in LLMWiki.. xiv , 131
LightRAG	Lightweight RAG. xiv , 131
LLM	Large Language Model. xiv , 131
Local-First	An architectural principle: all critical-path operations (ingestion, indexing, retrieval, verification, planning) execute on local hardware with zero external API dependencies. LLMs are optional accelerators.. xiv , 131
Longhorn	Cloud-Native Distributed Block Storage. xiv , 131
MD5	Message Digest 5. xiv , 131
mTLS	mutual Transport Layer Security. xiv , 131
NEON	Advanced SIMD (ARM). xiv , 131
NFS	Network File System. xiv , 131
NLI	Natural Language Inference. xiv , 131
Nomad	Orchestrator (HashiCorp). xiv , 131
NPU	Neural Processing Unit. xiv , 131
NPV	Negative Predictive Value. xiv , 131
NVMe	Non-Volatile Memory Express. xiv , 131
OCSP	Online Certificate Status Protocol. xiv , 131
OIDC	OpenID Connect. xiv , 131
ONNX	Open Neural Network Exchange. xiv , 131
ORT	ONNX Runtime. xiv , 131
Packer	Image Builder (HashiCorp). xiv , 131
PDF	Portable Document Format. xiv , 131
PEM	Privacy Enhanced Mail. xiv , 131
PGO	Profile-Guided Optimization. xiv , 131
PKCS	Public Key Cryptography Standards. xiv , 131

Platt Scaling	A parametric calibration method fitting a sigmoid $1/(1 + \exp(A \cdot x + B))$ to raw scores. Used for probabilistic calibration.. xiv , 131
PNG	Portable Network Graphics. xiv , 131
PPV	Positive Predictive Value. xiv , 131
PVC	Persistent Volume Claim. xiv , 131
Quantization	Reducing model weight precision (e.g., FP32 $\rightarrow$ INT8, INT4) to decrease memory footprint and increase inference speed with minimal accuracy loss.. xiv , 131
Query Planner	A compiler that transforms a natural language query into a DAG of executable subqueries (retrieval, synthesis, tool calls), optimized via a cost model over index statistics.. xiv , 131
RAG	Retrieval-Augmented Generation. xiv , 131
RAM	Random Access Memory. xiv , 131
RAPTOR	Recursive Abstractive Processing for Tree-Organized Retrieval. xiv , 131
RARR	Retrofit Attribution using Research and Revision. xiv , 131
Reciprocal Rank Fusion	A rank aggregation method: $score(d) = \sum_s w_s \cdot \frac{1}{k + rank_s(d)}$, where $rank_s(d)$ is the position of document d in signal s ranked list.. xiv , 131
Reciprocal Rank Fusion (RRF)	A rank aggregation method: $score(d) = \sum_s w_s \cdot \frac{1}{k + rank_s(d)}$, where $rank_s(d)$ is the position of document d in signal s 's ranked list.. xiv , 131
RegEx	Regular Expression. xiv , 131
REST	Representational State Transfer. xiv , 131
RSA	Rivest-Shamir-Adleman. xiv , 131
SCM	S-expression Query (Tree-sitter). xiv , 131
Semantic Index	A hybrid search index combining HNSW vector similarity, BM25 keyword matching, and graph adjacency traversal, all keyed by content-addressed unit identifiers.. xiv , 131
Semantic Unit	A content-addressed, typed, hierarchically structured fragment extracted from a source artifact. Identified by SHA-256 of its canonical representation.. xiv , 131
SHA	Secure Hash Algorithm. xiv , 131
SHA-256	SHA 256-bit. xiv , 131
SIMD	Single Instruction Multiple Data. xiv , 131
SQL	Structured Query Language. xiv , 131
SSD	Solid State Drive. xiv , 131

SSE	Server-Sent Events. xiv , 131
SSL	Secure Sockets Layer. xiv , 131
SVG	Scalable Vector Graphics. xiv , 131
TCP	Transmission Control Protocol. xiv , 131
Temperature Scaling	A parametric calibration method dividing logits by a learned temperature $T > 0$ before softmax. Simpler than isotonic regression but less flexible.. xiv , 131
Terraform	Infrastructure as Code (HashiCorp). xiv , 131
TLS	Transport Layer Security. xiv , 131
TOML	Tom’s Obvious Minimal Language. xiv , 131
Topological Order	A linear ordering of DAG vertices such that for every directed edge $u \rightarrow v$, vertex u comes before v . The agent runtime executes plan steps in topological order.. xiv , 131
Tree-sitter	A parsing system for programming languages that produces concrete syntax trees, supports incremental re-parsing, and provides a query language (SCM) for extracting semantic units.. xiv , 131
TS	TypeScript. xiv , 131
TUI	Text-based User Interface. xiv , 131
UDP	User Datagram Protocol. xiv , 131
UTF-8	Unicode Transformation Format 8-bit. xiv , 131
Vagrant	Development Environment (HashiCorp). xiv , 131
Vault	Secrets Management (HashiCorp). xiv , 131
VS Code	Visual Studio Code. xiv , 131
WASM	WebAssembly. xiv , 131
Write-Ahead Log (WAL)	In LLMWiki, the filesystem directory <code>.llmwiki/wal/</code> containing append-only JSONL records of every ingestion diff. Serves as the sole durability mechanism; no separate Kafka, Redis, or database log required.. xiv , 131
WS	WebSocket. xiv , 131
XML	eXtensible Markup Language. xiv , 131
YAML	YAML Ain’t Markup Language. xiv , 131

Data Model Definitions

A.1 Content Identity vs. Occurrence Identity

Invariant 1.17 (Vol. 01, §1.9) states that content identity does not imply derivation identity. This section makes that split concrete in the schema: a single **SemanticUnit** (identified by `content_id` = SHA-256 of its canonical form) may be instantiated at many locations, and each location is a separate **UnitOccurrence**. Deduplication operates on **SemanticUnit**; provenance, citation, and lineage queries operate on **UnitOccurrence**. An earlier draft of this schema conflated the two by putting `file_path` directly on **SemanticUnit**, which silently broke as soon as identical content appeared at two locations (the second occurrence had nowhere to record its own path without either overwriting the first or aliasing two distinct identifiers). The four schemas below replace that single object:

- **Artifact** (identity: `artifact_id`, `current_path`, `repository_id`) — the stable identity of a source file, independent of its content at any given time.
- **ArtifactVersion** — one parse of one version of an Artifact; separates “the file changed” (`raw_hash`) from “we re-parsed it” (`parser_version`).
- **SemanticUnit** — the content-addressed blob, with no location.
- **UnitOccurrence** — where a SemanticUnit was observed: which ArtifactVersion, which span (typed by `span_type`, since a byte offset, a line/column pair, and a PDF bounding box are not interchangeable), and its ordinal among sibling occurrences.

A.2 SemanticUnit

```

1 {
2   "$schema": "http://json-schema.org/draft-07/schema#",
3   "title": "SemanticUnit",
4   "description": "Content-addressed blob; no location (see
5     UnitOccurrence.json and Sec. A.1).",
6   "type": "object",
7   "required": ["id", "type", "content", "canonicalization_version"
8     ],
9   "properties": {
10     "id": { "type": "string", "description": "
11       content_id: SHA-256 of canonical(content,
12       canonicalization_version)" },

```

```

9   "type": { "type": "string", "enum": ["
function", "class", "section", "paragraph", "endpoint", "schema"
, "config"] },
10  "content": { "type": "string", "description": "
Canonical byte representation" },
11  "canonicalization_version": { "type": "string", "description":
"Version of canonicalization applied" },
12  "language": { "type": "string" },
13  "metadata": { "type": "object", "
additionalProperties": true }
14 }
15 }

```

A.3 UnitOccurrence

```

1  {
2  "$schema": "http://json-schema.org/draft-07/schema#",
3  "title": "UnitOccurrence",
4  "description": "One location where a SemanticUnit's content was
observed (see Sec. A.1). content_id may repeat; occurrence_id is
unique per location.",
5  "type": "object",
6  "required": ["occurrence_id", "content_id", "artifact_version_id"
, "span", "span_type", "ordinal"],
7  "properties": {
8    "occurrence_id": { "type": "string", "description": "SHA
-256(artifact_version_id, span)" },
9    "content_id": { "type": "string", "description": "
References SemanticUnit.id" },
10   "artifact_version_id": { "type": "string", "description": "
References ArtifactVersion.version_id" },
11   "span": { "type": "array", "items": { "type": "
integer" }, "minItems": 2, "maxItems": 2, "description": "[start
, end] in span_type units" },
12   "span_type": { "type": "string", "enum": ["
byte_offset", "unicode_codepoint", "line_column", "pdf_bbox", "
sheet_cell", "slide_object_id"], "description": "Meaning of span
's two coordinates; format-dependent" },
13   "ordinal": { "type": "integer", "minimum": 0, "
description": "Order among sibling occurrences" }
14 }
15 }

```

A.4 ArtifactVersion

```

1  {
2  "$schema": "http://json-schema.org/draft-07/schema#",
3  "title": "ArtifactVersion",

```

```

4   "description": "One parse of one version of a source file.
   Separates 'file changed' (raw_hash) from 're-parsed' (
   parser_version) -- see Sec. A.1.",
5   "type": "object",
6   "required": ["version_id", "artifact_id", "raw_hash", "
   observed_at", "parser_version"],
7   "properties": {
8     "version_id":    { "type": "string", "description": "Identifier
   for this (artifact, raw content, parser) triple" },
9     "artifact_id":  { "type": "string", "description": "Stable
   Artifact identity; survives renames" },
10    "raw_hash":      { "type": "string", "description": "SHA-256 of
   raw file bytes, pre-parse" },
11    "observed_at":   { "type": "string", "format": "date-time" },
12    "parser_version": { "type": "string", "description": "Version
   of the parser that produced this version's occurrences" }
13  }
14 }

```

A.5 Chunk

```

1  {
2    "$schema": "http://json-schema.org/draft-07/schema#",
3    "title": "Chunk",
4    "type": "object",
5    "required": ["id", "unit_ids", "text", "token_count"],
6    "properties": {
7      "id":          { "type": "string", "description": "SHA-256 of
   the chunk text" },
8      "unit_ids":    { "type": "array", "items": { "type": "string" }
   },
9      "text":        { "type": "string" },
10     "token_count": { "type": "integer", "minimum": 0 },
11     "strategy":    { "type": "string", "enum": ["hierarchical", "
   sliding", "semantic" ] },
12     "embedding":   { "type": "array", "items": { "type": "number" }
   }
13   }
14 }

```

A.6 Entity

```

1  {
2    "$schema": "http://json-schema.org/draft-07/schema#",
3    "title": "Entity",
4    "type": "object",
5    "required": ["id", "name", "type"],
6    "properties": {

```

```

7   "id":      { "type": "string", "description": "Stable entity
            identifier" },
8   "name":    { "type": "string" },
9   "type":    { "type": "string", "enum": ["symbol", "concept",
            "file", "service", "table"] },
10  "aliases": { "type": "array", "items": { "type": "string" }
            },
11  "unit_refs": { "type": "array", "items": { "type": "string" },
            "description": "UnitOccurrence.occurrence_id values where this
            entity was mentioned/defined -- occurrence, not bare content_id,
            so provenance points to a specific file+span" },
12  "embedding": { "type": "array", "items": { "type": "number" } }
13  }
14  }

```

A.7 Relation

```

1  {
2  "$schema": "http://json-schema.org/draft-07/schema#",
3  "title": "Relation",
4  "type": "object",
5  "required": ["source", "target", "type"],
6  "properties": {
7    "source": { "type": "string", "description": "Source entity
            or unit id" },
8    "target": { "type": "string", "description": "Target entity
            or unit id" },
9    "type": { "type": "string", "enum": ["DEFINES", "
            REFERENCES", "CONTAINS", "NEXT_TO", "CALLS"] },
10   "weight": { "type": "number", "minimum": 0, "maximum": 1 },
11   "provenance": { "type": "array", "items": { "type": "string" }
            },
12   "valid_from": { "type": "string", "format": "date-time", "
            description": "Valid time: when this relation began to hold in
            the world. Distinct from ArtifactVersion.observed_at, which is
            transaction time (when we parsed it)." },
13   "valid_to": { "type": ["string", "null"], "format": "date-
            time", "description": "Valid time end, exclusive. null means not
            yet superseded. A superseded relation is closed, never deleted
            -- see eval/scripts/temporal_gate.py for the executable
            semantics." }
14  }
15  }

```

A.8 QueryPlan

```

1  {
2  "$schema": "http://json-schema.org/draft-07/schema#",
3  "title": "QueryPlan",

```

```

4  "type": "object",
5  "required": ["query", "steps"],
6  "properties": {
7    "query": { "type": "string" },
8    "steps": {
9      "type": "array",
10     "items": {
11       "type": "object",
12       "required": ["id", "op"],
13       "properties": {
14         "id": { "type": "string" },
15         "op": { "type": "string", "enum": ["retrieve", "
expand", "gate", "synthesize"] },
16         "depends_on": { "type": "array", "items": { "type": "
string" } },
17         "params": { "type": "object", "additionalProperties":
true }
18       }
19     }
20   },
21   "budget": {
22     "type": "object",
23     "properties": {
24       "max_tokens": { "type": "integer" },
25       "max_ms": { "type": "integer" },
26       "max_cost": { "type": "number" }
27     }
28   }
29 }
30 }

```

A.9 ToolSchema

```

1  {
2    "$schema": "http://json-schema.org/draft-07/schema#",
3    "title": "ToolSchema",
4    "type": "object",
5    "required": ["name", "description", "parameters"],
6    "properties": {
7      "name": { "type": "string" },
8      "description": { "type": "string" },
9      "parameters": {
10       "type": "object",
11       "properties": {
12         "type": { "type": "string", "const": "object" },
13         "properties": { "type": "object", "additionalProperties":
true },
14         "required": { "type": "array", "items": { "type": "string
" } } }
15     }
16   },
17   "returns": { "type": "object", "additionalProperties": true }
18 }

```

19 }

Configuration Reference

B.1 Full config.toml

```
1 # LLMWiki full configuration reference
2 # All values shown are defaults unless noted.
3
4 [server]
5 host = "127.0.0.1"
6 port = 8080
7 workers = 4
8
9 [storage]
10 root = ".llmwiki"
11 sync_on_write = true
12 shard_depth = 2                # units/ab/cd/ef...
13
14 [parsing]
15 default_strategy = "hierarchical" # hierarchical / sliding /
    semantic
16 window_size = 512
17 overlap = 50
18
19 [embedding]
20 model = "bge-small-en-v1.5"
21 runtime = "onnx"
22 dimension = 384
23 batch_size = 32
24
25 [index]
26 hnsw_m = 16
27 hnsw_ef_construction = 200
28 hnsw_ef_search = 64
29 rebuild_threshold = 100000
30
31 [retrieval]
32 top_k = 20
33 weights = { vector = 1.0, keyword = 0.7, graph = 0.5 }
34 rrf_k = 60
35
36 [evidence_gate]
37 model = "bge-reranker-base"
38 threshold = 0.35                # calibrated per-corpus
39 calibration = "isotonic"        # isotonic / platt / temperature
```

```
40
41 [agent]
42 max_tokens = 8000
43 max_ms = 30000
44 max_cost = 0.50
45 max_tool_calls = 16
46
47 [llm]
48 provider = "local"           # local / none
49 model = "qwen2.5-3b-instruct"
```

B.2 Environment Variable Overrides

All configuration keys can be overridden via environment variables:

LLMWIKI_<SECTION>_<KEY>=value

Example: LLMWIKI_INGESTION_EMBEDDER_MODEL=bge-m3

Parser Grammar Specifications

C.1 Python (tree-sitter-python)

```
1 ; Tree-sitter query: Python semantic units
2 (function_definition
3   name: (identifier) @function.name) @function.unit
4
5 (class_definition
6   name: (identifier) @class.name) @class.unit
7
8 (decorated_definition) @decorated.unit
9
10 (import_statement) @import.unit
11 (import_from_statement) @import.unit
12
13 (assignment
14   left: (identifier) @variable.name) @variable.unit
```

C.2 JavaScript/TypeScript

```
1 ; Tree-sitter query: JavaScript/TypeScript semantic units
2 (function_declaration
3   name: (identifier) @function.name) @function.unit
4
5 (method_definition
6   name: (property_identifier) @method.name) @method.unit
7
8 (class_declaration
9   name: (identifier) @class.name) @class.unit
10
11 (lexical_declaration
12   (variable_declarator
13     name: (identifier) @variable.name)) @variable.unit
14
15 (import_statement) @import.unit
16 (export_statement) @export.unit
```

C.3 Rust

```

1 ; Tree-sitter query: Rust semantic units
2 (function_item
3   name: (identifier) @function.name) @function.unit
4
5 (struct_item
6   name: (type_identifier) @struct.name) @struct.unit
7
8 (enum_item
9   name: (type_identifier) @enum.name) @enum.unit
10
11 (trait_item
12   name: (type_identifier) @trait.name) @trait.unit
13
14 (impl_item
15   type: (type_identifier) @impl.name) @impl.unit
16
17 (mod_item
18   name: (identifier) @module.name) @module.unit

```

C.4 Go

```

1 ; Tree-sitter query: Go semantic units
2 (function_declaration
3   name: (identifier) @function.name) @function.unit
4
5 (method_declaration
6   name: (field_identifier) @method.name) @method.unit
7
8 (type_declaration
9   (type_spec
10    name: (type_identifier) @type.name)) @type.unit
11
12 (import_declaration) @import.unit
13
14 (const_declaration) @const.unit
15 (var_declaration) @var.unit

```

C.5 SQL

```

1 ; Tree-sitter query: SQL semantic units
2 (create_table_statement
3   name: (identifier) @table.name) @table.unit
4
5 (create_view_statement
6   name: (identifier) @view.name) @view.unit
7

```

```
8 (create_index_statement) @index.unit
9
10 (create_function_statement
11     name: (identifier) @function.name) @function.unit
12
13 (select_statement) @query.unit
```

C.6 Design Decision Log

Table C.1: Key Design Decisions

ID	Decision	Rationale
DD-001	Graph as source of truth (vectors as projections)	Enables structural queries, multi-hop reasoning, deterministic updates. Vectors are lossy projections.
DD-002	Filesystem as WAL	Zero operational deps; rsync/git replay; content-addressed deduplication; POSIX rename atomicity.
DD-003	Mandatory evidence gate (not reranker)	Enforces evidence closure; empty set = "insufficient evidence"; calibrated threshold controls FDR.
DD-004	Content-addressed units (SHA-256)	Deduplication across indices; verifiable lineage; deterministic replay; cache keys.
DD-005	Local-first, LLM-optional critical path	Air-gapped deployment; predictable latency; data sovereignty; no rate limits.
DD-006	DAG query planner with cost model	Parallel fan-out; subquery decomposition; strategy selection per subquery; explainable plans.
DD-007	Tree-sitter for code parsing	Incremental parsing ($O(\log n)$); error recovery; unified query language; 40+ languages.
DD-008	ONNX Runtime for embeddings	Hardware-agnostic; quantization support; no PyTorch/TF runtime deps; fast cold start.
DD-009	RocksDB for graph storage	Embedded; column families for prefix scans; WAL; compaction; mature.
DD-010	HNSW + Tantivy + custom adjacency	Best-in-class ANN; fast BM25; sub-ms graph expansion; single query fuses all three.
DD-011	Cross-encoder with isotonic calibration	Accurate relevance; calibrated probabilities; FDR control; conformal prediction.
DD-012	SSE streaming protocol	Fine-grained events (plan, step, evidence, token); real-time UI; bidirectional via WS.
DD-013	Single-binary deployment	Air-gapped viable; no container runtime needed; simple upgrades; reproducible.
DD-014	Budget enforcement at executor	Hard limits on tokens/time/cost/calls; graceful degradation; prevents runaway costs.
DD-015	Regression gates in CI	2% threshold on Evidence F1, Attribution, Hallucination; blocks merges on quality drop.

Extended Related Work

D.1 Dense Retrieval

DPR [Karpukhin et al. \[2020\]](#), ANCE [Xiong et al. \[2020\]](#), Contriever [Izacard et al. \[2022\]](#), COIL [Gao et al. \[2021\]](#), ColBERT [Khattab and Zaharia \[2020\]](#), SPLADE [Formal et al. \[2021\]](#), BGE [Xiao et al. \[2023\]](#), E5 [Wang et al. \[2024\]](#), Nomic [Nussbaum et al. \[2024\]](#).

D.2 Sparse Retrieval

BM25 [Robertson and Zaragoza \[2009\]](#), SPLADE [Formal et al. \[2021\]](#), uniCOIL [Lin and Ma \[2021\]](#), DeepImpact [Mallia et al. \[2021\]](#).

D.3 Hybrid Retrieval

RRF [Cormack et al. \[2009\]](#), Hybrid Search [Lin et al. \[2021\]](#), Dense-Sparse Interpolation [Lin and Lin \[2022\]](#), RRF with learned weights [Zhuang and Zuccon \[2022\]](#).

D.4 Graph-Enhanced Retrieval

GraphRAG [Edge et al. \[2024\]](#), HippoRAG [Gutiérrez et al. \[2024\]](#), LightRAG [Guo et al. \[2024b\]](#), Graph RAG Survey [Peng et al. \[2024\]](#), G-Retriever [He et al. \[2024\]](#).

D.5 Recursive/Tree-Based Retrieval

RAPTOR [Sarathi et al. \[2024\]](#), Tree-of-Thoughts [Yao et al. \[2023\]](#), Graph-of-Thoughts [Besta et al. \[2023\]](#).

D.6 Evidence and Attribution

RARR [Gao et al. \[2023\]](#), Verifiability Evaluation [Liu et al. \[2023\]](#), Faithful RAG [Chen et al. \[2024\]](#), Citation Generation [Menick et al. \[2023\]](#), Verification [Welleck et al. \[2022\]](#).

D.7 Local/Private LLMs

Llama 2 [Touvron et al. \[2023\]](#), Llama 3 [Dubey et al. \[2024\]](#), Mistral [Jiang et al. \[2023a\]](#), CodeLlama [Roziere et al. \[2023\]](#), StarCoder [Li et al. \[2023\]](#), DeepSeek-Coder [Guo et al.](#)

[2024a], Phi-3 [Abdin et al. \[2024\]](#), Gemma [Gemma Team \[2024\]](#).

D.8 Vector Databases

FAISS [Johnson et al. \[2019\]](#), Milvus [Milvus \[2024\]](#), Weaviate [Weaviate \[2024\]](#), Qdrant [Qdrant \[2024\]](#), ChromaDB [Chroma \[2024\]](#), pgvector [pgvector \[2024\]](#), Elasticsearch [Elastic \[2024\]](#), OpenSearch [OpenSearch \[2024\]](#), Solr [Apache Software Foundation \[2024\]](#).

D.9 Graph Databases

Neo4j [Webber \[2010\]](#), Kuzu [Kuzu \[2024\]](#), Redwood [Tantivy \[2024\]](#), SQLite [SQLite Consortium \[2024\]](#), RocksDB [Meta Platforms \[2019\]](#).

D.10 Parsing and Embedding

Tree-sitter [Brunsfield \[2024\]](#), ONNX Runtime [ONNX Runtime \[2024\]](#), BGE-M3 [Xiao et al. \[2023\]](#), E5 [Wang et al. \[2024\]](#), Nomic Embed [Nussbaum et al. \[2024\]](#), Jina Embeddings [Jina AI \[2024\]](#).

Bibliography

- Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, and Harkirat Behl. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.
- Apache Software Foundation. Apache solr: Open-source enterprise search platform. <https://solr.apache.org/>, 2024.
- Ananto Nayan Bala. What survives into context: A diagnostic for budget-constrained multi-hop RAG and when submodular evidence packing improves it. *arXiv preprint arXiv:2607.00725*, 2026.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoeffer. Graph of thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2308.09687*, 2023.
- Max Brunsfeld. Tree-sitter: An incremental parsing system for programming tools. <https://tree-sitter.github.io/tree-sitter/>, 2024.
- Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking large language models in retrieval-augmented generation. *Proceedings of AAAI*, 2024.
- Chroma. Chroma: The ai-native open-source embedding database. <https://www.trychroma.com/>, 2024.
- Gordon V Cormack, Charles L A Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of SIGIR*, pages 758–759, 2009.
- Discord. Discord developer documentation: Gateway and gateway events, 2026a. <https://docs.discord.com/developers/events/gateway>. Accessed 2026-07-29.
- Discord. Discord developer documentation: Message resource, 2026b. <https://docs.discord.com/developers/resources/message>. Accessed 2026-07-29.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, and Angela Fan. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Elastic. Elasticsearch: The distributed search and analytics engine. <https://www.elastic.co/elasticsearch>, 2024.

- Stephan Felber, Bernardo Hummes Flores, and Hugo Rincon Galeana. A sheaf-theoretic characterization of tasks in distributed systems. *arXiv preprint arXiv:2503.02556*, 2025.
- Thibault Formal, Benjamin Piwowarski, and Stephane Clinchant. Splade: Sparse lexical and expansion model for first stage ranking. In *Proceedings of SIGIR*, pages 2288–2292, 2021.
- Luyu Gao, Zhuyun Dai, and Jamie Callan. Coil: Revisit exact lexical match in information retrieval with contextualized inverted list. In *Proceedings of NAACL-HLT*, pages 3030–3042, 2021.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Rarr: Researching and revising what language models say, using language models. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 16477–16508, 2023.
- Gemma Team. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- Robert Ghrist and Cassie Ding. Precision-graded cohomology and arithmetic persistence for network sheaves. *arXiv preprint arXiv:2511.00677*, 2025.
- Google. Kythe: A pluggable, (mostly) language-agnostic ecosystem for building tools that work with code. <https://kythe.io/>. Accessed 2026-07-27; no dated publication.
- Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, and Y K Li. Deepseek-coder: When the large language model meets programming. *arXiv preprint arXiv:2401.14196*, 2024a.
- Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. Lightrag: Simple and fast retrieval-augmented generation. *arXiv preprint arXiv:2410.05779*, 2024b.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024.
- Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *arXiv preprint arXiv:2402.07630*, 2024.
- John J Hopfield. Kinetic proofreading: A new mechanism for reducing errors in biosynthetic processes requiring high specificity. *Proceedings of the National Academy of Sciences*, 71(10):4135–4139, 1974.
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*, 2022.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, and Lucile Saulnier. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023a.

- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLM-Lingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, 2023b. doi:[10.18653/v1/2023.emnlp-main.825](https://doi.org/10.18653/v1/2023.emnlp-main.825).
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 1658–1677, 2024. doi:[10.18653/v1/2024.acl-long.91](https://doi.org/10.18653/v1/2024.acl-long.91).
- Jina AI. Jina embeddings: Multilingual embedding models. <https://jina.ai/embeddings/>, 2024.
- Jeff Johnson, Matthijs Douze, and Herve Jegou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- Zhixin Kang, Hui Chen, Siyang Li, Shou-Wen Wang, and Bin Zhou. Evolving strategies for lineage tracing: Genetic markers, synthetic barcodes, and natural variants. *Cell Stem Cell*, 2026.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, pages 6769–6781, 2020.
- Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of SIGIR*, pages 39–48, 2020.
- Kirby, D. and collaborators. Parallel reactions on a single t cell receptor offer a robust kinetic proofreading mechanism. *Proceedings of the National Academy of Sciences*, 2025. <https://www.pnas.org/doi/10.1073/pnas.2514057122>.
- Kuzu. Kuzu: An embeddable property graph database management system. <https://kuzudb.com/>, 2024.
- Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, and Jenny Chim. Starcoder: May the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.
- Hui Lin and Jeff Bilmes. Multi-document summarization via budgeted maximization of submodular functions. In *Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, pages 912–920, 2010.
- Jimmy Lin and Xueguang Ma. A few brief notes on deepimpact, coil, and a conceptual framework for information retrieval techniques. *arXiv preprint arXiv:2106.14807*, 2021.
- Jimmy Lin, Rodrigo Nogueira, and Andrew Yates. Pretrained transformers for text ranking: Bert and beyond. *Synthesis Lectures on Human Language Technologies*, 2021.
- Sheng-Chieh Lin and Jimmy Lin. Densifying sparse representations for passage retrieval by representational slicing. *arXiv preprint arXiv:2112.04666*, 2022.
- Nelson F. Liu, Tianyi Zhang, and Percy Liang. Evaluating verifiability in generative search engines. *arXiv preprint arXiv:2304.09848*, 2023.

- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi:[10.1162/tacl_a_00638](https://doi.org/10.1162/tacl_a_00638).
- Antonio Mallia, Omar Khattab, Torsten Suel, and Nicola Tonellotto. Learning passage impacts for inverted indexes. In *Proceedings of SIGIR*, pages 1723–1727, 2021.
- Mao et al. Multimolecular proofreading overcomes the activity-fidelity trade-off. *Cell Systems*, 2026. [https://www.cell.com/cell-systems/abstract/S2405-4712\(26\)00140-7](https://www.cell.com/cell-systems/abstract/S2405-4712(26)00140-7).
- Ivan Matveev. Token-oriented object notation vs json: A benchmark of plain and constrained decoding generation. *arXiv preprint arXiv:2603.03306*, 2026.
- Jacob Menick, Maja Trebacz, Vladimir Mikulik, John Aslanides, Francis Song, Martin Chadwick, Mia Glaese, Susannah Young, Lucy Campbell-Gillingham, Geoffrey Irving, and Nat McAleese. Teaching language models to support answers with verified quotes. *arXiv preprint arXiv:2203.11147*, 2023.
- Meta Open Source. Indexing code at scale with glean, 2024. Engineering at Meta; <https://engineering.fb.com/2024/12/19/developer-tools/glean-open-source-code-indexing/>. Accessed 2026-07-27.
- Meta Platforms. Rocksdb: A persistent key-value store for fast storage environments. <https://rocksdb.org/>, 2019.
- Milvus. Milvus: An open-source vector database. <https://milvus.io/>, 2024.
- A. Möller, E. E. O. Ishida, J. Peloton, O. Vidal Velázquez, J. Soon, B. Martin, M. Cluver, M. Leoni, and E. Taylor. Real-time active learning for optimised spectroscopic follow-up: Enhancing early sn ia classification with the fink broker. *arXiv preprint arXiv:2502.19555*, 2025.
- Jacques Ninio. Kinetic amplification of enzyme discrimination. *Biochimie*, 57(5):587–595, 1975.
- Alexander H Nitz, Tito Dal Canton, Derek Davis, and Steven Reyes. Pycbc live: Rapid detection of gravitational waves from compact binary mergers. *Physical Review D*, 98(2):024050, 2018.
- Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder. *arXiv preprint arXiv:2402.01613*, 2024.
- ONNX Runtime. Onnx runtime: Cross-platform accelerated machine learning. <https://onnxruntime.ai/>, 2024.
- OpenAI. Openai api model guidance, 2026. <https://developers.openai.com/api/docs/guides/latest-model>. Accessed 2026-07-29.
- OpenSearch. Opensearch: Open-source search and analytics suite. <https://opensearch.org/>, 2024.

- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*, 2024.
- Chao Peng, Bin Wang, Zhilei Long, and Jinfang Sheng. AdaGReS: Adaptive greedy context selection via redundancy-aware scoring for token-budgeted RAG. *arXiv preprint arXiv:2512.25052*, 2025.
- pgvector. pgvector: Open-source vector similarity search for postgres. <https://github.com/pgvector/pgvector>, 2024.
- Qdrant. Qdrant: Vector database for the next generation of ai applications. <https://qdrant.tech/>, 2024.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, and Jeremy Rapin. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubham Khanna, Anna Goldie, and Christopher D. Manning. Raptor: Recursive abstractive processing for tree-organized retrieval. *arXiv preprint arXiv:2401.18059*, 2024.
- M. Schmidt, F. Caccioli, and T. Aste. Spectral coarse-graining and rescaling for preserving structural and dynamical properties in graphs. *Physical Review E*, 112:034303, 2025.
- Aidan Sedgewick, Christa Gall, Luca Izzo, Adriano Agnello, Charlotte R. Angus, Jens Hjorth, and Arthur Kadela. Aas2rto: Automated alert streams to real-time observations—preparing for rapid follow-up of transient objects in the era of lsst. *arXiv preprint arXiv:2501.06968*, 2025.
- Jeffrey Seely. Sheaf cohomology of linear predictive coding networks. *arXiv preprint arXiv:2511.11092*, 2025.
- SQLite Consortium. Sqlite: A small, fast, self-contained sql database engine. <https://www.sqlite.org/>, 2024.
- Tantivy. Tantivy: A full-text search engine library in rust. <https://github.com/quickwit-oss/tantivy>, 2024.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, and Shruti Bhosale. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2024.
- Qingyue Wang, Liang Ding, Yanan Cao, Zhiliang Tian, Shi Wang, Dacheng Tao, and Li Guo. Recursively summarizing enables long-term dialogue memory in large language models. *arXiv preprint arXiv:2308.15022*, 2023.
- Michal Was, Marie-Anne Bizouard, Violette Brisson, Fabien Cavalier, Michel Davier, Patrice Hello, Nicolas Leroy, Florent Robinet, and Miltiadis Vavoulidis. On the background estimation by time slides in a network of gravitational wave detectors. *Classical and Quantum Gravity*, 27(1):015005, 2010.
- Weaviate. Weaviate: The ai-native vector database. <https://weaviate.io/>, 2024.
- Jim Webber. Neo4j: The graph database platform. <https://neo4j.com/>, 2010.
- Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct. *arXiv preprint arXiv:2211.00053*, 2022.
- Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.
- Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. *arXiv preprint arXiv:2007.00808*, 2020.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2023.
- Yuan-Jhe Yin, Bo-Yu Chen, and Berlin Chen. A novel LLM-based two-stage summarization approach for long dialogues. *arXiv preprint arXiv:2410.06520*, 2024.
- Yajie Zhang, Ce Yu, Chao Sun, Jizeng Wei, Junhan Ju, and Shanjiang Tang. Solving online resource-constrained scheduling for follow-up observation in astronomy: A reinforcement learning approach. *arXiv preprint arXiv:2502.11134*, 2025.
- Shengyao Zhuang and Guido Zuccon. Fast passage re-ranking with contextualized exact term matching and efficient passage expansion. *arXiv preprint arXiv:2108.08513*, 2022.
- A. Zwaans, S. Seidel, M. Manceau, and T. Stadler. A bayesian phylodynamic inference framework for single-cell crispr/cas9 lineage tracing barcode data with dependent target sites. *Philosophical Transactions of the Royal Society B*, 380(1919), 2025.